Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 1_COD_SB

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Arjun is working on a project where he needs to analyze a series of integers within a specified range [A, B]. The project involves identifying certain numbers that are not divisible by any smaller positive integers other than 1 and themselves. These numbers are crucial for the next step of his analysis, as they have unique mathematical properties.

Arjun has been asked to implement an efficient way to find all such numbers in a given range. Given that the range can be large, he needs to ensure the solution is optimized.

Can you help him implement this task using the Simple Sieve of Eratosthenes?

Input Format

24uecs2047

The first line of input consists of an integer A, representing the starting range.

The second line of input consists of an integer B, representing the ending range.

Output Format

The output prints a space-separated list of numbers that are not divisible by any smaller positive integers other than 1 and themselves. within the range [A, B].

Refer to the sample output for formatting specifications

Sample Test Case

Input: 2
10
Output: 2 3 5 7

Answer

```
#include<stdio.h>
int s[1000001],i,j,a,b;
int main(){
   scanf("%d%d",&a,&b);
   for(i=2;i<=b;i++)s[i]=1;
   for(i=2;i*i<=b;i++)
   if(s[i])for(j=i*i;j<=b;j+=i)s[j]=0;
   for(i=a;i<=b;i++)
   if(s[i])printf("%d",i);
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Pradeep Narwal is analyzing multiple numerical ranges to identify prime numbers within each range. He is given several ranges defined by a starting number A and an ending number B. Using the Sieve of Eratosthenes, Pradeep wants to efficiently determine all prime numbers that lie within each range.

Write a program to ensure that all prime numbers between A and B (inclusive) are printed for every given range.

Input Format

The first line of input consists of an integer Q, representing the number of ranges.

The next Q lines each contain two integers A and B , representing the start and end of each range.

Output Format

For each range, print all prime numbers between A and B (inclusive) in a single line.

Prime numbers must be printed in ascending order, separated by a single space.

If a range contains no prime numbers, print an empty line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
1 5
10 15
Output: 2 3 5
11 13

Answer

```
#include<stdio.h>
int s[1000001];
int main(){
   int q,a,b,i,j;
   for(i=2;i<=100000;i++)s[i]=1;
   for(i=2;i*i<=100000;i++)
   if(s[i])for(j=i*i;j<=100000;j+=i)s[j]=0;
   scanf("%d",&q);
   while(q--){
```

```
    scanf("%d%d",&a,&b);
    for(i=a;i<=b;i++)if(s[i])printf("%d",i);
  }
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Joanna is developing a utility tool for a math club's quiz event. One of the rounds involves finding the greatest common divisor (GCD) of a list of numbers. Instead of computing it manually for each pair, she wants to automate the task using code.

She decides to use Euclid's Algorithm, a well-known and efficient method to compute the GCD of two numbers. For a list of numbers, she will apply the algorithm iteratively across all elements in the array.

Help Joanna implement a program that takes an array of positive integers and computes the GCD of the entire array using Euclid's Algorithm.

Make sure the program also handles special cases:

If the array is empty, return -1.If the array has only one number, return that number as the GCD.

Example

Input:

3

12 18 24

Output:

6

Explanation:

GCD(12, 18) = 6GCD(6, 24) = 6Final result = 6

Input Format

The first line of input contains an integer n, representing the number of elements in the array.

The second line contains n space-separated positive integers, each representing an element of the array.

Output Format

The output consists of a single line:

- If the array contains two or more elements, print a single integer representing the GCD of all array elements.
- If the array contains only one element, print that element as the GCD.
- If the array is empty, print -1 as a convention to indicate undefined GCD.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2
8 12

Output: 4

Answer

```
#include<stdio.h>
int g(int a,int b){return b?g(b,a%b):a;}
int main(){
   int n,a,r=-1,i;
   scanf("%d",&n);
   if(!n){printf("-1");return 0;}
   for(i=0;i<n;i++){scanf("%d",&a);r=i?g(r,a):a;}
   printf("%d",r);
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Raychel is working on a calendar synchronization tool where she needs to determine how often two repeating tasks will coincide. Each task occurs after a fixed number of days, and she must find the least number of days after which both tasks align again — in other words, the least common multiple (LCM) of the two intervals.

Raychel knows that the most efficient way to compute the LCM is by first calculating the greatest common divisor (GCD) using Euclid's Algorithm, and then using that result to derive the LCM. She also wants to make sure the program works for large inputs without running into overflow errors.

Help Raychel implement a program that calculates the LCM of two non-negative integers by first using Euclid's Algorithm to compute their GCD, and then deriving the LCM from it.

Example

Input:

15 20

Explanation:

GCD(15, 20) = 5LCM = (15 / 5) × 20 = 3 × 20 = 60

Output:

60

Input Format

The input consists of a single line containing two space-separated integers 'a' and 'b', representing the two input numbers.

Output Format

The output consists of a single line:

24uecs2047

- If both 'a' and 'b' are non-zero, print a single integer — the least number that is divisible by both.
- If either 'a' or b is '0', print 0 as the result since no common multiple exists in such cases.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 15 20

Output: 60

Answer

```
#include<stdio.h>
long long g(long long a,long long b)
{return b?g(b,a%b):a;}
int main(){
   long long a,b;
   scanf("%lld%lld",&a,&b);
   if(!a||!b)printf("0");
   else printf("%lld",a/g(a,b)*b);
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement:

A software company is developing a mathematical tool for students to help them understand factorization and modular arithmetic. As part of the tool, they need a function that calculates the product of all positive divisors (factors) of a given integer, with the result taken modulo 1000000007. The tool is intended to assist in solving mathematical problems and providing insights into number theory.

Your task is to implement a function that computes the product of all factors of a given integer n, and then display the result modulo 1000000007.

Examples :

Input: 12

Output: 1728

1 * 2 * 3 * 4 * 6 * 12 = 1728

Input Format

The input consists of a single integer n.

Output Format

The output displays a single integer which is the product of all factors of n modulo 1000000007.

Refer to the sample output for better understanding.

Sample Test Case

Input: 12
Output: 1728

Answer

```
#include<stdio.h>
#define m 1000000007
int main(){
   long long n,p=1;
   scanf("%lld",&n);
   for(long long i=1;i<=n;i++)
   if(n%i==0)p=p*i%m;
   printf("%lld",p);
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 1_COD_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

In the field of cryptography, especially in public-key algorithms like RSA, it is crucial to identify pairs of numbers that are coprime — i.e., they do not share any common divisors other than 1.

Before generating secure keys, cryptographic systems often verify whether two large numbers are coprime. This process uses a classical method known as Euclid's Algorithm, which efficiently determines the greatest common divisor (GCD).

You are given a list of number pairs. Your task is to determine whether each pair of integers is coprime using Euclid's Algorithm. If the GCD of the two numbers is 1, output "Coprime". Otherwise, output "Not Coprime".

Example

24uecs2047

Input:

3

15 28

12 35

8 8

Explanation:

GCD(15, 28) = 1 Coprime

GCD(12, 35) = 1 Coprime

GCD(8, 8) = 8 Not Coprime

Output:

Coprime

Coprime

Not Coprime

Input Format

- The first line contains an integer 't' — the number of pairs to check.
- Each of the next 't' lines contains two space-separated integers 'a' and 'b'.

Output Format

- For each pair, output a single line:
- "Coprime" if the two numbers are coprime
- "Not Coprime" otherwise

- Each output must appear on a new line, in the same order as the input pairs.

24uecs2047

Refer to the sample output for formatting specifictions.

Sample Test Case

Input: 1
17 29

Output: Coprime

Answer

```
// You are using GCC
#include <stdio.h>

// Function to calculate GCD using Euclid's Algorithm
long long findGCD(long long a, long long b) {
    while (b != 0) {
        long long temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main() {
    int t;
    // Read the number of test cases
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        long long a, b;
        // Read the pair of integers
        scanf("%lld %lld", &a, &b);

        // Check if GCD is 1
        if (findGCD(a, b) == 1) {
            printf("Coprime\n");
        } else {
            printf("Not Coprime\n");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Sai Kishore is working on a tool that identifies special numbers in a range. These numbers must have only two divisors: 1 and themselves. His task is to count how many of these special numbers exist below a given threshold N. Since N can be as large as one million, Sai Kishore needs an efficient way to find these numbers in a given range.

Sai Kishore decides to use a technique called Sieve of Eratosthenes, which is well-suited for efficiently identifying these special numbers.

Can you help him implement a program that counts how many such numbers exist below N?

Example:

Input:

10

Output:

4

Explanation:

The numbers less than 10 that have only two divisors (1 and themselves) are: 2, 3, 5, and 7. Thus, the output is 4.

Input Format

The input consists of a single integer N, representing the upper bound for the search.

Output Format

The output should be a single integer, representing the count of special numbers less than N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

Output: 4

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    // We need to find primes strictly LESS than N
    if (n <= 2) {
        printf("0\n");
        return 0;
    }

    // Create a boolean array to mark primality
    // Using a static array or dynamic allocation for 1,000,000
    bool is_prime[n];
    memset(is_prime, true, sizeof(is_prime));

    is_prime[0] = is_prime[1] = false;

    for (int p = 2; p * p < n; p++) {
        // If is_prime[p] is not changed, then it is a prime
        if (is_prime[p] == true) {
            // Update all multiples of p starting from p*p
            for (int i = p * p; i < n; i += p)
                is_prime[i] = false;
        }
    }

    // Count the primes found
    int count = 0;
    for (int i = 2; i < n; i++) {
```

```
        if (is_prime[i]) {
            count++;
        }
    }

    printf("%d\n", count);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Arjun is analyzing a numerical range to find special pairs of prime numbers. He is given a range defined by two integers L and R, and a target value T. Within the range [L,R], Arjun wants to identify all pairs of prime numbers whose sum equals T.

Write a program to ensure that all such valid prime pairs are printed in ascending order.

Input Format

The first line contains three space-separated integers:

- L starting value of the range
- R ending value of the range
- T target sum

Output Format

The output prints each valid pair of prime numbers whose sum equals T, one pair per line.

Each line should contain two space-separated integers representing a valid prime pair.

If no such pairs exist, no output should be printed.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 1 40 10

Output: 3 7
5 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

#define MAX 1000001

// Global array to handle large memory allocation outside the stack
bool is_prime[MAX];

void sieve() {
    for (int i = 0; i < MAX; i++) is_prime[i] = true;
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; p * p < MAX; p++) {
        if (is_prime[p]) {
            for (int i = p * p; i < MAX; i += p)
                is_prime[i] = false;
        }
    }
}

int main() {
    int L, R, T;
    if (scanf("%d %d %d", &L, &R, &T) != 3) return 0;

    sieve();

    // Iterate through the range [L, R]
    for (int p1 = L; p1 <= R; p1++) {
        // Check if the first number is prime
        if (is_prime[p1]) {
            int p2 = T - p1;
```

```
        // Condition 1: p2 must be within the range [L, R]
        // Condition 2: p2 must be prime
        // Condition 3: p1 <= p2 to maintain ascending order and avoid double
printing
        if (p2 >= L && p2 <= R && p2 >= p1 && is_prime[p2]) {
            printf("%d %d\n", p1, p2);
        }
    }
  }

  return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Bella is developing a number puzzle game where each level involves simplifying equations using number theory. In one level, she comes across two large positive integers and is asked to reduce them using their greatest common divisor (GCD).

However, the puzzle has an extra twist:

She must express the GCD as a linear combination of the two numbers – that is, she needs to find two integers x and y such that: GCD(a, b) = a * x + b * y

Help Bella write a program that, given two integers a and b, finds:

The GCD of a and bThe coefficients 'x' and y" that satisfy the linear combination condition

Example

Input

48 18

Output:

6 -1 3

Explanation:

The GCD of 48 and 18 is 6.

Using the Extended Euclidean Algorithm, we can find integers x = -1 and y = 3 such that:

48 × (-1) + 18 × 3 = -48 + 54 = 6

This satisfies Bézout's Identity: GCD(48, 18) = 48x + 18y = 6.

Input Format

- The input consists of a single line with two space-separated integers 'a' and 'b'

Output Format

The output prints three space-separated integers in a single line:

- The first integer is the GCD of a and b.
- The second integer is the coefficient x such that a * x + b * y = GCD.
- The third integer is the coefficient y such that a * x + b * y = GCD.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 45 21
Output: 3 1 -2

Answer

```
// You are using GCC
#include <stdio.h>

/**
 * Extended Euclidean Algorithm
 * Returns the GCD and updates x and y via pointers
 */
long long extendedGCD(long long a, long long b, long long *x, long long *y) {
    // Base Case
    if (a == 0) {
        *x = 0;
```

```
    *y = 1;
    return b;
  }

  long long x1, y1; // To store results of recursive call
  long long gcd = extendedGCD(b % a, a, &x1, &y1);

  // Update x and y using results of recursive call
  *x = y1 - (b / a) * x1;
  *y = x1;

  return gcd;
}

int main() {
  long long a, b;
  if (scanf("%lld %lld", &a, &b) != 2) return 0;

  long long x, y;

  // We call it as (a, b) but the logic inside handles (b%a, a)
  // To match the output format of sample tests, we ensure consistent ordering
  long long gcd = extendedGCD(a, b, &x, &y);

  printf("%lld %lld %lld\n", gcd, x, y);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 2_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neha is organizing a sequence of integers where some elements are zeroes. She wants to rearrange the sequence so that all zeroes are moved to the end while keeping the relative order of the non-zero elements intact. Help Neha perform this task efficiently using the two-pointer technique to modify the sequence in place.

Input Format

The first line of input consists of an integer n, representing the size of the array nums[].

The second line contains n integers, representing the elements of the array nums[].

Output Format

24uecs2047

The output prints the modified array nums[], with all zeroes moved to the end and the relative order of the non-zero elements maintained.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
0 1 0 3 12

Output: 1 3 12 0 0

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int pos = 0;
    for (int i = 0; i < n; i++) {
        if (nums[i] != 0) {
            int temp = nums[pos];
            nums[pos] = nums[i];
            nums[i] = temp;
            pos++;
        }
    }

    for (int i = 0; i < n; i++) {
        printf("%d%s", nums[i], (i == n - 1 ? "" : " "));
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Arjun has a list of integers representing a specific sequence, and he wants to find its next permutation in lexicographical order. If the sequence is the largest permutation possible, he should rearrange it into the smallest permutation. Help Arjun solve this problem efficiently using the two-pointer technique to rearrange the sequence in place with constant extra memory.

Input Format

The first line of input consists of an integer n, representing the size of the array nums[].

The second line contains n integers, representing the elements of the array nums[].

Output Format

The output prints "Next permutation:" followed by n space-separated integers on the next line, representing the next lexicographically greater permutation of the given sequence.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: Next permutation:
1 3 2

Answer

```
// You are using GCC
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
```

```
    *b = temp;
}

void reverse(int nums[], int start, int end) {
    while (start < end) {
        swap(&nums[start], &nums[end]);
        start++;
        end--;
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int i = n - 2;
    // Step 1: Find the first decreasing element from the right
    while (i >= 0 && nums[i] >= nums[i + 1]) {
        i--;
    }

    if (i >= 0) {
        // Step 2: Find the element to swap with
        int j = n - 1;
        while (nums[j] <= nums[i]) {
            j--;
        }
        // Step 3: Swap pivot with successor
        swap(&nums[i], &nums[j]);
    }

    // Step 4: Reverse the suffix
    reverse(nums, i + 1, n - 1);

    printf("Next permutation:\n");
    for (int k = 0; k < n; k++) {
        printf("%d%s", nums[k], (k == n - 1 ? "" : " "));
```

```
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Priya is searching for a specific word (needle) in a large text (haystack). She needs to find the index of the first occurrence of the word in the text. If the word is not present, she should return -1. Help Priya solve this efficiently using the two-pointer technique.

Input Format

The first line of input consists of a string haystack, representing the text in which the word is to be searched.

The second line of input consists of a string needle, representing the word to search for in the text.

Output Format

The output prints "The index of the first occurrence of needle in haystack is: X", where X represents the index value.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: sadbutsad
sad
Output: The index of the first occurrence of needle in haystack is: 0

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
int main() {
  char haystack[101], needle[101];

  if (scanf("%s %s", haystack, needle) < 2) {
    return 0;
  }

  int h_len = strlen(haystack);
  int n_len = strlen(needle);
  int result = -1;

  for (int i = 0; i <= h_len - n_len; i++) {
    int j;
    for (j = 0; j < n_len; j++) {
      if (haystack[i + j] != needle[j]) {
        break;
      }
    }
    if (j == n_len) {
      result = i;
      break;
    }
  }

  printf("The index of the first occurrence of needle in haystack is: %d", result);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ramesh is working with an array of integers and needs to remove all occurrences of a specific value from it.

To accomplish this efficiently, he uses the Two-Pointer technique, where one pointer scans the array and another pointer tracks the position to place valid elements. Given an integer array nums and an integer val, Ramesh must modify the array in-place so that all elements equal to val are removed while preserving the order of the remaining elements.

Write a program to ensure that the number of remaining elements and the modified array are printed correctly.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

The third line contains an integer val, representing the value to be removed from the array.

Output Format

The first line prints "Number of elements after removal: " followed by count of elements not equal to val.

The second line prints "Modified array:"

Then print the modified array elements (up to index k) separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 2 2 3
3
Output: Number of elements after removal: 2
Modified array:
2 2

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
```

```
    int n, val;
    if (scanf("%d", &n) != 1) return 0;

    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    if (scanf("%d", &val) != 1) return 0;

    int k = 0;
    for (int i = 0; i < n; i++) {
        if (nums[i] != val) {
            nums[k] = nums[i];
            k++;
        }
    }

    printf("Number of elements after removal: %d\n", k);
    printf("Modified array:\n");
    for (int i = 0; i < k; i++) {
        printf("%d%s", nums[i], (i == k - 1 ? "" : " "));
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Anil is working with a sorted array of integers and wants to remove duplicate elements efficiently. To solve this problem, he uses the Two-Pointer technique, where one pointer scans the array and another pointer keeps track of the position to store unique elements. Given a sorted integer array nums, Anil must modify the array in-place so that each unique element appears only once.

Write a program to ensure that the number of unique elements and the modified array are printed correctly.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the sorted array.

Output Format

The first line prints "Number of unique elements: k"

On the next line, print "Modified array:"

Then, print the first k elements of the modified array, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8
-1 -1 0 0 1 1 2 2

Output: Number of unique elements: 4
Modified array:
-1 0 1 2

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    if (n == 0) {
        printf("Number of unique elements: 0\nModified array:\n");
        return 0;
    }

    int nums[n];
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int k = 1;
    for (int i = 1; i < n; i++) {
        if (nums[i] != nums[k - 1]) {
            nums[k] = nums[i];
            k++;
        }
    }

    printf("Number of unique elements: %d\n", k);
    printf("Modified array:\n");
    for (int i = 0; i < k; i++) {
        printf("%d%s", nums[i], (i == k - 1 ? "" : " "));
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Karthik is organizing marbles of three different colors: red, white, and blue. Each color is represented by an integer—0 for red, 1 for white, and 2 for blue. Karthik wants to sort the marbles so that all marbles of the same color are adjacent, and they are arranged in the order red, white, and blue. Help Karthik solve this problem efficiently using the two-pointer technique.

Input Format

The first line of input consists of an integer n, representing the number of marbles in the array nums[].

The second line contains n integers, representing the marbles' colors, where each integer is either 0, 1, or 2.

Output Format

24uecs2047

The output prints the sorted array, where all marbles are grouped by color in the order red (0), white (1), and blue (2).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 0 2 1 1 0

Output: 0 0 1 1 2 2

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int low = 0;
    int mid = 0;
    int high = n - 1;

    while (mid <= high) {
        if (nums[mid] == 0) {
            int temp = nums[low];
            nums[low] = nums[mid];
            nums[mid] = temp;
            low++;
            mid++;
        } else if (nums[mid] == 1) {
            mid++;
        } else {
            int temp = nums[mid];
            nums[mid] = nums[high];
```

```
            nums[high] = temp;
            high--;
        }
    }

    for (int i = 0; i < n; i++) {
        printf("%d", nums[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Kavya is an avid reader who loves finding hidden patterns in texts. One day, she stumbled upon a string of characters and wondered about the longest palindromic substring it contains. Help Kavya identify this substring using an efficient algorithm based on the two-pointer technique.

Input Format

The input consists of a single string s.

Output Format

The output displays "Longest Palindromic Substring: " followed by the longest palindrome found.

Refer the sample output for format specifications.

Sample Test Case

Input: babad

Output: Longest Palindromic Substring: bab

Answer

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[21]; // max length 20 + 1 for null character
    scanf("%s", s);

    int n = strlen(s);
    int start = 0, maxLen = 1;

    // Helper function: expand around center
    for (int i = 0; i < n; i++) {
        // Odd length palindrome
        int l = i, r = i;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--;
            r++;
        }

        // Even length palindrome
        l = i; r = i + 1;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--;
            r++;
        }
    }

    printf("Longest Palindromic Substring: ");
    for (int i = start; i < start + maxLen; i++)
        printf("%c", s[i]);
    printf("\n");

    return 0;
```

}

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rohit is analyzing an array of integers to find special combinations of numbers. Using the Two-Pointer technique, he wants to identify all unique triplets in the array whose sum is equal to 0. Given an integer array, Rohit must find and print all such triplets, ensuring that no duplicate triplets are included in the result.

Write a program to ensure that all valid triplets with sum zero are printed correctly.

Example 1

Input:

6

-1 0 1 2 -1 -4

Output:

-1 -1 2

-1 0 1

Explanation:

nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.

nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.

nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.

The distinct triplets are [-1,0,1] and [-1,-1,2].

Notice that the order of the output and the order of the triplets does not matter.

Example 2

Input:

3

0 1 1

Output:

No triplets found with sum 0.

Explanation:

The only possible triplet does not sum up to 0.

Example 3

Input:

3

0 0 0

Output:

0 0 0

Explanation:

The only possible triplet sums up to 0.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints each unique triplet whose sum is 0 on a separate line.

Each triplet must be printed in ascending order (as they appear after sorting).

If no such triplets exist, print "No triplets found with sum 0."

24uecs2047

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
-1 0 1 2 -1 -4

Output: -1 -1 2
-1 0 1

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort
int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    // Sort the array
    qsort(nums, n, sizeof(int), cmp);

    int found = 0;

    for (int i = 0; i < n - 2; i++) {
        // Skip duplicates for i
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1;
        int right = n - 1;

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
```

```
            printf("%d %d %d\n", nums[i], nums[left], nums[right]);
            found = 1;

            // Skip duplicates for left and right
            int tempLeft = nums[left], tempRight = nums[right];
            while (left < right && nums[left] == tempLeft) left++;
            while (left < right && nums[right] == tempRight) right--;
        } else if (sum < 0) {
            left++;
        } else {
            right--;
        }
    }
  }

  if (!found)
    printf("No triplets found with sum 0.\n");

  return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Rahul is given an array of integers and a target value T. Using the Two-Pointer technique, he wants to find two distinct elements in the array such that the sum of the pair is closest to the target value. To apply this technique efficiently, the array must first be sorted.

Write a program to ensure that the pair of numbers whose sum is closest to T is identified and printed correctly.

Input Format

The first line contains two space-separated integers n and T, where:

- n is the number of elements in the array
- T is the target sum

The second line contains n space-separated integers, representing the elements

of the array.

Output Format

The output prints two space separated integers representing the pair whose sum is closest to the target.

The output pair must be printed in ascending order.

If multiple pairs have the same closest sum, print the first pair found by the two-pointer traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 50
10 22 28 29 30 40

Output: 10 40

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Comparison function for qsort
int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n, T;
    scanf("%d %d", &n, &T);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    // Sort the array
    qsort(arr, n, sizeof(int), cmp);
```

```
    int left = 0, right = n - 1;
    int closestSum = arr[0] + arr[1];
    int pair1 = arr[0], pair2 = arr[1];

    while (left < right) {
        int sum = arr[left] + arr[right];

        // Update closest sum if needed
        if (abs(sum - T) < abs(closestSum - T)) {
            closestSum = sum;
            pair1 = arr[left];
            pair2 = arr[right];
        }

        if (sum < T)
            left++;
        else
            right--;
    }

    // Ensure ascending order in output
    if (pair1 > pair2) {
        int temp = pair1;
        pair1 = pair2;
        pair2 = temp;
    }

    printf("%d %d\n", pair1, pair2);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 3_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. The main idea behind the Knuth-Morris-Pratt algorithm is to:

Answer

Avoid redundant checking of characters that have already been matched

Status : Correct ***Marks : 1/1***

2. The Boyer-Moore algorithm uses which two rules to determine the shift amount?

Answer

Bad character rule and good prefix rule

Status : Wrong ***Marks : 0/1***

24uecs2047

3. The Boyer-Moore algorithm is particularly efficient when

Answer

The pattern contains repetitive elements

Status : Correct ***Marks : 1/1***

4. In the KMP algorithm, if the prefix table at position ii has a value of kk, what does it imply?

Answer

The length of the longest proper prefix which is also a suffix up to position ii is kk.

Status : Correct ***Marks : 1/1***

5. The concept of prefix and suffix is used in which of the following algorithms?

Answer

Knuth-Morris-Pratt (KMP) algorithm

Status : Correct ***Marks : 1/1***

6. What is the main purpose of Manacher's Algorithm?

Answer

To find the longest palindromic substring in linear time

Status : Correct ***Marks : 1/1***

7. In the Z-algorithm, the Z-array is used to store:

Answer

The length of the longest substring starting at each position that matches the prefix

Status : Correct ***Marks : 1/1***

24uecs2047

8. Which application is most efficiently solved using Z-algorithm?

Answer

Finding all occurrences of a pattern in a text

Status : Correct ***Marks : 1/1***

9. In Z-algorithm, when Z[k] is greater than 0, what does it imply?

Answer

Substring starting at k matches the prefix for length Z[k]

Status : Correct ***Marks : 1/1***

10. Which of the following best explains why Manacher's algorithm is preferable for palindrome problems over expand-around-center approaches?

Answer

Avoids recomputation of overlapping palindromes by mirroring results around the center

Status : Correct ***Marks : 1/1***

11. In bioinformatics applications, KMP is often used instead of Boyer-Moore. Why?

Answer

DNA has only 4 characters, reducing Boyer-Moore's skipping advantage

Status : Correct ***Marks : 1/1***

12. In KMP, the LPS array for pattern "ABABAC" is:

Answer

[0,0,1,2,3,0]

Status : Correct ***Marks : 1/1***

13. Which is true about Manacher's algorithm with respect to handling even-length palindromes?

Answer

Uses a modified string with separators between characters

Status : Correct ***Marks : 1/1***

14. How is Z-algorithm used for pattern search in text efficiently?

Answer

Concatenate P + "$" + T and compute Z-array

Status : Correct ***Marks : 1/1***

15. Which of the following strings has the largest number of palindromic substrings using Manacher’s algorithm?

Answer

"ababa"

Status : Wrong ***Marks : 0/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 3_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Anil is working on a project that involves counting occurrences of a specific pattern within a given text. He needs a program that efficiently finds and counts all occurrences of a pattern within a given text using the Knuth-Morris-Pratt (KMP) algorithm.

Your task is to help Anil by developing a program that takes a text and a pattern as input and counts the number of times the pattern appears in the text using the KMP algorithm.

Input Format

The first line contains a string txt, representing the text in which Anil wants to search for the pattern.

24uecs2047

The second line contains a string pat, representing the pattern Anil wants to search for in the text.

Output Format

The output displays a single integer, representing the count of occurrences of the pattern pat in the text txt.

If the pattern is not found, nothing will be printed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: AABAACAADAABAABA
AABA

Output: 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

// Function to compute the LPS (Longest Prefix Suffix) array
void computeLPSArray(char* pat, int M, int* lps) {
    int len = 0; // Length of the previous longest prefix suffix
    lps[0] = 0;  // lps[0] is always 0
    int i = 1;

    while (i < M) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
```

```
        }
    }
}

// Function to count occurrences using KMP algorithm
void KMPSearch(char* pat, char* txt) {
    int M = strlen(pat);
    int N = strlen(txt);

    // Create lps[] that will hold the longest prefix suffix values for pattern
    int lps[M];
    computeLPSArray(pat, M, lps);

    int i = 0; // index for txt[]
    int j = 0; // index for pat[]
    int count = 0;

    while (i < N) {
        if (pat[j] == txt[i]) {
            j++;
            i++;
        }

        if (j == M) {
            count++;
            j = lps[j - 1]; // Reset j to continue searching for overlaps
        } else if (i < N && pat[j] != txt[i]) {
            // Mismatch after j matches
            if (j != 0)
                j = lps[j - 1];
            else
                i = i + 1;
        }
    }

    // Output the result only if count > 0 as per requirements
    if (count > 0) {
        printf("%d\n", count);
    }
}

int main() {
```

```
    char txt[51];
    char pat[26];

    // Read input strings, handling potential newline characters
    if (scanf("%50[^\n]%*c", txt) == 0) return 0;
    if (scanf("%25[^\n]%*c", pat) == 0) return 0;

    KMPSearch(pat, txt);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Rohan is developing a text analysis tool to efficiently find repeated patterns inside large documents. To speed up the searching process, he uses the Boyer–Moore string matching algorithm, specifically relying on the Bad Character Heuristic as the main data structure/technique.

Given a text string and a pattern string, write a program to ensure that the task is done by counting how many times the pattern occurs in the text.

Input Format

The first line contains a string representing the text (may contain spaces and special characters).

The second line contains a string representing the pattern to be searched.

Output Format

The output prints a single integer representing the number of occurrences of the pattern in the text.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: abababababa
ab
Output: 4

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <limits.h>

#define NO_OF_CHARS 256

// Preprocessing function for the Bad Character Heuristic
void badCharHeuristic(char *str, int size, int badchar[NO_OF_CHARS]) {
    // Initialize all occurrences as -1
    for (int i = 0; i < NO_OF_CHARS; i++) {
        badchar[i] = -1;
    }

    // Fill the actual value of last occurrence of a character
    for (int i = 0; i < size; i++) {
        badchar[(int)str[i]] = i;
    }
}

// Function to count occurrences using Boyer-Moore (Bad Character)
void search(char *txt, char *pat) {
    int m = strlen(pat);
    int n = strlen(txt);

    int badchar[NO_OF_CHARS];

    // Preprocess the pattern
    badCharHeuristic(pat, m, badchar);

    int s = 0; // s is shift of the pattern with respect to text
    int count = 0;

    while (s <= (n - m)) {
        int j = m - 1;

        // Keep reducing index j of pattern while characters are matching
```

```
        while (j >= 0 && pat[j] == txt[s + j]) {
            j--;
        }

        // If the pattern is present at current shift, then index j will become -1
        if (j < 0) {
            count++;

            /* Shift the pattern so that the next character in text aligns
               with its last occurrence in pattern.
               The condition (s+m < n) checks if we are at the end of the text. */
            s += (s + m < n) ? m - badchar[(int)txt[s + m]] : 1;
        } else {
            /* Shift the pattern so that the bad character in text aligns
               with its last occurrence in pattern.
               We use max function to ensure we always get a positive shift. */
            int shift = j - badchar[(int)txt[s + j]];
            s += (1 > shift) ? 1 : shift;
        }
    }

    printf("%d\n", count);
}

int main() {
    char txt[101];
    char pat[101];

    // Using %[^\n] to read strings including spaces
    if (scanf("%[^\n]%*c", txt) == 0) return 0;
    if (scanf("%[^\n]%*c", pat) == 0) return 0;

    search(txt, pat);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Kiran is working on a string analysis module where he needs to identify repeating patterns efficiently. To achieve this, he uses the LPS (Longest Prefix Suffix) array from the KMP string matching algorithm as the main data structure.

Given a string, write a program to ensure that the task is done by finding the length of the longest proper prefix that is also a suffix, ensuring the prefix and suffix do not overlap.

Example

Input:

aabcdaabc

Output:

4

Explanation:

The string "aabc" is the longest prefix which is also a suffix. So the length of the string "aabc" is printed as 4.

Input Format

The input consists of a string S, representing the input string.

Output Format

The output prints an integer representing the length of the longest proper prefix which is also a suffix.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabcdaabc
Output: 4

Answer

24uecs2047

```c
// You are using GCC
#include <stdio.h>
#include <string.h>

int main() {
    char s[101];
    if (scanf("%s", s) != 1) return 0;

    int n = strlen(s);
    int lps[n];
    lps[0] = 0; // LPS of a single character is always 0

    // Standard KMP LPS array construction
    int len = 0;
    int i = 1;
    while (i < n) {
        if (s[i] == s[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }

    // The result is the value for the full string length
    int res = lps[n - 1];

    // Constraint: Ensure prefix and suffix do not overlap
    // If res > n/2, we move to the next smaller prefix-suffix
    while (res > n / 2) {
        res = lps[res - 1];
    }

    printf("%d\n", res);

    return 0;
```

}

Status : Correct ***Marks : 10/10***

4. Problem Statement

Liam is a software engineer who loves analyzing strings for patterns. One day, while reviewing usernames in a system, he noticed that some usernames read the same backward and forward (palindromes). He wants to count all palindromic sequences of characters in a username efficiently.

To accomplish this, Liam decides to use Manacher's Algorithm, which finds all palindromic substrings in linear time.

Given a string representing the username, help Liam count all the possible palindromic substrings using Manacher's Algorithm.

Input Format

The input consists of a single line containing the string S.

The string consists of lowercase English letters only.

Output Format

The output prints a single integer, representing the total number of palindromic substrings present in the username.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: abaa
Output: 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
void countPalindromes(char *s) {
    int n = strlen(s);
    if (n == 0) return;

    // Preprocess string: "aba" -> "#a#b#a#"
    // Length becomes 2*n + 1
    int new_len = 2 * n + 1;
    char t[205]; // Sufficient for constraint length 100
    int p[205] = {0};

    for (int i = 0; i < n; i++) {
        t[2 * i] = '#';
        t[2 * i + 1] = s[i];
    }
    t[2 * n] = '#';
    t[new_len] = '\0';

    int center = 0, right = 0;
    long long total_count = 0;

    for (int i = 0; i < new_len; i++) {
        // Find the mirror of i with respect to center
        int mirror = 2 * center - i;

        // If i is within the right boundary, take the minimum
        if (i < right) {
            p[i] = (right - i < p[mirror]) ? right - i : p[mirror];
        }

        // Attempt to expand around i
        while (i + p[i] + 1 < new_len && i - p[i] - 1 >= 0 &&
               t[i + p[i] + 1] == t[i - p[i] - 1]) {
            p[i]++;
        }

        // Update center and right boundary if we expanded past 'right'
        if (i + p[i] > right) {
            center = i;
            right = i + p[i];
        }
```

```
        // In the transformed string, p[i] / 2 (rounded up)
        // gives the count of palindromes centered here
        total_count += (p[i] + 1) / 2;
    }

    printf("%lld\n", total_count);
}

int main() {
    char s[101];
    if (scanf("%s", s) == 1) {
        countPalindromes(s);
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Aryan is working on a text processing project where he needs to analyze strings to find patterns efficiently. He decides to use the Z-algorithm, which for each position in a string, computes the length of the longest substring starting from that position which is also a prefix of the string.

Your task is to implement a program that computes the Z-array for a given string.

Input Format

The input consists of a single line containing the string S.

S may contain only uppercase and lowercase English letters.

Output Format

The output prints the Z-array of S as a sequence of integers separated by spaces.

The i-th element of the Z-array represents the length of the longest substring starting at position i which matches the prefix of S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabxaab

Output: 0 1 0 0 3 1 0

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

void computeZArray(char *S) {
    int n = strlen(S);
    int Z[n];

    // Z[0] is usually defined as 0 or the length of the string.
    // Based on sample cases, we use 0.
    Z[0] = 0;

    int L = 0, R = 0;

    for (int i = 1; i < n; i++) {
        if (i > R) {
            // Case 1: i is outside the current Z-box
            L = R = i;
            while (R < n && S[R] == S[R - L]) {
                R++;
            }
            Z[i] = R - L;
            R--;
        } else {
            // Case 2: i is inside the current Z-box [L, R]
            int k = i - L;

            // If the value at Z[k] doesn't stretch past R
            if (Z[k] < R - i + 1) {
                Z[i] = Z[k];
            } else {
                // Otherwise, we must manually compare characters
```

```
            L = i;
            while (R < n && S[R] == S[R - L]) {
                R++;
            }
            Z[i] = R - L;
            R--;
        }
    }
}

// Print the Z-array
for (int i = 0; i < n; i++) {
    printf("%d ", Z[i]);
}
printf("\n");
}

int main() {
    char S[101];
    if (scanf("%s", S) == 1) {
        computeZArray(S);
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 3_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Dhiya is a data analyst working for a data compression company. One of her tasks is to analyze logs that are compressed using a simple format where each character is followed by the number of times it appears. For example, "A3B2" represents the string "AAABB".

Her goal is to identify whether a specific pattern exists in the original, uncompressed message and, if so, determine all starting and ending indices of the pattern's occurrence in the expanded string. To perform this efficiently, Dhiya decides to first expand the compressed string and then apply pattern matching.

Your task is to help Dhiya by building a program using Boyer-Moore pattern matching algorithm that performs this operation.

24uecs2047

Input Format

The first line of input contains a string s consisting of characters followed by digits representing the compressed text.

The second line of input contains a string p representing the pattern string.

Output Format

The first line of output prints "Expanded Text: X" where X represents the expanded text.

The second line of output prints "Matches found (start_index end_index):" if match found.

For each match of the pattern in the expanded text, print the starting and ending indices (0-based) separated by a space (Print one pair per line).

If no match is found, print "No match found.".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: A3B2C4D1
CCC
Output: Expanded Text: AAABBCCCCD
Matches found (start_index end_index):
5 7
6 8

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

#define MAX_EXP 10000
#define NO_OF_CHARS 256
```

```
// Function to expand the compressed string (e.g., A3B2 -> AAABB)
void expandString(char* s, char* expanded) {
    int len = strlen(s);
    int k = 0;
    for (int i = 0; i < len; ) {
        char ch = s[i++];
        int freq = 0;
        // Handle multi-digit numbers
        while (i < len && isdigit(s[i])) {
            freq = freq * 10 + (s[i] - '0');
            i++;
        }
        for (int j = 0; j < freq; j++) {
            expanded[k++] = ch;
        }
    }
    expanded[k] = '\0';
}

// Preprocessing for Boyer-Moore Bad Character Heuristic
void badCharHeuristic(char* str, int size, int badchar[NO_OF_CHARS]) {
    for (int i = 0; i < NO_OF_CHARS; i++)
        badchar[i] = -1;
    for (int i = 0; i < size; i++)
        badchar[(int)str[i]] = i;
}

// Boyer-Moore Search function
void boyerMooreSearch(char* txt, char* pat) {
    int m = strlen(pat);
    int n = strlen(txt);
    int badchar[NO_OF_CHARS];
    int found = 0;

    badCharHeuristic(pat, m, badchar);

    int s = 0; // s is shift of the pattern with respect to text
    while (s <= (n - m)) {
        int j = m - 1;

        while (j >= 0 && pat[j] == txt[s + j])
            j--;
```

```
        if (j < 0) {
            if (found == 0) {
                printf("Matches found (start_index end_index):\n");
                found = 1;
            }
            // Print start and end index (inclusive)
            printf("%d %d\n", s, s + m - 1);

            // Shift pattern so that the next character in text aligns
            // with its last occurrence in pattern
            s += (s + m < n) ? m - badchar[(int)txt[s + m]] : 1;
        } else {
            int shift = j - badchar[(int)txt[s + j]];
            s += (1 > shift) ? 1 : shift;
        }
    }

    if (!found) {
        printf("No match found.\n");
    }
}

int main() {
    char s[1001], p[1001];
    char expanded[MAX_EXP] = "";

    // Read compressed text and pattern
    if (scanf("%s", s) != 1) return 0;
    if (scanf("%s", p) != 1) return 0;

    expandString(s, expanded);
    printf("Expanded Text: %s\n", expanded);

    boyerMooreSearch(expanded, p);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Sanya is developing a text analysis tool and wants to identify the longest palindromic sequence in a given string. A palindrome is a string that reads the same forwards and backwards.

Your task is to write a program that takes a string as input and returns the longest palindromic substring it contains. If there are multiple substrings of the same maximum length, returning any one of them is acceptable.

Input Format

The input consists of a single line containing the string S.

S may consist of uppercase and lowercase English letters.

Output Format

The output prints a single string representing the longest palindromic substring found in S.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: babad
Output: bab

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

// Function to expand around the center and return the length of the palindrome
int expandAroundCenter(char *s, int left, int right, int n) {
   while (left >= 0 && right < n && s[left] == s[right]) {
      left--;
      right++;
   }
   // Return length of the palindrome found
```

```
    // Formula: (right - 1) - (left + 1) + 1 = right - left - 1
    return right - left - 1;
}

int main() {
    char s[101];
    if (scanf("%s", s) != 1) return 0;

    int n = strlen(s);
    if (n < 1) return 0;

    int start = 0, maxLength = 1;

    for (int i = 0; i < n; i++) {
        // Case 1: Odd length (center is s[i])
        int len1 = expandAroundCenter(s, i, i, n);
        // Case 2: Even length (center is between s[i] and s[i+1])
        int len2 = expandAroundCenter(s, i, i + 1, n);

        // Pick the larger of the two
        int len = (len1 > len2) ? len1 : len2;

        // Update global maximum if a longer palindrome is found
        if (len > maxLength) {
            maxLength = len;
            // Calculate starting index of the palindrome
            start = i - (len - 1) / 2;
        }
    }

    // Print the longest palindromic substring
    for (int i = start; i < start + maxLength; i++) {
        printf("%c", s[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rita is working on string pattern matching using the Z-Algorithm. She wants to find all starting indices where a given pattern string occurs in a text string efficiently.

Write a program that takes a text string and a pattern string as input and outputs a list of indices (0-based) where the pattern occurs. If the pattern does not occur in the text, output an empty list.

Input Format

The first line contains the text string txt.

The second line contains the pattern string pat.

Output Format

The output prints a list of integers representing the starting indices of all occurrences of the pattern in the text.

If no match is found, print [].

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abesdu
edu
Output: []

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

void computeZ(char* concat, int n, int* Z) {
    int L = 0, R = 0;
    for (int i = 1; i < n; i++) {
        if (i > R) {
```

24uecs2047

```
            L = R = i;
            while (R < n && concat[R] == concat[R - L])
                R++;
            Z[i] = R - L;
            R--;
        } else {
            int k = i - L;
            if (Z[k] < R - i + 1) {
                Z[i] = Z[k];
            } else {
                L = i;
                while (R < n && concat[R] == concat[R - L])
                    R++;
                Z[i] = R - L;
                R--;
            }
        }
    }
}

int main() {
    char txt[1001], pat[101];
    // Use %[^\n] to handle potential spaces in text
    if (scanf("%[^\n]%*c", txt) == 0) return 0;
    if (scanf("%[^\n]%*c", pat) == 0) return 0;

    int M = strlen(pat);
    int N = strlen(txt);

    // Create concatenated string: pat + $ + txt
    char concat[1105];
    sprintf(concat, "%s$%s", pat, txt);
    int totalLen = strlen(concat);

    int Z[totalLen];
    memset(Z, 0, sizeof(Z));
    computeZ(concat, totalLen, Z);

    int first = 1;
    printf("[");
    for (int i = 0; i < totalLen; i++) {
        // If Z[i] matches pattern length, match found in text
```

```
        if (Z[i] == M) {
            if (!first) printf(", ");
            // Index in original text is i - (pattern length + 1 separator)
            printf("%d", i - M - 1);
            first = 0;
        }
    }
    printf("]\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Neha is building a plagiarism-detection feature where she must compare two text strings to find similarities. To efficiently detect overlaps between the strings, she uses the KMP (Knuth–Morris–Pratt) string matching algorithm, relying on the LPS (Longest Prefix Suffix) array as the main data structure.

Given two strings, write a program to ensure that the task is done by finding the longest common substring present in both strings. If no common substring exists, the program should clearly indicate that result.

Input Format

The first line contains a string representing the first text.

The second line contains a string representing the second text.

Output Format

The output prints the longest common substring if it exists.

If there is no common substring, print "There is no common substring".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abcdef
abc
Output: abc

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

void findLongestCommonSubstring(char* s1, char* s2) {
    int n1 = strlen(s1);
    int n2 = strlen(s2);
    int maxLen = 0;
    int endIdx = 0;

    // dp[i][j] stores the length of the longest common suffix
    // of substrings s1[0...i-1] and s2[0...j-1]
    int dp[101][101];

    // Initialize the DP table
    for (int i = 0; i <= n1; i++) {
        for (int j = 0; j <= n2; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            } else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
                if (dp[i][j] > maxLen) {
                    maxLen = dp[i][j];
                    endIdx = i; // Store the end position in s1
                }
            } else {
                dp[i][j] = 0;
            }
        }
    }

    if (maxLen == 0) {
        printf("There is no common substring\n");
    } else {
        // Print the substring starting from (endIdx - maxLen)
```

```
        for (int i = endIdx - maxLen; i < endIdx; i++) {
            printf("%c", s1[i]);
        }
        printf("\n");
    }
}

int main() {
    char s1[101], s2[101];

    // Reading strings using scanf; handles strings without spaces
    if (scanf("%s", s1) == 1 && scanf("%s", s2) == 1) {
        findLongestCommonSubstring(s1, s2);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 4_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. The user performs the following operations on a stack of size 5 then, which of the following is the correct statement for the 1stack?

push(1);
pop();
push(2);
push(3);
pop();
push(2);
pop();
pop();
push(4);
pop();
pop();
push(5);

24uecs2047

Answer

Underflow occurs

Status : Correct ***Marks : 1/1***

2. If the sequence of operations - push (1), push (2), pop, push (1), push (2), pop, pop, pop, push (2), pop are performed on a stack, the sequence of popped out values

Answer

2,2,1,1,2

Status : Correct ***Marks : 1/1***

3. What is the result of the following operation?

Top (Push (S, X))

Answer

X

Status : Correct ***Marks : 1/1***

4. Stack A has the entries a, b, c (with a on top). Stack B is empty. An entry popped out of stack A can be printed immediately or pushed to stack B. An entry popped out of the stack B can only be printed. In this arrangement, which of the following permutations of a, b, c are not possible?

Answer

c a b

Status : Correct ***Marks : 1/1***

5. Pushing an element into the stack already has five elements. The stack size is 5, then the stack becomes

Answer

Overflow

Status : Correct ***Marks : 1/1***

6. In a Stack, if a user tries to remove an Element from Empty stack it is called

Answer

Underflow

Status : Correct ***Marks : 1/1***

7. The user performs the following operations on the stack of size 5 then at the end of the last operation, the total number of elements present in the stack is

```
push(1);
pop();
push(2);
push(3);
pop();
push(4);
pop();
pop();
push(5);
```

Answer

1

Status : Correct ***Marks : 1/1***

8. What will be the output after performing these sequence of operations

```
push(20);
push(4);
top();
pop();
```

```
pop();
pop();
push(5);
top();
```

Answer

5

Status : Wrong ***Marks : 0/1***

9. A normal queue, if implemented using an array of size MAX_SIZE, gets full when _____.

Answer

rear = MAX_SIZE – 1

Status : Correct ***Marks : 1/1***

10. If the elements 'A', 'B', 'C', and 'D' are placed in a queue and are deleted one at a time, in what order will they be removed?

Answer

ABCD

Status : Correct ***Marks : 1/1***

11. After performing this set of operations, what does the final queue contain?

```
InsertFront(10);
InsertFront(20);
InsertRear(30);
DeleteFront();
InsertRear(40);
InsertRear(10);
DeleteRear();
InsertRear(15);
display();
```

Answer

10 30 40 15

Status : Correct ***Marks : 1/1***

12. What is the purpose of defining the constant SIZE in a queue implementation using an array?

Answer

To limit the maximum number of elements in the queue

Status : Correct ***Marks : 1/1***

13. What does the condition front = NULL indicate in inserting an element into a queue?

Answer

The queue is empty

Status : Correct ***Marks : 1/1***

14. Which of the following is a valid condition for a successful dequeue operation in the array implementation of a queue?

Answer

None of the mentioned options

Status : Correct ***Marks : 1/1***

15. The essential condition that is checked before insertion in a queue is?

Answer

Overflow

Status : Correct ***Marks : 1/1***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 4_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neo is experimenting with text transformations and decides to use a stack to reverse the sequence of words in his sentence. He provides a line of text and expects the program to utilize a stack to reorder the words so that the last word appears first and the first word appears last.

Write a program to ensure this is done correctly using a stack-based approach.

Example

Input: I am Neo

Output: Neo am I

24uecs2047

Input Format

The input consists of a single line of input containing a sentence with words separated by spaces.

Output Format

The output prints a single line of output containing the words of the sentence reversed in order, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Hello World

Output: World Hello

Answer

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define MAX_LIMIT 105
struct Stack{
   int top;
   char words[MAX_LIMIT][MAX_LIMIT];
};
void push(struct Stack* s, char* word){
   strcpy(s->words[++(s->top)], word);
}
char* pop(struct Stack* s){
   return s->words[(s->top)--];
}
int main(){
   char sentence[MAX_LIMIT];
   struct Stack s;
   s.top=-1;
   if(fgets(sentence, sizeof(sentence), stdin)==NULL){
      return 0;
   }
   sentence[strcspn(sentence,"\n")]='\0';
   char* token=strtok(sentence," ");
```

```
    while(token!=NULL){
        push(&s, token);
        token=strtok(NULL, " ");
    }
    while(s.top!=-1){
        printf("%s", pop(&s));
        if(s.top!=-1){
            printf(" ");
        }
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Give a string, remove duplicate letters from the string s such that each letter appears just once using Stack. The lexicographical order of your result must be the least of all feasible outcomes.

Input Format

The input consists of a string.

Output Format

The output displays the string in the lexicographical order with unique characters.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: bcabc
Output: abc

Answer

```
#include<stdio.h>
#include<string.h>
```

```
#include<stdbool.h>
#define MAX 100
void solve(){
    char s[MAX];
    if(scanf("%s",s)==EOF) return;
    int len=strlen(s);
    int count[26]={0};
    bool visited[26]={false};
    char stack[MAX];
    int top=-1;
    for (int i=0;i<len;i++){
        count[s[i]-'a']++;
    }
        for(int i=0;i<len;i++){
        int idx=s[i]-'a';
        count[idx]--;
        if(visited[idx]){
            continue;
        }
        while(top!=-1 && stack[top]>s[i] && count[stack[top]-'a']>0){
            visited[stack[top]-'a']=false;
            top--;
        }
        stack[++top]=s[i];
        visited[idx]=true;
    }
    stack[++top]='\0';
    printf("%s", stack);
}
int main(){
    solve();
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Mira is learning about palindromes and wants to build a program that can check whether a given string is a palindrome using a stack-based approach. A palindrome reads the same backward as forward.

Write a program to ensure that the task is done by pushing characters to a stack and comparing them in reverse order to determine if the string is a palindrome.

Input Format

The input contains a single line containing a string with space-separated characters.

Output Format

The output consists of ,

If the string is a palindrome, print "It's a palindrome!"

Otherwise, print "Not a palindrome"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam
Output: It's a palindrome!

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 200

char stack[MAX];
int top = -1;

void push(char c) {
    stack[++top] = c;
}

char pop() {
```

```
    return stack[top--];
}

int main() {
    char input[MAX];
    char processed[MAX];
    int i, j = 0;

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }

    for (i = 0; input[i] != '\0'; i++) {
        if (isalpha(input[i])) {
            processed[j++] = tolower(input[i]);
        }
    }
    processed[j] = '\0';

    for (i = 0; i < j; i++) {
        push(processed[i]);
    }

    int isPalindrome = 1;
    for (i = 0; i < j; i++) {
        if (processed[i] != pop()) {
            isPalindrome = 0;
            break;
        }
    }

    if (isPalindrome) {
        printf("It's a palindrome!");
    } else {
        printf("Not a palindrome");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Gokul is working on a program to rearrange elements in a queue using an array. Given a queue of integers, he needs to modify it by interleaving its first half with the second half. Write a program to help Gokul achieve this task.

Note: Interleaving involves merging elements from two separate halves in an alternating fashion to create a new sequence.

Example

Input

6

1 2 3 7 9 5

Output

1 7 2 9 3 5

Explanation

The interleave rearranges elements in a queue by alternating between two halves. For instance, given input [1, 2, 3, 7, 9, 5], it splits into [1, 2, 3] and [7, 9, 5], then interleaves them as [1, 7, 2, 9, 3, 5]. It achieves this by splitting the queue into two halves and then merging them interleaved.

Input Format

The first line of the input consists of an integer n, representing the number of elements in the queue.

The second line consists of n space-separated integers, representing the queue elements.

Output Format

The output prints n space-separated integers, representing the rearranged queue after interleaving its first half with the second half.

24uecs2047

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
1 2 3 7 9 5

Output: 1 7 2 9 3 5

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;

    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int queue[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }

    int half = n / 2;
    int firstHalf[half];
    int secondHalf[half];

    for (int i = 0; i < half; i++) {
        firstHalf[i] = queue[i];
    }
    for (int i = 0; i < half; i++) {
        secondHalf[i] = queue[i + half];
    }

    for (int i = 0; i < half; i++) {
        printf("%d %d", firstHalf[i], secondHalf[i]);
        if (i < half - 1) {
            printf(" ");
        }
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Sanjay is learning about queues in his data structures class and he wants to write a program that counts the number of distinct elements in the queue. Your task is to guide him in this.

Example

Input:

7

45 33 29 45 45 46 39

Output:

5

Explanation:

The distinct elements in the queue are 45, 33, 29, 46, 39. So, the count is 5.

Input Format

The first line of input consists of an integer M, representing the number of elements in the queue.

The second line consists of M space-separated integers, representing the queue elements.

Output Format

The output prints an integer, representing the number of distinct elements in the queue.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
30 24 43 31 30
Output: 4

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int M;

    if (scanf("%d", &M) != 1) {
        return 0;
    }

    if (M == 0) {
        printf("0");
        return 0;
    }

    int queue[100];
    for (int i = 0; i < M; i++) {
        scanf("%d", &queue[i]);
    }

    int distinctCount = 0;

    for (int i = 0; i < M; i++) {
        int isDuplicate = 0;

        for (int j = 0; j < i; j++) {
            if (queue[i] == queue[j]) {
                isDuplicate = 1;
                break;
            }
        }

        if (!isDuplicate) {
            distinctCount++;
        }
```

```
    }
    printf("%d", distinctCount);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 4_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

A financial analytics company is developing a tool to analyze stock price trends. The tool needs to identify the Next Greater Price for each stock price in a given list of daily prices. For a given stock price, the Next Greater Price is defined as the first price appearing after it in the list that is greater. If no such price exists, the result should be -1.

Note: The Next greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1.

Examples:

24uecs2047

Input Format

The first line of input consists of an integer n, representing the size of the array.

The second line consists of n space-separated stack of integers.

Output Format

The output prints "Next Greater Elements: " followed by an array of integers, representing the next greater elements in the stack.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
4 5 2 25

Output: Next Greater Elements: 5 25 25 -1

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;

    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Next Greater Elements: ");

    for (int i = 0; i < n; i++) {
        int nextGreater = -1;

        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[j] > arr[i]) {
                nextGreater = arr[j];
                break;
            }
        }

        printf("%d", nextGreater);

        if (i < n - 1) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Riya is working on a text-cleaning utility where certain unwanted patterns must be removed from user input. She uses a stack data structure to process the string character by character.

Given a string containing alphabets, digits, and special characters, write a program to ensure that the task is done by removing every digit along with the closest non-digit character immediately to its left. The remaining characters should be preserved in their original order and printed as the final cleaned string.

Example:

Input:

s = "cb34"

Output:

""

Explanation:

Start with an empty stack.

Push c stack: ['c']

Push b stack: ['c','b']

Encounter 3 pop 'b' stack: ['c']

Encounter 4 pop 'c' stack: []

Result: empty string

Input Format

The input consists of a string s without spaces, consisting of letters, digits, and special characters.

Output Format

The output prints the resulting string after performing the required removals.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abc

Output: abc

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 105

int main() {
    char s[MAX];
    char stack[MAX];
    int top = -1;

    if (scanf("%s", s) != 1) {
        return 0;
```

```
    }

    for (int i = 0; s[i] != '\0'; i++) {
        if (isdigit(s[i])) {
            if (top >= 0) {
                top--;
            }
        } else {
            stack[++top] = s[i];
        }
    }

    for (int i = 0; i <= top; i++) {
        printf("%c", stack[i]);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Kavya is developing a real-time analytics system that processes a continuous stream of numerical data. To efficiently track trends, she uses a Queue data structure to maintain incoming values and compute statistics incrementally.

Given a sequence of integers arriving one by one, write a program to ensure that the task is done by calculating and printing the moving average of all elements received so far after each new input. The average should be displayed up to two decimal places.

Example

Input

3

1 2 3

Output

1.00 1.50 2.00

Explanation

The moving average of the first element is 1.00.

The next elements are 1 and 2, the moving average is (1 + 2) / 2 = 1.50.

The next elements are 1, 2, and 3, the moving average is (1 + 2 + 3) / 3 = 2.00.

Input Format

The first line contains an integer n, representing the number of elements in the data stream.

The second line contains n space-separated integers representing the incoming values.

Output Format

The output prints n floating-point numbers, each representing the moving average after processing the corresponding element.

Each value should be printed with two decimal places, separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: 1.00 1.50 2.00

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;

    if (scanf("%d", &n) != 1) {
```

24uecs2047

```
        return 0;
    }

    int val;
    double runningSum = 0;

    for (int i = 1; i <= n; i++) {
        scanf("%d", &val);
        runningSum += val;

        printf("%.2f", runningSum / i);

        if (i < n) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

In a small business, Jane is tasked with organizing customer feedback scores for a new product. Each customer has rated the product, and Jane needs to arrange these ratings in a specific order to analyze the feedback more effectively.

Specifically, she wants to rearrange the ratings such that the most frequent ratings appear first. If two ratings have the same frequency, the smaller rating should come first. To accomplish this, assist Jane with a program to process a queue of customer ratings and rearrange them based on their frequencies.

Input Format

The first line contains an integer n, which represents the number of customer ratings.

The second line contains n integers, each representing a customer rating.

Output Format

The output displays the rearranged ratings in the order of their frequency (most frequent first).

If frequencies are the same, the smaller rating should come first.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
2 3 2 3 2 3 2
Output: 2 2 2 2 3 3 3

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int ratings[n];
    int freq[100] = {0};

    for (int i = 0; i < n; i++) {
        scanf("%d", &ratings[i]);
        freq[ratings[i]]++;
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            int a = ratings[j];
            int b = ratings[j + 1];

            if (freq[a] < freq[b]) {
                int temp = ratings[j];
                ratings[j] = ratings[j + 1];
                ratings[j + 1] = temp;
```

```
            }
            else if (freq[a] == freq[b]) {
                if (a > b) {
                    int temp = ratings[j];
                    ratings[j] = ratings[j + 1];
                    ratings[j + 1] = temp;
                }
            }
        }
    }

    for (int i = 0; i < n; i++) {
        printf("%d", ratings[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 7

Section 1 : MCQ

1. After adding a new node at the end of a linked list, what does the current last node become?

Answer

The second last node

Status : Correct ***Marks : 1/1***

2. When inserting a new element at the beginning of a linked list, what happens to the current head of the list?

Answer

It becomes the second element

Status : Correct ***Marks : 1/1***

24uecs2047

3. Which of the following are true about a singly linked list?

Answer

Each node has a reference to both the previous and next node.

Status : Wrong ***Marks : 0/1***

4. When inserting a new node into a singly linked list, which of the following are the possible cases for insertion?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

5. Given the linked list: 5 -> 10 -> 15 -> 20 -> 25 -> NULL. What will be the output of traversing the list and printing each node's data?

Answer

5 10 15 20 25

Status : Correct ***Marks : 1/1***

6. Which of the following is the process of finding information in a linked list?

Answer

Start from the tail of the list and search backward

Status : Wrong ***Marks : 0/1***

7. Which of the following actions needs to be taken when the last item in a linked list is deleted?

Answer

Delete the entire linked list

24uecs2047

Status : Wrong ***Marks : 0/1***

8. Which pointer helps in traversing a doubly linked list in reverse order?

Answer

prev

Status : Correct ***Marks : 1/1***

9. Which of the following is true about the last node in a doubly linked list?

Answer

Its next pointer is NULL

Status : Correct ***Marks : 1/1***

10. What will be the effect of setting the prev pointer of a node to NULL in a doubly linked list?

Answer

The node will become the new tail

Status : Wrong ***Marks : 0/1***

11. How do you delete a node from the middle of a doubly linked list?

Answer

Update the next pointer of the previous node

Status : Wrong ***Marks : 0/1***

12. What happens if we insert a node at the beginning of a doubly linked list?

Answer

The previous pointer of the head node is not updated

24uecs2047

Status : Wrong ***Marks : 0/1***

13. How many pointers does a node in a doubly linked list have?

Answer

2

Status : Correct ***Marks : 1/1***

14. In deletion from the end of a doubly linked list, which pointer needs to be updated?

Answer

tail->next = head

Status : Wrong ***Marks : 0/1***

15. Which of the following is true about a Doubly Linked List?

Answer

All nodes must contain two NULL pointers

Status : Wrong ***Marks : 0/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 5_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Arun is tasked with implementing a program to reverse a singly linked list. The program should populate the list elements by inserting them at the beginning, traverse the list and reverse the order of its elements.

Since Arun is new to programming, you have to guide him through the task.

Input Format

The first line of input consists of an integer N, representing the linked list size.

The second line consists of N space-separated list elements.

Output Format

The first line of output prints the list elements after inserting them at the beginning.

24uecs2047

The second line prints the reversed linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
23 85 47 62 31

Output: 31 62 47 85 23
23 85 47 62 31

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

void insertAtBeginning(struct Node** head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = *head;
   *head = newNode;
}

void reverseList(struct Node** head) {
   struct Node *prev = NULL, *current = *head, *next = NULL;
   while (current != NULL) {
      next = current->next;
      current->next = prev;
      prev = current;
      current = next;
   }
   *head = prev;
}
```

```
void printList(struct Node* head) {
    while (head != NULL) {
        printf("%d", head->data);
        if (head->next != NULL) {
            printf(" ");
        }
        head = head->next;
    }
    printf(" \n");
}

int main() {
    int n, val;
    struct Node* head = NULL;

    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertAtBeginning(&head, val);
    }

    printList(head);
    reverseList(&head);
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

In a computer science course project, students are required to write a program to insert values at the end of a singly linked list and remove the last node from the list. The program should traverse the list and delete the last node, adjusting pointers accordingly.

Assist students in the task.

Input Format

The first line of input consists of an integer N, representing the list size.

The second line consists of N space-separated integers, representing the list elements.

Output Format

The output prints space-separated integers, representing the list after the deletion of the last element.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8
54 25 14 38 55 73 95 82

Output: 54 25 14 38 55 73 95

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

void insertAtEnd(struct Node** head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;

   if (*head == NULL) {
      *head = newNode;
      return;
   }

   struct Node* temp = *head;
```

```
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void deleteLastNode(struct Node** head) {
    if (*head == NULL || (*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }

    struct Node* temp = *head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    free(temp->next);
    temp->next = NULL;
}

void printList(struct Node* head) {
    while (head != NULL) {
        printf("%d", head->data);
        if (head->next != NULL) {
            printf(" ");
        }
        head = head->next;
    }
}

int main() {
    int n, val;
    struct Node* head = NULL;

    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertAtEnd(&head, val);
    }
```

```
    deleteLastNode(&head);
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Dev is tasked with creating a program that efficiently finds the middle element of a linked list. The program should take user input to populate the linked list by inserting each element into the front of the list and then determining the middle element.

Assist Dev, as he needs to ensure that the middle element is accurately identified from the constructed singly linked list:

If it's an odd-length linked list, return the middle element.If it's an even-length linked list, return the second middle element of the two elements.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The first line of output displays the linked list after inserting elements at the front.

The second line displays "Middle Element: " followed by the middle element of the linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
Output: 50 40 30 20 10
Middle Element: 30

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

void insertAtFront(struct Node** head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = *head;
   *head = newNode;
}

void printList(struct Node* head) {
   struct Node* temp = head;
   while (temp != NULL) {
      printf("%d", temp->data);
      if (temp->next != NULL) {
         printf(" ");
      }
      temp = temp->next;
   }
   printf(" \n");
}

int findMiddle(struct Node* head) {
   if (head == NULL) return -1;

   struct Node* slow = head;
   struct Node* fast = head;
```

```
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->data;
}

int main() {
    int n, val;
    struct Node* head = NULL;

    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertAtFront(&head, val);
    }

    printList(head);

    if (n > 0) {
        printf("Middle Element: %d", findMiddle(head));
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Saran is tasked with implementing a doubly linked list in data structures, allowing users to append elements to the end of the list, and remove the last element. He needs to ensure efficient memory management in the implementation.

Assist him with a suitable program.

Input Format

The first line of input consists of an integer n, representing the size of the list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The output displays integers, representing the list after removing the last element, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
48 22 5 62 87
Output: 48 22 5 62

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

void append(struct Node** head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;

   if (*head == NULL) {
      newNode->prev = NULL;
      *head = newNode;
      return;
   }

   struct Node* temp = *head;
   while (temp->next != NULL) {
```

```
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;
}

void removeLast(struct Node** head) {
    if (*head == NULL) return;

    struct Node* temp = *head;

    if (temp->next == NULL) {
        free(temp);
        *head = NULL;
        return;
    }

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->prev->next = NULL;
    free(temp);
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) {
            printf(" ");
        }
        temp = temp->next;
    }
}

int main() {
    int n, val;
    struct Node* head = NULL;

    if (scanf("%d", &n) != 1) return 0;
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        append(&head, val);
    }

    removeLast(&head);
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement:

Ravi is building a simple application to manage a doubly linked list of integers. He wants to insert new integers into the list and delete a node at a specific position. After deleting it, he needs to display the updated list.

Write a program to ensure that the task is done by performing insertion, deletion at a given position, and then printing the modified list.

Input Format

The first line contains an integer n, representing the number of elements to be inserted into the list.

The second line contains n space-separated integers, representing the values to insert.

The third line contains an integer pos, representing the position of the node to be deleted from the list.

Output Format

If the given position is valid, print "Node at position X with value Y deleted successfully".

If the position is invalid, print "Position X not found in the list."

In both cases, print

"Data present in the list:"

followed by the updated list elements on the next line, space separated."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
4
Output: Node at position 4 with value 40 deleted successfully.
Data present in the list:
10 20 30 50

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

void insertAtEnd(struct Node** head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   if (*head == NULL) {
      newNode->prev = NULL;
      *head = newNode;
```

```
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}

void deleteAtPosition(struct Node** head, int pos) {
    if (*head == NULL) {
        printf("Position %d not found in the list.\n", pos);
        return;
    }

    struct Node* temp = *head;
    int count = 1;

    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL || pos < 1) {
        printf("Position %d not found in the list.\n", pos);
        return;
    }

    printf("Node at position %d with value %d deleted successfully.\n", pos, temp-
>data);

    if (*head == temp) *head = temp->next;
    if (temp->next != NULL) temp->next->prev = temp->prev;
    if (temp->prev != NULL) temp->prev->next = temp->next;

    free(temp);
}

void printList(struct Node* head) {
    printf("Data present in the list:\n");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
```

```
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
}

int main() {
    int n, val, pos;
    struct Node* head = NULL;

    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertAtEnd(&head, val);
    }
    if (scanf("%d", &pos) != 1) return 0;

    deleteAtPosition(&head, pos);
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given a singly linked list and two integers M and N, implement a program to traverse the linked list such that you retain every M node and then delete the next N nodes.

Continue this process until the end of the linked list.

Input Format

The first line consists of two integers M and N, separated by a space, representing the values to retain and delete.

The second line consists of a series of integers separated by spaces, representing the values of the linked list nodes.

The last input is -1 which will terminate the inputs.

24uecs2047

Output Format

The output prints the linked list after applying the process of retaining every M node and deleting the next N nodes represented as a series of integers separated by "->".

Sample Test Case

Input: 2 2
1 2 3 4 5 6 7 8 -1

Output: 1->2->5->6

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

void insert(struct Node** head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   if (*head == NULL) {
      *head = newNode;
      return;
   }
   struct Node* temp = *head;
   while (temp->next != NULL) temp = temp->next;
   temp->next = newNode;
}

void skipMdeleteN(struct Node* head, int M, int N) {
   struct Node* curr = head;
   struct Node* t;
   int count;

   while (curr) {
      for (count = 1; count < M && curr != NULL; count++) {
         curr = curr->next;
```

```
        }

        if (curr == NULL) return;

        t = curr->next;
        for (count = 1; count <= N && t != NULL; count++) {
            struct Node* temp = t;
            t = t->next;
            free(temp);
        }

        curr->next = t;
        curr = t;
    }
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) {
            printf("->");
        }
        temp = temp->next;
    }
}

int main() {
    int M, N, val;
    struct Node* head = NULL;

    if (scanf("%d %d", &M, &N) != 2) return 0;

    while (scanf("%d", &val) == 1 && val != -1) {
        insert(&head, val);
    }

    skipMdeleteN(head, M, N);
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you are working on a text processing tool and need to implement a feature that allows users to insert characters at a specific position.

Implement a program that takes user inputs to create a singly linked list of characters and inserts a new character after a given index in the list.

Input Format

The first line of input consists of an integer N, representing the number of characters in the linked list.

The second line consists of a sequence of N characters, representing the linked list.

The third line consists of an integer index, representing the index(0-based) after which the new character node needs to be inserted.

The fourth line consists of a character value representing the character to be inserted after the given index.

Output Format

If the provided index is out of bounds (larger than the list size):

1. The first line prints "Invalid index".
2. The second line prints "Updated list: " followed by the unchanged linked list values.

Otherwise, prints "Updated list: " followed by the updated space separated linked list after inserting the new character after the given index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
a b c d e
2
X

Output: Updated list: a b c X d e

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   char data;
   struct Node* next;
};

void insertAtEnd(struct Node** head, char val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   if (*head == NULL) {
      *head = newNode;
      return;
   }
   struct Node* temp = *head;
   while (temp->next != NULL) temp = temp->next;
   temp->next = newNode;
}

void printList(struct Node* head) {
   printf("Updated list: ");
   struct Node* temp = head;
   while (temp != NULL) {
      printf("%c", temp->data);
      if (temp->next != NULL) printf(" ");
      temp = temp->next;
   }
   printf(" ");
}
```

```
int main() {
    int n, index;
    char val, newChar;
    struct Node* head = NULL;

    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf(" %c", &val);
        insertAtEnd(&head, val);
    }

    if (scanf("%d", &index) != 1) return 0;
    scanf(" %c", &newChar);

    struct Node* temp = head;
    int currIdx = 0;
    while (temp != NULL && currIdx < index) {
        temp = temp->next;
        currIdx++;
    }

    if (temp == NULL || index < 0) {
        printf("Invalid index\n");
    } else {
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = newChar;
        newNode->next = temp->next;
        temp->next = newNode;
    }

    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Imagine Anu is tasked with finding the middle element of a doubly linked

list. Given a doubly linked list where each node contains an integer value and is inserted at the end, implement a program to find the middle element of the list. If the number of nodes is even, return the middle element pair.

Input Format

The first line of input consists of an integer N, representing the number of nodes in the doubly linked list.

The second line consists of N space-separated integers, representing the values of the nodes in the doubly linked list.

Output Format

The first line of output prints the space-separated elements of the doubly linked list.

The second line prints the middle element(s) of the doubly linked list, depending on whether the number of nodes is odd or even.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
Output: 10 20 30 40 50
30

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

void insertAtEnd(struct Node** head, int val) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;
    if (*head == NULL) {
        newNode->prev = NULL;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf(" \n");
}

void findMiddle(struct Node* head, int n) {
    if (head == NULL) return;

    struct Node* slow = head;
    struct Node* fast = head;

    if (n % 2 != 0) {
        // Odd length: Tortoise and Hare
        while (fast != NULL && fast->next != NULL) {
            slow = slow->next;
            fast = fast->next->next;
        }
        printf("%d", slow->data);
    } else {
        // Even length: Find the two middle elements
        // Move slow to the (n/2)th element
        for (int i = 1; i < n / 2; i++) {
            slow = slow->next;
```

```
    }
    printf("%d %d", slow->data, slow->next->data);
  }
}

int main() {
  int n, val;
  struct Node* head = NULL;

  if (scanf("%d", &n) != 1) return 0;

  for (int i = 0; i < n; i++) {
    scanf("%d", &val);
    insertAtEnd(&head, val);
  }

  printList(head);
  findMiddle(head, n);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Pranav wants to clockwise rotate a doubly linked list by a specified number of positions. He needs your help to implement a program to achieve this. Given a doubly linked list and an integer representing the number of positions to rotate, write a program to rotate the list clockwise.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated linked list elements.

The third line consists of an integer k, representing the number of positions to rotate the list.

Output Format

The output displays the elements of the doubly linked list after rotating it by k positions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
1
Output: 5 1 2 3 4

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

void append(struct Node** head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   if (*head == NULL) {
      newNode->prev = NULL;
      *head = newNode;
      return;
   }
   struct Node* temp = *head;
   while (temp->next != NULL) temp = temp->next;
   temp->next = newNode;
   newNode->prev = temp;
}
```

```
void rotateClockwise(struct Node** head, int k, int n) {
    if (*head == NULL || k == 0 || k % n == 0) return;

    // A clockwise rotation by k is equivalent to
    // moving the head pointer (n-k) steps forward.
    struct Node* current = *head;

    // 1. Find the tail and connect it to the head to make it circular
    struct Node* tail = *head;
    while (tail->next != NULL) {
        tail = tail->next;
    }
    tail->next = *head;
    (*head)->prev = tail;

    // 2. Find the new tail: (n - k) steps from the start
    int stepsToNewTail = n - k;
    struct Node* newTail = *head;
    for (int i = 1; i < stepsToNewTail; i++) {
        newTail = newTail->next;
    }

    // 3. Set the new head and break the circular connection
    *head = newTail->next;
    (*head)->prev = NULL;
    newTail->next = NULL;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
}

int main() {
    int n, val, k;
    struct Node* head = NULL;

    if (scanf("%d", &n) != 1) return 0;
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        append(&head, val);
    }
    if (scanf("%d", &k) != 1) return 0;

    if (n > 1) {
        rotateClockwise(&head, k, n);
    }

    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 6_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. In Radix Sort, what happens if the input array contains numbers with different lengths?

Answer

The shorter numbers are padded with zeros.

Status : Correct ***Marks : 1/1***

2. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct ***Marks : 1/1***

24uecs2047

3. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct ***Marks : 1/1***

4. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Largest element

Status : Correct ***Marks : 1/1***

5. Bucket sort is most efficient in the case when _____

Answer

The input is uniformly distributed

Status : Correct ***Marks : 1/1***

6. Which of the following is not necessarily a stable sorting algorithm?

Answer

All of the mentioned options

Status : Wrong ***Marks : 0/1***

7. Which step is essential to implement Bucket Sort correctly?

Answer

Dividing the range of input into intervals (buckets)

Status : Correct ***Marks : 1/1***

24uecs2047

8. Which of the following is a characteristic of Radix Sort?

Answer

It is a stable sorting algorithm.

Status : Correct ***Marks : 1/1***

9. A database contains the following employee IDs: [1263, 2910, 8742, 3891, 2156]. Using Radix Sort, which of the following is the correct order of employee IDs after the first pass (sorting by the least significant digit)?

Answer

[2910, 3891, 8742, 1263, 2156]

Status : Correct ***Marks : 1/1***

10. What is the purpose of the counting sort algorithm in Radix Sort?

Answer

To sort the elements based on the current digit.

Status : Correct ***Marks : 1/1***

11. When does Heap Sort perform at its worst?

Answer

When all elements of the input array are the same.

Status : Correct ***Marks : 1/1***

12. How does Counting Sort handle duplicate elements?

Answer

It preserves their relative order

Status : Correct ***Marks : 1/1***

24uecs2047

13. What is the purpose of the count array in Counting Sort?

Answer

To store the count of each unique element of the input array at their respective indices

Status : Correct ***Marks : 1/1***

14. What is the purpose of calculating the prefix sum in the Counting Sort algorithm?

Answer

To determine the final positions of elements in the output array

Status : Correct ***Marks : 1/1***

15. What is the initial value of each element in the count array in Counting Sort?

Answer

0

Status : Correct ***Marks : 1/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 6_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Krish is participating in a coding competition and has been given an array of integers. His task is to sort the array using the Counting Sort algorithm.

Your goal is to help Krish by implementing the countSort method to sort the array in non-decreasing order.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

24uecs2047

The output prints a single line containing N space-separated integers representing the sorted array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
4 2 2 8 3

Output: 2 2 3 4 8

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

void countSort(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }

    int* count = (int*)calloc(max + 1, sizeof(int));

    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    int index = 0;
    for (int i = 0; i <= max; i++) {
        while (count[i] > 0) {
            arr[index++] = i;
            count[i]--;
        }
    }

    free(count);
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    countSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Raj, a keen problem solver, has come across an intriguing numerical puzzle that involves sorting numbers in ascending order. Eager to crack the code, he decides to implement the Radix Sort algorithm.

Your task is to assist Raj in designing a program to solve this numerical puzzle.

Input Format

The first line contains an integer n, denoting the number of elements to be sorted.

The second line consists of n space-separated integers, representing the numbers to be sorted.

Output Format

The output prints the array after it has been sorted using the Radix Sort algorithm, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
543 986 217 765 329

Output: 217 329 543 765 986

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];
    return max;
}

void countSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    for (int i = 0; i < n; i++)
```

```
        arr[i] = output[i];
}

void radixSort(int arr[], int n) {
    int m = getMax(arr, n);

    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    radixSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

int findKthLargest(int arr[], int n, int k) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    for (int i = n - 1; i >= n - k; i--) {
        if (i == n - k) {
            return arr[0];
        }
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
    return -1;
}

int main() {
    int n, k;
    if (scanf("%d", &n) != 1) return 0;

    int* arr = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (scanf("%d", &k) != 1) {
        free(arr);
        return 0;
    }
```

```
    printf("%d", findKthLargest(arr, n, k));

    free(arr);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
banana pineapple mango chikoo jack star guava lemon tomato orange
Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

// You are using GCC

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void swap(char arr[][50], int i, int j) {
    char temp[50];
    strcpy(temp, arr[i]);
    strcpy(arr[i], arr[j]);
    strcpy(arr[j], temp);
}

void heapify(char arr[][50], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;

    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;

    if (largest != i) {
        swap(arr, i, largest);
        heapify(arr, n, largest);
    }
}

void heapSort(char arr[][50], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(arr, 0, i);
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
```

```
    char words[n][50];
    for (int i = 0; i < n; i++) {
        scanf("%s", words[i]);
    }

    heapSort(words, n);

    for (int i = 0; i < n; i++) {
        printf("%s", words[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

You're organizing a gift-giving event where participants exchange gifts of varying values. To ensure fairness, you use the Bucket Sort algorithm. You gather the participants' gift values and initialize a count array.

Then, you iterate through the values, updating the count array. Next, you sort the values in descending order by iterating through the count array. Finally, you display the sorted values, ensuring fairness in the gift exchange.

Input Format

The first line contains an integer N, representing the number of gift values.

The second line contains N space-separated integers representing the gift values brought by the participants. Each gift value is an integer between 1 and 999, inclusive.

Output Format

The output displays a single line containing N space-separated integers representing the sorted gift values in descending order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
156 378 453 278 281 987
Output: 987 453 378 281 278 156

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n, val;
    int count[1000] = {0};

    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        if (scanf("%d", &val) == 1) {
            count[val]++;
        }
    }

    int first = 1;
    for (int i = 999; i >= 1; i--) {
        while (count[i] > 0) {
            if (!first) {
                printf(" ");
            }
            printf("%d", i);
            first = 0;
            count[i]--;
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 6_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Yuvana, an avid programmer, is keen on mastering various sorting algorithms and has recently delved into Counting Sort. As a practice exercise, she is given an array of N integers where each element ranges from 1 to N-1. Some numbers may appear multiple times in the array. Her task is to identify and print all numbers that appear more than once. If no such duplicates are found, she should print -1.

Help Yuvana by implementing a function that efficiently finds and prints duplicate numbers using the Counting Sort technique.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

24uecs2047

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints all the duplicate numbers that appear more than once in the array, each separated by a space.

If no duplicates are found, print -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 3 4 2 2

Output: 2

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    int max_val = 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }

    int *count = (int *)calloc(max_val + 1, sizeof(int));

    for (int i = 0; i < n; i++) {
```

```
        count[arr[i]]++;
    }

    int foundDuplicate = 0;
    int first = 1;

    for (int i = 0; i <= max_val; i++) {
        if (count[i] > 1) {
            if (!first) {
                printf(" ");
            }
            printf("%d", i);
            foundDuplicate = 1;
            first = 0;
        }
    }

    if (!foundDuplicate) {
        printf("-1");
    }

    free(count);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Anu is tasked with organizing a list of fruit names stored as strings. To ensure that the list is easy to navigate, she decides to sort the strings in ascending alphabetical order.

Being a fan of efficient sorting algorithms, Anu opts to use radix sort for this task. Help Anu with a program that can take a list of fruit names as input and output the sorted list.

Input Format

The first line of input consists of an integer n, representing the number of fruit names in the list.

The next n lines each consist of a string, representing the names of the fruits.

Output Format

The output prints n lines containing the sorted list of fruit names in ascending alphabetical order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
apple
banana
orange
grape

Output: apple
banana
grape
orange

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void countSort(char arr[][105], int n, int pos) {
    char output[n][105];
    int count[256] = {0};

    for (int i = 0; i < n; i++) {
        int charIndex = (int)(unsigned char)arr[i][pos];
        count[charIndex]++;
    }

    for (int i = 1; i < 256; i++) {
        count[i] += count[i - 1];
    }
```

```
    for (int i = n - 1; i >= 0; i--) {
        int charIndex = (int)(unsigned char)arr[i][pos];
        strcpy(output[count[charIndex] - 1], arr[i]);
        count[charIndex]--;
    }

    for (int i = 0; i < n; i++) {
        strcpy(arr[i], output[i]);
    }
}

void radixSort(char arr[][105], int n) {
    int maxLen = 0;
    for (int i = 0; i < n; i++) {
        int len = strlen(arr[i]);
        if (len > maxLen) maxLen = len;
    }

    for (int pos = maxLen - 1; pos >= 0; pos--) {
        countSort(arr, n, pos);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    char fruits[n][105];
    for (int i = 0; i < n; i++) {
        scanf("%s", fruits[i]);
    }

    radixSort(fruits, n);

    for (int i = 0; i < n; i++) {
        printf("%s", fruits[i]);
        if (i < n - 1) {
            printf("\n");
        }
    }

    return 0;
```

}

Status : Correct ***Marks : 10/10***

3. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5
Output: 5th largest Salary: 28000

Answer

// You are using GCC

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

int findKthLargest(int arr[], int n, int k) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    int result;
    for (int i = 0; i < k; i++) {
        result = arr[0];
        arr[0] = arr[n - 1 - i];
        heapify(arr, n - 1 - i, 0);
    }
    return result;
}

int main() {
    int n, k;
```

```
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (scanf("%d", &k) != 1) return 0;

    int result = findKthLargest(arr, n, k);
    printf("%dth largest Salary: %d", k, result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Manage a warehouse inventory with unique barcode numbers utilizing a specialized sorting algorithm designed for barcodes with exactly 16 digits. Input the number of barcode numbers (N) and their corresponding values.

Employ the Bucket Sort algorithm to distribute and sort the numbers based on their digits. Display the sorted barcode numbers, ensuring an efficient and organized warehouse inventory management system tailored for barcode systems with 16 digits.

Input Format

The first line contains an integer N, representing the number of barcode numbers to be sorted.

The following N lines contain exactly 16-digit long long integers, each representing a unique barcode number.

Output Format

The output displays a single line containing "Sorted barcode numbers:" followed by the sorted barcode numbers in ascending order. Each barcode number is displayed on a separate line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2345678901234567
1234567890123456
3456789012345678
5678901234567890
4567890123456789

Output: Sorted barcode numbers:
1234567890123456
2345678901234567
3456789012345678
4567890123456789
5678901234567890

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

void bucketSort(long long arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                long long temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long barcodes[n];
    for (int i = 0; i < n; i++) {
```

```
        scanf("%lld", &barcodes[i]);
    }

    bucketSort(barcodes, n);

    printf("Sorted barcode numbers:\n");
    for (int i = 0; i < n; i++) {
        printf("%lld", barcodes[i]);
        if (i < n - 1) {
            printf("\n");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. Which of the following operations can be used to traverse a Binary Search Tree (BST) in ascending order?

Answer

Inorder traversal

Status : Correct ***Marks : 1/1***

2. Find the pre-order traversal of the given binary search tree.

Answer

13, 2, 1, 4, 14, 18

Status : Correct ***Marks : 1/1***

24uecs2047

3. Find the postorder traversal of the given binary search tree.

Answer

1, 4, 2, 18, 14, 13

Status : Correct ***Marks : 1/1***

4. Find the inorder traversal of the given binary search tree.

Answer

1, 2, 4, 6, 7, 9, 10, 14

Status : Correct ***Marks : 1/1***

5. When comparing a value to be searched with the root node in a BST?

Answer

If it's larger, go to the right subtree

Status : Correct ***Marks : 1/1***

6. In a binary search tree (BST), if the value to be searched is smaller than the value of the root, where should the search proceed?

Answer

Left subtree

Status : Correct ***Marks : 1/1***

7. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the right child of the node 12?

Answer

24uecs2047

16

Status : Correct ***Marks : 1/1***

8. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the left child of the node 16?

Answer

14

Status : Correct ***Marks : 1/1***

9. The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the post-order traversal sequence of the same tree?

Answer

15, 10, 23, 25, 20, 35, 42, 39, 30

Status : Correct ***Marks : 1/1***

10. In a binary search tree, which of the following traversals is used for getting ascending order values?

Answer

In-order

Status : Correct ***Marks : 1/1***

11. Consider the given binary search tree, if the root node is deleted, then the new root can be _____.

Answer

48 or 59

Status : Correct ***Marks : 1/1***

12. After deleting a node in a binary search tree, how is the replacement node determined if the node to be deleted has only one child?

Answer

Replacement node is the child node itself

Status : Correct ***Marks : 1/1***

13. What will be the in-order traversal sequence of the following binary search tree, after deleting node 40?

Answer

8 12 20 22 30

Status : Correct ***Marks : 1/1***

14. Which of the following nodes would replace node 40 if it were to be deleted?

Answer

90

Status : Wrong ***Marks : 0/1***

15. Which of the following statements is true about deleting a node with one child from a binary search tree?

Answer

The child node replaces the deleted node

Status : Correct ***Marks : 1/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 7_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 37.5

Section 1 : Coding

1. Problem Statement

A publishing company is building a digital index system for storing book references. Each reference is represented by a single uppercase character. To ensure efficient organization, the system uses a Binary Search Tree (BST):

If the new character is smaller than the current node, it is placed in the left subtree.If the new character is greater, it is placed in the right subtree.

Once all references are stored in the BST, the system needs to perform a postorder traversal to generate a summary sequence, which is then used for indexing and cross-referencing.

Your task is to help the publishing company build the BST from the given characters and print the postorder traversal.

24uecs2047

Example:

Input:

5

Z E W T Y

Output:

Postorder traversal: T Y W E Z

Input Format

The first line consists of an integer N, representing the number of characters.

The second line consists of N space-separated characters.

Output Format

The output displays the post-order traversal of the given inputs as space-separated.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
Z E W T Y
Output: Postorder traversal: T Y W E Z

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Structure for a BST node
struct Node {
   char data;
   struct Node *left, *right;
};
```

24uecs2047

```
// Function to create a new node
struct Node* newNode(char item) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = item;
    temp->left = temp->right = NULL;
    return temp;
}

// Function to insert a character into the BST
struct Node* insert(struct Node* node, char data) {
    if (node == NULL) return newNode(data);

    if (data < node->data)
        node->left = insert(node->left, data);
    else if (data > node->data)
        node->right = insert(node->right, data);

    return node;
}

// Function for Postorder Traversal: Left -> Right -> Root
void postorderTraversal(struct Node* root) {
    if (root != NULL) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%c ", root->data);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct Node* root = NULL;
    char val;

    for (int i = 0; i < n; i++) {
        scanf(" %c", &val); // Note the space before %c to skip whitespace
        root = insert(root, val);
    }

    printf("Postorder traversal: ");
```

```
    postorderTraversal(root);
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Thiru is learning about binary search trees and wants to practice creating and traversing them. He needs to write a program that takes a series of characters as input and constructs a binary search tree with those characters.

After constructing the tree, Thiru wants to perform an in-order traversal and print all the characters in sorted order.

Input Format

The first line contains an integer n, representing the number of characters to be inserted into the binary search tree.

The second line contains n space-separated characters c_i, representing the characters to be inserted into the binary search tree.

Output Format

The output prints a single line containing all the characters of the binary search tree in sorted order, obtained by performing an in-order traversal of the tree, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
r e f s
Output: e f r s

24uecs2047

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// 1. Structure definition (Java doesn't have 'struct')
struct Node {
    char data;
    struct Node *left, *right;
};

// 2. Memory allocation using malloc (Java uses 'new')
struct Node* createNode(char value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, char value) {
    if (root == NULL) return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}

// 3. In-order Traversal logic
void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%c ", root->data);
        inorderTraversal(root->right);
    }
}

int main() {
    int n;
```

```
    // Standard C input handling
    if (scanf("%d", &n) != 1) return 0;

    struct Node* root = NULL;
    char ch;

    for (int i = 0; i < n; i++) {
        // The space before %c is vital in C to skip the newline character
        scanf(" %c", &ch);
        root = insert(root, ch);
    }

    inorderTraversal(root);
    printf("\n");

    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

3. Problem Statement

Thilak is studying binary search trees (BSTs) and wants to implement a program that constructs a binary search tree from a given set of characters. He aims to perform a pre-order traversal of the constructed tree to understand its structure better.

Your task is to help Thilak by designing a program that accomplishes this.

Input Format

The first line of input consists of an integer N, representing the number of characters to be inserted into the BST.

The second line consists of N space-separated characters, representing the data elements to be inserted into the BST.

Output Format

The output prints the pre-order traversal of the constructed BST.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
C A E B D

Output: C A B E D

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Structure for the BST node
struct Node {
    char data;
    struct Node *left, *right;
};

// Function to create a new tree node
struct Node* createNode(char data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Function to insert a character into the BST
struct Node* insert(struct Node* node, char data) {
    if (node == NULL) {
        return createNode(data);
    }

    if (data < node->data) {
        node->left = insert(node->left, data);
    } else if (data > node->data) {
        node->right = insert(node->right, data);
    }

    return node;
```

```
}

// Pre-order Traversal: Root -> Left -> Right
void preOrder(struct Node* root) {
    if (root != NULL) {
        printf("%c ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct Node* root = NULL;
    char val;

    for (int i = 0; i < n; i++) {
        // Space before %c skips any preceding whitespace or newlines
        scanf(" %c", &val);
        root = insert(root, val);
    }

    preOrder(root);
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Jake is learning about binary search trees(BST) and their operations. He wants to implement a program that can delete a node from a BST based on the given key value and print the remaining nodes in an in-order traversal.

Assist Jake in the program.

Input Format

The first line of input consists of an integer n, representing the number of elements in BST.

The second line consists of n space-separated integers, representing the elements of the tree.

The third line consists of an integer x, representing the key value of the node to be deleted.

Output Format

The first line of output prints "Before deletion: " followed by the in-order traversal of the initial BST.

The second line prints "After deletion: " followed by the in-order traversal after the deletion of the key value.

If the key value is not present in the BST, print the original tree as it is.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
8 6 4 3 1
4
Output: Before deletion: 1 3 4 6 8
After deletion: 1 3 6 8

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};
```

24uecs2047

```
// Function to createw node
struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->left =wNode->righ

// Standard BST insertion
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return createNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
        root->right = insert(root->right, val);
    return root;
}

// Find the minimum value node in a subtree (In-order Successor)
struct Node* minValueNode(struct Node* node) {
    struct Node* current = node;
    while (current && current->left != NULL)
        current = current->left;
    return current;
}

// Function to delete a node
struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Node with only one child or no child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
```

```
        }

        // Node with two children: Get the in-order successor
        struct Node* temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    int n, val, x;
    if (scanf("%d", &n) != 1) return 0;

    struct Node* root = NULL;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }

    scanf("%d", &x);

    printf("Before deletion: ");
    inorder(root);
    printf("\n");

    root = deleteNode(root, x);

    printf("After deletion: ");
    inorder(root);
    printf("\n");

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

5. Problem Statement

You are required to implement basic operations on a Binary Search Tree (BST), like insertion and searching.

Insertion: Given a list of integers, construct a Binary Search Tree by repeatedly inserting each integer into the tree according to the rules of a BST.

Searching: Given an integer, search for its presence in the constructed Binary Search Tree. Print whether the integer is found or not.

Write a program to calculate this efficiently.

Input Format

The first line of input consists of an integer n, representing the number of nodes in the binary search tree.

The second line consists of the values of the nodes, separated by space as integers.

The third line consists of an integer representing, the value that is to be searched.

Output Format

The output prints, "Value <value> is found in the tree." if the given value is present, otherwise it prints: "Value <value> is not found in the tree."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
8 3 10 1 6 14 23
6
Output: Value 6 is found in the tree.

Answer

```
#include<stdio.h>
#include <stdlib.h>

// Structure for a BST node
struct Node {
   int data;
   struct Node *left, *right;
};

// Function to create a new node
struct Node* createNode(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = newNode->right = NULL;
   return newNode;
}

// Function to insert a value into the BST
struct Node* insert(struct Node* root, int value) {
   if (root == NULL) {
      return createNode(value);
   }

   if (value < root->data) {
      root->left = insert(root->left, value);
   } else if (value > root->data) {
      root->right = insert(root->right, value);
   }

   return root;
}

// Function to search for a value in the BST
int search(struct Node* root, int key) {
   // Base Cases: root is null or key is present at root
   if (root == NULL) {
      return 0;
   }
   if (root->data == key) {
      return 1;
   }
```

```
    // Key is greater than root's data
    if (key > root->data) {
        return search(root->right, key);
    }

    // Key is smaller than root's data
    return search(root->left, key);
}

int main() {
    int n, val, searchKey;

    // Read the number of nodes
    if (scanf("%d", &n) != 1) return 0;

    struct Node* root = NULL;

    // Construct the BST
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }

    // Read the search key
    scanf("%d", &searchKey);

    // Perform search and print output
    if (search(root, searchKey)) {
        printf("Value %d is found in the tree.\n", searchKey);
    } else {
        printf("Value %d is not found in the tree.\n", searchKey);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 7_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 37.5

Section 1 : Coding

1. Problem Statement

Given a BST, write an efficient function to delete a given key in it.

There are three cases that can be considered:

Deleting a node with no children: remove the node from the tree.Deleting a node with two childrenDeleting a node with one child

Input Format

The first line of the input consists of an integer n representing, the number of nodes.

The second line of the input consists of integers space-separated representing, input node values.

The third line of the input consists of an integer q representing, the node value to

be deleted.

Output Format

The output prints the node values before and after the deletion of the key value in the format:

"Before deletion of key value:"

"After deleting key value:"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 30 5 6 7
30
Output: Before deletion of key value:
5 6 7 10 30
After deleting key value:
5 6 7 10

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
   int data;
   struct Node *left, *right;
} Node;

Node* newNode(int val) {
   Node* node = (Node*)malloc(sizeof(Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}

Node* insert(Node* root, int val) {
```

```
    if (root == NULL) return newNode(val);
    if (val < root->data) root->left = insert(root->left, val);
    else if (val > root->data) root->right = insert(root->right, val);
    // ignore duplicates
    return root;
}

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

Node* minValueNode(Node* node) {
    Node* cur = node;
    while (cur && cur->left!= NULL) cur = cur->left;
    return cur;
}

Node* deleteNode(Node* root, int key) {
    if (root == NULL) return root;

    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    } else {
        // Case 1: No child / leaf
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        // Case 3: One child
        else if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
```

```
        }
        // Case 2: Two children - get inorder successor
        Node* temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

int main() {
    int n;
    scanf("%d", &n);

    Node* root = NULL;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        root = insert(root, val);
    }

    int q;
    scanf("%d", &q);

    printf("Before deletion of key value:\n");
    inorder(root);
    printf("\n");

    root = deleteNode(root, q);

    printf("After deleting key value:\n");
    inorder(root);
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Revi is studying binary search trees (BSTs) and wants to explore all the

unique BSTs that can be constructed using distinct integers from 1 to n.
He decides to write a program that generates all such BSTs and prints their preorder traversals.

Write a program to ensure that the preorder traversal of all possible BSTs constructed from numbers 1 to n is printed, each on a new line.

Example

Input

3

Output

1 2 3

1 3 2

2 1 3

3 1 2

3 2 1

Explanation

Input Format

The input consists of a single integer n, representing the number of nodes.

Output Format

Each line contains the preorder traversal (space-separated) of one possible BST using values from 1 to n.

Each traversal is printed on a new line.

There should be no blank lines between the outputs, and the order of output may vary depending on the internal recursive tree generation.

24uecs2047

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 1 2 3
1 3 2
2 1 3
3 1 2
3 2 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* newNode(int val) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

void preorder(Node* root, int* first) {
    if (root == NULL) return;
    if (!*first) printf(" ");
    printf("%d", root->data);
    *first = 0;
    preorder(root->left, first);
    preorder(root->right, first);
}

// Returns array of roots of all BSTs for values [start, end]
Node** constructTrees(int start, int end, int* count) {
    Node** trees = NULL;
    *count = 0;

    if (start > end) {
```

```
        trees = (Node**)malloc(sizeof(Node*));
        trees[0] = NULL;
        *count = 1;
        return trees;
    }

    for (int i = start; i <= end; i++) {
        int leftCount, rightCount;
        Node** leftTrees = constructTrees(start, i - 1, &leftCount);
        Node** rightTrees = constructTrees(i + 1, end, &rightCount);

        for (int j = 0; j < leftCount; j++) {
            for (int k = 0; k < rightCount; k++) {
                Node* root = newNode(i);
                root->left = leftTrees[j];
                root->right = rightTrees[k];

                (*count)++;
                trees = (Node**)realloc(trees, (*count) * sizeof(Node*));
                trees[*count - 1] = root;
            }
        }
        free(leftTrees);
        free(rightTrees);
    }
    return trees;
}

void freeTree(Node* root) {
    if (!root) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    int n;
    scanf("%d", &n);

    int count;
    Node** allTrees = constructTrees(1, n, &count);
```

```
    for (int i = 0; i < count; i++) {
        int first = 1;
        preorder(allTrees[i], &first);
        printf("\n");
    }

    // cleanup
    for (int i = 0; i < count; i++) freeTree(allTrees[i]);
    free(allTrees);

    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

3. Problem Statement

Jai is fascinated by binary trees and wants to create and manipulate them. He needs your help to implement basic operations like insertion and deletion in a binary search tree.

Given a sequence of characters representing the values to be inserted into a binary search tree, followed by a character V to be deleted from the tree, your task is to construct the binary search tree and delete the specified character from it.

Your program should perform the following operations:

Construct a binary search tree from the given sequence of characters.Delete the specified character V from the constructed tree.Print the elements of the tree in sorted order after deletion.

Input Format

The first line contains an integer N, denoting the number of characters in the sequence.

The second line contains N space-separated characters, representing the sequence of characters to be inserted into the binary search tree.

The third line contains a single character V (which is guaranteed to be present in

the tree), representing the value to be deleted from the binary search tree.

Output Format

The output prints a single line containing the elements of the binary search tree in sorted order after deleting the specified character V.

If the specified value is not available in the tree, print the given input values in order traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
F B G A D
B

Output: A D F G

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node {
    char data;
    struct Node *left, *right;
} Node;

Node* newNode(char val) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

Node* insert(Node* root, char val) {
    if (root == NULL) return newNode(val);
    if (val < root->data) root->left = insert(root->left, val);
    else if (val > root->data) root->right = insert(root->right, val);
```

```
    // duplicates not inserted per BST property
    return root;
}

void inorder(Node* root, int* first) {
    if (root == NULL) return;
    inorder(root->left, first);
    if (!*first) printf(" ");
    printf("%c", root->data);
    *first = 0;
    inorder(root->right, first);
}

Node* minValueNode(Node* node) {
    while (node && node->left!= NULL) node = node->left;
    return node;
}

// Returns true if key was found and deleted
Node* deleteNode(Node* root, char key, bool* deleted) {
    if (root == NULL) return root;

    if (key < root->data) {
        root->left = deleteNode(root->left, key, deleted);
    } else if (key > root->data) {
        root->right = deleteNode(root->right, key, deleted);
    } else {
        *deleted = true; // found the key
        // Node with 0 or 1 child
        if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
        }
        // Node with 2 children: get inorder successor
        Node* temp = minValueNode(root->right);
        root->data = temp->data;
        bool dummy = false;
```

```
        root->right = deleteNode(root->right, temp->data, &dummy);
    }
    return root;
}

int main() {
    int N;
    scanf("%d", &N);

    Node* root = NULL;
    for (int i = 0; i < N; i++) {
        char ch;
        scanf(" %c", &ch); // space before %c skips whitespace
        root = insert(root, ch);
    }

    char V;
    scanf(" %c", &V);

    bool deleted = false;
    root = deleteNode(root, V, &deleted);

    int first = 1;
    inorder(root, &first);
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement:

Arun is exploring operations on binary search trees (BST). He wants to write a program that takes an unsorted array of distinct integers and constructs a BST by inserting elements in the given order (not height-balanced).

After constructing the BST, he wants to delete all nodes at the deepest level of the tree and perform a post-order traversal to print the remaining

nodes.

Example

Input:

7

1 7 4 9 2 11 12

Output:

Before deleting: 2 4 12 11 9 7 1

After deleting: 2 4 11 9 7 1

Explanation:

The BST is constructed by inserting elements in the given order: 1, 7, 4, 9, 2, 11, 12. This creates a regular BST (not height-balanced) where each element is inserted according to BST properties.

The tree is traversed in post-order before deleting the deepest level, showing: 2, 4, 12, 11, 9, 7, 1.

The deepest level nodes (in this case, node 12) are then deleted.

The tree is traversed in post-order after deleting the deepest level, showing: 2, 4, 11, 9, 7, 1.

Input Format

The first line contains an integer n, representing the number of nodes to be inserted in the BST.

The second line contains n space-separated integers, representing the elements to be inserted into the BST in the given order.

Output Format

The first line prints "Before deleting: " followed by the space-separated elements of the tree in post-order traversal before deleting the deepest level, ending with a space and newline.

24uecs2047

The second line prints "After deleting: " followed by the space-separated elements of the tree in post-order traversal after deleting the deepest level, ending with a space and newline.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
1 7 4 9 2 11 12
Output: Before deleting: 2 4 12 11 9 7 1
After deleting: 2 4 11 9 7 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
   int data;
   struct Node *left, *right;
} Node;

Node* newNode(int val) {
   Node* node = (Node*)malloc(sizeof(Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}

Node* insert(Node* root, int val) {
   if (root == NULL) return newNode(val);
   if (val < root->data) root->left = insert(root->left, val);
   else root->right = insert(root->right, val);
   return root;
}

void postorder(Node* root, int* first) {
   if (root == NULL) return;
```

```
    postorder(root->left, first);
    postorder(root->right, first);
    if (!*first) printf(" ");
    printf("%d", root->data);
    *first = 0;
}

int maxDepth(Node* root) {
    if (root == NULL) return 0;
    int l = maxDepth(root->left);
    int r = maxDepth(root->right);
    return (l > r? l : r) + 1;
}

// Delete nodes at given depth. Returns new root after deletion.
Node* deleteLevel(Node* root, int level, int target) {
    if (root == NULL) return NULL;

    // If we are at target level, delete this node
    if (level == target) {
        free(root);
        return NULL;
    }

    // Otherwise recurse deeper
    root->left = deleteLevel(root->left, level + 1, target);
    root->right = deleteLevel(root->right, level + 1, target);
    return root;
}

int main() {
    int n;
    scanf("%d", &n);

    Node* root = NULL;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        root = insert(root, val);
    }

    printf("Before deleting: ");
```

```
    int first = 1;
    postorder(root, &first);
    printf(" \n");

    int depth = maxDepth(root);
    root = deleteLevel(root, 1, depth); // root is level 1

    printf("After deleting: ");
    first = 1;
    postorder(root, &first);
    printf(" \n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 8_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. Which data structure is used in Breadth First Search?

Answer

Queue

Status : Correct ***Marks : 1/1***

2. Which of the following problems can be solved by using both BFS & DFS?

Answer

Finding the shortest path between two vertices in an unweighted graph

Status : Correct ***Marks : 1/1***

24uecs2047

3. In BFS, what happens when a node is visited for the second time?

Answer

It is skipped and the traversal continues to the next unvisited node

Status : Correct ***Marks : 1/1***

4. Given two vertices in a graph s and t, which of the two traversals (BFS and DFS) can be used to find if there is path from s to t?

Answer

Both BFS and DFS

Status : Correct ***Marks : 1/1***

5. What will be the result of depth-first traversal in the following tree?

Answer

1 2 4 5 3

Status : Correct ***Marks : 1/1***

6. The Breadth First Search (BFS) algorithm has been implemented using the queue data structure. Which one of the following is a possible order of visiting the nodes in the graph below?

Answer

POQNMR

Status : Correct ***Marks : 1/1***

7. In the _____ traversal, we process all of a vertex's descendants before we move to an adjacent vertex.

Answer

24uecs2047

Depth First

Status : Correct ***Marks : 1/1***

8. Which of the following conditions is sufficient to detect a cycle in a directed graph?

Answer

There is an edge from the currently visited node to an ancestor of the currently visited node in the DFS forest.

Status : Correct ***Marks : 1/1***

9. Regarding the implementation of Breadth First Search using queues, what is the maximum distance between two nodes present in the queue? (considering each edge length 1)

Answer

At most 1

Status : Correct ***Marks : 1/1***

10. Given the following adjacency matrix of a graph(G) determine the number of components in the G.

[0 1 1 0 0 0],

[1 0 1 0 0 0],

[1 1 0 0 0 0],

[0 0 0 0 1 0],

[0 0 0 1 0 0],

[0 0 0 0 0 0].

Answer

3

Status : Correct ***Marks : 1/1***

11. Which of the following statements is true about DFS?

Answer

DFS can be used to detect cycles in a graph.

Status : Correct ***Marks : 1/1***

12. Consider the following graph.

Apply breadth-first search traversal of the above graph. Which of the following traversal is possible if the start vertex is G? (Assume lexicographic ordering)

Answer

GAHIJBCEKD

Status : Wrong ***Marks : 0/1***

13. Which of the following is a possible result of depth-first traversal of the given graph(consider 1 to be the source element)?

Answer

1 2 3 4 5

Status : Correct ***Marks : 1/1***

14. In an adjacency list representation of a graph, what does each node in the list represent?

Answer

Edges

Status : Wrong ***Marks : 0/1***

15. The number of elements in the adjacency matrix of a graph having 7

24uecs2047

vertices is _____.

Answer

49

Status : Correct ***Marks : 1/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 8_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Sanjay is writing a program to manage an undirected graph using an adjacency list representation. Users will initially input the vertices to create the edges. The program will display the adjacency list of the graph.

Assist Sanjay in creating the program.

Input Format

The input consists of 5 nodes.

Edge information: ab ae bc bd be cd de.

Output Format

The output prints the adjacency list representation of the graph.

24uecs2047

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 2 3 4

Output: vertex 0: head -> 4 -> 1
vertex 1: head -> 4 -> 3 -> 2 -> 0
vertex 2: head -> 3 -> 1
vertex 3: head -> 4 -> 2 -> 1
vertex 4: head -> 3 -> 1 -> 0

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Structure for a node in the adjacency list
struct Node {
    int dest;
    struct Node* next;
};

// Structure for the adjacency list
struct AdjList {
    struct Node* head;
};

// Function to create a new node
struct Node* createNode(int dest) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

// Function to add an edge to an undirected graph
void addEdge(struct AdjList* adj, int u, int v) {
    // Add v to u's list
    struct Node* newNode = createNode(v);
    newNode->next = adj[u].head;
```

```
    adj[u].head = newNode;

    // Add u to v's list (Undirected)
    newNode = createNode(u);
    newNode->next = adj[v].head;
    adj[v].head = newNode;
}

int main() {
    int vertices[5];
    struct AdjList adj[11]; // Max node value is 10 per constraints

    // Initialize heads as NULL
    for (int i = 0; i <= 10; i++) {
        adj[i].head = NULL;
    }

    // Read the 5 nodes
    for (int i = 0; i < 5; i++) {
        if (scanf("%d", &vertices[i]) != 1) return 0;
    }

    // Based on the prompt description, we add the specific edges:
    // ab ae bc bd be cd de
    // Using the input order to determine a, b, c, d, e
    int a = vertices[0], b = vertices[1], c = vertices[2], d = vertices[3], e = vertices[4];

    addEdge(adj, a, b); // ab
    addEdge(adj, a, e); // ae
    addEdge(adj, b, c); // bc
    addEdge(adj, b, d); // bd
    addEdge(adj, b, e); // be
    addEdge(adj, c, d); // cd
    addEdge(adj, d, e); // de

    // Print the adjacency list for each vertex in the given order
    // Note: Sample output shows vertices sorted by value (0 to 4)
    for (int i = 0; i <= 10; i++) {
        // Check if this vertex was part of the input nodes
        int exists = 0;
        for(int k=0; k<5; k++) if(vertices[k] == i) exists = 1;
```

```
        if (exists) {
            printf("vertex %d: head", i);
            struct Node* temp = adj[i].head;
            while (temp) {
                printf(" -> %d", temp->dest);
                temp = temp->next;
            }
            printf("\n");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Sophia is designing a transportation network for a city with multiple interconnected districts. Each district is represented as a vertex, and every road connecting two districts is represented as a directed edge in a graph.

To evaluate connectivity, Sophia needs a program that performs a Breadth-First Search (BFS) traversal of the city's network starting from a specified source district, and outputs the order in which districts are visited.

Write a program to ensure that Sophia can perform the BFS traversal correctly and view the order of visited districts.

Input Format

The first line of input consists of two space-separated integers, v, and e, representing the total number of cities and the number of connections between cities.

The next e lines each contain two space-separated integers a and b, representing a directed edge from city a to city b.

The last line consists of a single integer s, representing the source city from which BFS traversal should start.

Output Format

The output prints the order in which the cities are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 7
0 1
0 2
1 3
1 4
2 4
3 5
4 5
0

Output: 0 1 2 3 4 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_V 100

// Simple queue implementation for BFS
int queue[MAX_V];
int front = 0, rear = 0;

void enqueue(int v) {
    queue[rear++] = v;
}

int dequeue() {
    return queue[front++];
}
```

```
bool isQueueEmpty() {
    return front == rear;
}

void bfs(int adj[MAX_V][MAX_V], int v, int start, int num_v) {
    bool visited[MAX_V] = {false};

    // Start with the source city
    visited[start] = true;
    enqueue(start);

    while (!isQueueEmpty()) {
        int current = dequeue();
        printf("%d ", current);

        // Check all possible connections from the current city
        for (int i = 0; i < num_v; i++) {
            if (adj[current][i] == 1 && !visited[i]) {
                visited[i] = true;
                enqueue(i);
            }
        }
    }
}

int main() {
    int v, e;
    if (scanf("%d %d", &v, &e) != 2) return 0;

    // Adjacency matrix to represent directed edges
    int adj[MAX_V][MAX_V] = {0};

    for (int i = 0; i < e; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        adj[a][b] = 1; // Directed edge from a to b
    }

    int source;
    scanf("%d", &source);

    bfs(adj, v, source, v);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Liam is developing a social media analytics tool for tracking user interactions. He needs to identify the connected components of users in the network by traversing a directed graph. Each user is represented as a vertex, and the connections between users are represented as directed edges. Liam's task is to implement a Depth-First Search (DFS) algorithm to perform a traversal of the network, starting from a given user's profile.

Can you help Liam by implementing the DFS algorithm to identify all connected users starting from a specified user profile?

Example

Input:

6 6

0 1

0 2

1 2

2 0

2 3

3 3

2

Output:

2 0 1 3

Explanation:

Starting from vertex 2, it visits and marks vertex 2 as visited.

Vertex 2 is printed.

Checking the neighbors of vertex 2: 0 and 3.

Vertex 0 is visited, so DFS is called with vertex 0.

Vertex 0 is visited and printed.

Vertex 1 is visited and printed.

Vertex 3 is visited and printed.

The final output is "2 0 1 3".

Input Format

The first line contains two space-separated integers n and m, representing the total number of users (vertices) and the number of connections (edges) in the network, respectively.

The next m lines contain two space-separated integers u and v, representing a directed connection from user u to user v.

The last line contains a single integer s, representing the ID of the source user profile from which the DFS traversal should start.

Output Format

The output displays a single line containing the user IDs in the order they were visited during the DFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 6
0 1
0 2
1 2
2 0
2 3
3 3

24uecs2047

2
Output: 2 0 1 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAX_USERS 100

// Function to perform Depth First Search
void performDFS(int user, int n, int adj[MAX_USERS][MAX_USERS], bool visited[])
{
    // Mark the current user as visited and print their ID
    visited[user] = true;
    printf("%d ", user);

    // Look for all users connected to the current user
    for (int neighbor = 0; neighbor < n; neighbor++) {
        // If there is a directed connection and the neighbor hasn't been visited
        if (adj[user][neighbor] == 1 && !visited[neighbor]) {
            performDFS(neighbor, n, adj, visited);
        }
    }
}

int main() {
    int n, m;
    // Read total users (vertices) and connections (edges)
    if (scanf("%d %d", &n, &m) != 2) return 0;

    // Use an adjacency matrix to represent the directed graph
    int adj[MAX_USERS][MAX_USERS] = {0};

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        // Directed edge from user u to user v
        if (u < n && v < n) {
            adj[u][v] = 1;
        }
    }
```

```
    int startUser;
    // Read the source user ID to start traversal
    scanf("%d", &startUser);

    bool visited[MAX_USERS] = {false};

    // Begin DFS from the source user
    if (startUser < n) {
        performDFS(startUser, n, adj, visited);
    }

    printf("\n");
    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Given a graph with V vertices numbered from 0 to V-1 and a list of edges, determine whether the graph is bipartite or not.

Note: A bipartite graph is a type of graph where the set of vertices can be divided into two disjoint sets, say U and V, such that every edge connects a vertex in U to a vertex in V, there are no edges between vertices within the same set.

Example:

Input: V = 4, edges[][]= [[0, 1], [0, 2], [1, 2], [2, 3]]

Output: No, the given graph is not Bipartite

Explanation

The graph is not bipartite because no matter how we try to color the nodes using two colors, there exists a cycle of odd length (like 1–2–0–1), which leads to a situation where two adjacent nodes end up with the same color. This violates the bipartite condition, which requires that no two connected

nodes share the same color.

Input: V = 4, edges[][] = [[0, 1], [1, 2], [2, 3]]

Output: Yes, the given graph is Bipartite

Explanation:

The given graph can be colored in two colors so, it is a bipartite graph.

Input Format

The first line contains two space-separated integers V and E, representing the number of vertices and edges respectively.

The next E lines each contain two space-separated integers u and v, indicating an undirected edge between vertices u and v.

Output Format

The output print "Yes, the given graph is Bipartite" if the graph is bipartite.

Otherwise, print "No, the given graph is not Bipartite".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4 3
0 1
1 2
2 3
Output: Yes, the given graph is Bipartite

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAXV 10
```

```
// Function to check if the graph is Bipartite using BFS
bool isBipartite(int adj[MAXV][MAXV], int V) {
    // color array: -1 means uncolored, 0 and 1 are the two colors
    int color[MAXV];
    for (int i = 0; i < V; i++) {
        color[i] = -1;
    }

    // Loop through all vertices to handle disconnected graphs
    for (int i = 0; i < V; i++) {
        if (color[i] == -1) {
            // BFS starting from vertex i
            int queue[MAXV];
            int front = 0, rear = 0;

            color[i] = 0;
            queue[rear++] = i;

            while (front < rear) {
                int u = queue[front++];

                for (int v = 0; v < V; v++) {
                    // If there is an edge between u and v
                    if (adj[u][v] == 1) {
                        // If neighbor v is not colored
                        if (color[v] == -1) {
                            color[v] = 1 - color[u];
                            queue[rear++] = v;
                        }
                        // If neighbor v has the same color as u
                        else if (color[v] == color[u]) {
                            return false;
                        }
                    }
                }
            }
        }
    }
    return true;
}

int main() {
```

```
    int V, E;
    if (scanf("%d %d", &V, &E) != 2) return 0;

    int adj[MAXV][MAXV] = {0};

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        // Undirected graph edges
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    if (isBipartite(adj, V)) {
        printf("Yes, the given graph is Bipartite\n");
    } else {
        printf("No, the given graph is not Bipartite\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

David is working on a project that involves traversing graphs for analyzing network connectivity. He is tasked with performing a Depth-First Search (DFS) traversal on a connected, undirected graph to explore all its vertices starting from a given node.

He needs help writing a program that can process multiple test cases where each test case describes a graph with its vertices and edges, and the program should output the DFS traversal order for each graph.

Can you assist David by writing a program that performs the DFS traversal for each test case?

Example

Input:

2

5 4

0 1 0 2 0 3 2 4

4 3

0 1 1 2 0 3

Output:

0 1 2 4 3

0 1 2 3

Explanation:

Visit all the vertices using a depth-first search algorithm using 0 as a source node.

Input Format

The first line of input contains an integer T denoting the number of test cases.

Each test case consists of two lines.

- The first line of each test case contains two integers N and E which denote the number of vertices and number of edges respectively.
- The second line of each test case contains E space-separated pairs u and v denoting that there is an edge from u to v.

Output Format

For each test case, the output displays the graph vertices in the depth-first traversal order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2

24uecs2047

5 4
0 1 0 2 0 3 2 4
4 3
0 1 1 2 0 3

Output: 0 1 2 4 3
0 1 2 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Structure for a node in the adjacency list
struct Node {
    int data;
    struct Node* next;
};

// Structure for the adjacency list array
struct Graph {
    int numVertices;
    struct Node** adjLists;
    bool* visited;
};

// Create a new node
struct Node* createNode(int v) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = v;
    newNode->next = NULL;
    return newNode;
}

// Create a graph
struct Graph* createGraph(int vertices) {
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct Node*));
    graph->visited = malloc(vertices * sizeof(bool));

    for (int i = 0; i < vertices; i++) {
```

```
        graph->adjLists[i] = NULL;
        graph->visited[i] = false;
    }
    return graph;
}

// Add an edge (Undirected)
// Note: To match sample output, we append to the end of the list
// to maintain input-based neighbor processing order.
void addEdge(struct Graph* graph, int u, int v) {
    // Add v to u's list
    struct Node* newNode = createNode(v);
    if (graph->adjLists[u] == NULL) {
        graph->adjLists[u] = newNode;
    } else {
        struct Node* temp = graph->adjLists[u];
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    // Add u to v's list
    newNode = createNode(u);
    if (graph->adjLists[v] == NULL) {
        graph->adjLists[v] = newNode;
    } else {
        struct Node* temp = graph->adjLists[v];
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }
}

// DFS Traversal
void DFS(struct Graph* graph, int vertex) {
    graph->visited[vertex] = true;
    printf("%d ", vertex);

    struct Node* adjList = graph->adjLists[vertex];
    while (adjList != NULL) {
        int connectedVertex = adjList->data;
        if (!graph->visited[connectedVertex]) {
            DFS(graph, connectedVertex);
        }
```

```
        adjList = adjList->next;
    }
}

// Helper function to free graph memory
void freeGraph(struct Graph* graph) {
    for (int i = 0; i < graph->numVertices; i++) {
        struct Node* temp = graph->adjLists[i];
        while (temp) {
            struct Node* toFree = temp;
            temp = temp->next;
            free(toFree);
        }
    }
    free(graph->adjLists);
    free(graph->visited);
    free(graph);
}

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;

    while (T--) {
        int N, E;
        if (scanf("%d %d", &N, &E) != 2) break;

        struct Graph* graph = createGraph(N);

        for (int i = 0; i < E; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            addEdge(graph, u, v);
        }

        DFS(graph, 0); // Start DFS from source node 0
        printf("\n");

        freeGraph(graph);
    }

    return 0;
```

}

Status : Correct ***Marks : 10/10***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an M × N matrix of characters, find the length of the longest path in the matrix starting from a given character.

In alphabetical order, all characters in the longest path should be increasing and consecutive. We are allowed to search the string in all eight possible directions, i.e., North, West, South, East, North-East, North-West, South-East, and South-West.

Use depth-first search (DFS) to solve this problem.

For example, consider the following matrix of characters:

The length of the longest path with consecutive characters starting from

24uecs2047

character C is 6.

The longest path is:

Input Format

The first line of input is an integer n representing the row size.

The second line of input is an integer m representing the column size.

The following lines of input are the characters of the matrix.

The last line of the input is the character for which the length of the longest path is to be found.

Output Format

The output prints "The length of the longest path with consecutive characters starting from character <startChar> is <length>"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5
d e h x b
a o g p e
d d c f d
e b e a s
c d y e n
c
Output: The length of the longest path with consecutive characters starting from character c is 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
int R, C;
char matrix[10][10];
int memo[10][10];

// Direction arrays for 8 directions
int dx[] = {-1, -1, -1, 0, 0, 1, 1, 1};
int dy[] = {-1, 0, 1, -1, 1, -1, 0, 1};

// Helper function to find maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// DFS function to find the longest consecutive path from (r, c)
int findLongestFrom(int r, int c) {
    // If already calculated, return memoized value
    if (memo[r][c] != -1) {
        return memo[r][c];
    }

    int maxLength = 1;

    // Explore all 8 directions
    for (int i = 0; i < 8; i++) {
        int nr = r + dx[i];
        int nc = c + dy[i];

        // Check boundaries and alphabetical consecutiveness
        if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
            if (matrix[nr][nc] == matrix[r][c] + 1) {
                maxLength = max(maxLength, 1 + findLongestFrom(nr, nc));
            }
        }
    }

    return memo[r][c] = maxLength;
}

int main() {
    // Read dimensions
    if (scanf("%d %d", &R, &C) != 2) return 0;
```

```
    // Read matrix characters
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            scanf(" %c", &matrix[i][j]);
        }
    }

    char startChar;
    scanf(" %c", &startChar);

    // Initialize memo table with -1
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            memo[i][j] = -1;
        }
    }

    int overallMax = 0;

    // Start DFS from every cell containing the startChar
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (matrix[i][j] == startChar) {
                overallMax = max(overallMax, findLongestFrom(i, j));
            }
        }
    }

    printf("The length of the longest path with consecutive characters starting
from character %c is %d\n", startChar, overallMax);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Priya is developing a learning module to simulate breadth-first traversal of directed graphs. Given multiple test cases where each describes a directed graph with vertices and edges, write a program to perform BFS starting

from vertex 0 and print the traversal order for each test case.

Example

Input:

1

5 4

0 1 0 2 0 3 2 4

Output:

0 1 2 3 4

Explanation:

For the given graph, the BFS is 0 1 2 3 4.

Input Format

The first line of input contains an integer T, denoting the number of test cases.

Then T test cases follow. Each test case consists of two lines.

The descriptions of the test cases are as follows:

1. The first line of each test case contains two space-separated integers, N and E, which denote the number of vertices and edges, respectively.
2. The second line of each test case contains E space-separated pairs u and v, denoting that there is an edge from u to v.

Output Format

For each test case, the output prints the BFS order of the graph vertices.

Each vertex should appear once in the order it is visited, separated by a space.

Refer to the sample input and output for formatting specifications.

24uecs2047

Sample Test Case

Input: 2
5 4
0 1 0 2 0 3 2 4
3 2
0 1 0 2

Output: 0 1 2 3 4
0 1 2

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX_N 15

void solve() {
    int N, E;
    if (scanf("%d %d", &N, &E) != 2) return;

    int adj[MAX_N][MAX_N] = {0};
    bool visited[MAX_N] = {false};

    // Read E pairs of edges
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        // Only mark if within bounds
        if (u < N && v < N) {
            adj[u][v] = 1;
        }
    }

    // BFS Queue
    int queue[MAX_N];
    int front = 0, rear = 0;

    // Start BFS from vertex 0
    visited[0] = true;
    queue[rear++] = 0;
```

```
    while (front < rear) {
        int current = queue[front++];
        printf("%d ", current);

        // Check neighbors in increasing order
        for (int neighbor = 0; neighbor < N; neighbor++) {
            if (adj[current][neighbor] == 1 && !visited[neighbor]) {
                visited[neighbor] = true;
                queue[rear++] = neighbor;
            }
        }
    }
    printf("\n");
}

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;
    while (T--) {
        solve();
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Aayush is building a network verifier that must detect the presence of cycles in an undirected graph. Given a graph with a set number of vertices and edges, write a program to determine whether the graph contains at least one cycle using depth-first traversal.

Example

Input

N = 4, E = 4

Output

Yes

Explanation

The diagram clearly shows a cycle 0 to 2 to 1 to 0

Input

N = 4, E = 3

Output

No

Explanation

There is no cycle in the given graph.

Input Format

The first line of input consists of two integers, V and E, representing the number of vertices and edges separated by space, respectively.

The next E lines contain two space-separated integers u and v, describing an edge between vertices u and v.

Output Format

The output prints "Graph contains a cycle" if a cycle exists, otherwise prints "Graph does not contain a cycle".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 5
1 0
0 2
2 1
0 3
3 4

24uecs2047

Output: Graph contains a cycle

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAX_V 10

// DFS function to detect cycle
bool isCycleUtil(int v, int adj[MAX_V][MAX_V], bool visited[], int parent, int V) {
    visited[v] = true;

    for (int i = 0; i < V; i++) {
        // If there is an edge to neighbor i
        if (adj[v][i] == 1) {
            // If neighbor is not visited, recurse
            if (!visited[i]) {
                if (isCycleUtil(i, adj, visited, v, V))
                    return true;
            }
            // If neighbor is visited and not the parent, cycle found
            else if (i != parent) {
                return true;
            }
        }
    }
    return false;
}

bool hasCycle(int adj[MAX_V][MAX_V], int V) {
    bool visited[MAX_V] = {false};

    // Check all components (for disconnected graphs)
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            if (isCycleUtil(i, adj, visited, -1, V))
                return true;
        }
    }
    return false;
}
```

```
int main() {
    int V, E;
    if (scanf("%d %d", &V, &E) != 2) return 0;

    int adj[MAX_V][MAX_V] = {0};

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        // Undirected graph edges
        if (u < V && v < V) {
            adj[u][v] = 1;
            adj[v][u] = 1;
        }
    }

    if (hasCycle(adj, V)) {
        printf("Graph contains a cycle\n");
    } else {
        printf("Graph does not contain a cycle\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Emma is developing a graph traversal feature for a social networking platform. Each user is represented by a lowercase English letter (a to z), and connections between users are represented as directed edges.

Emma wants to perform a Breadth-First Search (BFS) starting from a given user, visiting friends in lexicographical order whenever multiple connections are available.

Write a program to help Emma implement this lexicographically sorted BFS traversal.

Input:

N = 10, M = 10, S = ‘a’,

edges[][2] = { { ‘a’, ‘y’ }, { ‘a’, ‘z’ }, { ‘a’, ‘p’ }, { ‘p’, ‘c’ }, { ‘p’, ‘b’ }, { ‘y’, ‘m’ }, { ‘y’, ‘l’ }, { ‘z’, ‘h’ }, { ‘z’, ‘g’ }, { ‘z’, ‘i’ } }

Output:

a p y z b c l m g h i

Input Format

The first line contains an integer N, representing the number of nodes (users) in the graph.

The second line contains an integer M, representing the number of edges (friendships) in the graph.

The next M lines each contain two space-separated lowercase characters u and v, indicating a directed edge from user u to user v.

The last line contains an integer S, the starting node (user) for the BFS traversal.

Output Format

The output prints the users in the order they are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
10
a y
a z
a p
p c
p b

y m
y l
z h
z g
z i
a

Output: a p y z b c l m g h i

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAX 26

// Standard Queue implementation
char queue[MAX];
int front = 0, rear = 0;

void enqueue(char c) {
    queue[rear++] = c;
}

char dequeue() {
    return queue[front++];
}

bool isQueueEmpty() {
    return front == rear;
}

void bfs(int adj[MAX][MAX], char startNode) {
    bool visited[MAX] = {false};

    // Mark the start node as visited and enqueue it
    visited[startNode - 'a'] = true;
    enqueue(startNode);

    while (!isQueueEmpty()) {
        char current = dequeue();
        printf("%c ", current);
```

```
        int u = current - 'a';
        // Iterate through all possible characters in alphabetical order
        for (int v = 0; v < MAX; v++) {
            // If there's a directed edge and the user hasn't been visited
            if (adj[u][v] == 1 && !visited[v]) {
                visited[v] = true;
                enqueue(v + 'a');
            }
        }
    }
    printf("\n");
}

int main() {
    int N, M;
    if (scanf("%d %d", &N, &M) != 2) return 0;

    int adj[MAX][MAX] = {0};

    // Read M edges
    for (int i = 0; i < M; i++) {
        char u, v;
        scanf(" %c %c", &u, &v);
        adj[u - 'a'][v - 'a'] = 1;
    }

    char startNode;
    scanf(" %c", &startNode);

    bfs(adj, startNode);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. In general, backtracking can be used to solve?

Answer

Combinatorial problems

Status : Correct ***Marks : 1/1***

2. Backtracking may lead to a solution that is _____.

Answer

optimal

Status : Wrong ***Marks : 0/1***

24uecs2047

3. What is the main idea behind Backtracking to check if a graph is connected?

Answer

Trying all possible paths from a starting vertex and marking visited vertices

Status : Correct ***Marks : 1/1***

4. In Backtracking, what is the purpose of marking vertices as visited during graph traversal?

Answer

To avoid revisiting the same vertex

Status : Correct ***Marks : 1/1***

5. In Backtracking, what is the base case for checking connectivity in an undirected graph?

Answer

All vertices are visited

Status : Correct ***Marks : 1/1***

6. In a Sudoku solver, when does the backtracking algorithm backtrack?

Answer

When no valid number can be placed in a cell

Status : Correct ***Marks : 1/1***

7. Which of the following problems cannot be solved using backtracking?

Answer

Finding shortest path in weighted graph

Status : Wrong ***Marks : 0/1***

24uecs2047

8. Which of the following is essential for backtracking to function correctly?

Answer

A recursive call with a rollback (backtrack step)

Status : Correct ***Marks : 1/1***

9. What is a base case in a backtracking algorithm?

Answer

A condition when a solution is found or invalid

Status : Correct ***Marks : 1/1***

10. Which data structure is commonly used in backtracking to store partial solutions?

Answer

Stack or List

Status : Correct ***Marks : 1/1***

11. In the Hamiltonian Cycle problem using backtracking, what must be true for a path to be valid?

Answer

All vertices are included once, and the last connects to the first

Status : Correct ***Marks : 1/1***

12. What is the primary technique used to solve the Rat in a Maze problem?

Answer

Backtracking

24uecs2047

Status : Correct ***Marks : 1/1***

13. Which of the following best describes the backtracking approach to generating permutations of a string?

Answer

Fix one character, recursively permute the rest, and backtrack

Status : Correct ***Marks : 1/1***

14. In backtracking solutions for maze problems, when do we backtrack from a cell?

Answer

When all directions from the current cell are either visited or blocked

Status : Correct ***Marks : 1/1***

15. Which of the following constraints are typically used in Sudoku backtracking?

Answer

All the mentioned options

Status : Correct ***Marks : 1/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 9_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers and a sum B, find all unique combinations in the array where the sum is equal to B. The same number may be chosen from the array any number of times to make B.

1. All numbers will be positive integers.

2. Elements in a combination (a1, a2, …, ak) must be in non-descending order. (ie, a1 ≤ a2 ≤ … ≤ ak).

3. The combinations themselves must be sorted in ascending order.

Examples:

Input:

24uecs2047

arr[] = {7,2,6,5}

B = 16

Output:

(2 2 2 2 2 2 2 2)

(2 2 2 2 2 6)

(2 2 2 5 5)

(2 2 5 7)

(2 2 6 6)

(2 7 7)

(5 5 6)

Input:

arr[] = {6,5,7,1,8,2,9,9,7,7,9}

B = 6

Output:

(1 1 1 1 1 1)

(1 1 1 1 2)

(1 1 2 2)

(1 5)

(2 2 2)

(6)

Input Format

The first line of input consists of space-separated positive integers representing the elements of the array.

The second line consists of a single positive integer representing the target sum B.

Output Format

The output prints "Combinations that sum to [B] are:" followed by the combinations in parentheses, each combination separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7 2 6 5
16
Output: Combinations that sum to 16 are:
(2 2 2 2 2 2 2 2) (2 2 2 2 2 6) (2 2 2 5 5) (2 2 5 7) (2 2 6 6) (2 7 7) (5 5 6)

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

void findCombinations(int* arr, int n, int target, int index, int* current, int currSize)
{
    // If target is reached
    if (target == 0) {
        printf("( ");
        for (int i = 0; i < currSize; i++) {
            printf("%d ", current[i]);
        }
        printf(") ");
        return;
    }

    // Explore candidates from the current index to allow reuse and maintain order
    for (int i = index; i < n; i++) {
        if (target - arr[i] >= 0) {
            current[currSize] = arr[i];
            findCombinations(arr, n, target - arr[i], i, current, currSize + 1);
        }
    }
}

// Comparison function for qsort
int compare(const void* a, const void* b) {
```

```
    return (*(int*)a - *(int*)b);
}

int main() {
    int tempArr[100];
    int n = 0;
    char ch;

    // Reading space-separated integers
    while (scanf("%d%c", &tempArr[n], &ch) >= 1) {
        n++;
        if (ch == '\n' || ch == '\r') break;
    }

    int target;
    scanf("%d", &target);

    // 1. Sort the input
    qsort(tempArr, n, sizeof(int), compare);

    // 2. Remove duplicates to ensure unique combinations
    int uniqueArr[100];
    int m = 0;
    for (int i = 0; i < n; i++) {
        if (i == 0 || tempArr[i] != tempArr[i - 1]) {
            uniqueArr[m++] = tempArr[i];
        }
    }

    int current[100];
    printf("Combinations that sum to %d are:\n", target);
    findCombinations(uniqueArr, m, target, 0, current, 0);
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you're helping a friend plan gift packages with a total budget of N. Your task is to find all unique ways to spend the entire budget on different combinations of gift prices, ensuring no combination is repeated by rearranging the prices.

Note:

By unique, it means that no matter that no other composition can be expressed as a permutation of the generated composition. For eg. [1,2,1] and [1, 1, 2] are not unique.You need to print all combinations in non-decreasing order, for eg. [1, 2, 1] or [1,1,2] will be printed as [1,1,2].The order of printing all the combinations should be in lexicographical order.

Use backtracking to solve the given problem.

Input Format

The input consists of a single integer, n, which represents the number to be broken down.

Output Format

The output consists of a list of lists, where each inner list represents a possible breakdown of the given number.

Each breakdown is represented as a list of integers that add up to the given number.

The breakdowns are printed in lexicographic order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
Output: [[1]]

Answer

```
// You are using GCC
#include <stdio.h>
```

```
void findGiftCombinations(int remaining, int minPrice, int path[], int pathSize, int
*isFirst) {
    // Base Case: Budget fully spent
    if (remaining == 0) {
        if (!(*isFirst)) {
            printf(", ");
        }
        printf("[");
        for (int i = 0; i < pathSize; i++) {
            printf("%d%s", path[i], (i == pathSize - 1 ? "" : ", "));
        }
        printf("]");
        *isFirst = 0;
        return;
    }

    // Try every possible price starting from the last added price
    for (int price = minPrice; price <= remaining; price++) {
        path[pathSize] = price;
        findGiftCombinations(remaining - price, price, path, pathSize + 1, isFirst);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    // Constraints: 1 <= N <= 25
    int path[25];
    int isFirst = 1;

    printf("[");
    findGiftCombinations(n, 1, path, 0, &isFirst);
    printf("]\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rohan is designing a security system that generates all possible rearrangements of a given access code made up of distinct characters. He inputs a string representing the access code into the system. The system must display every possible permutation of the characters without repetition.

Write a program to ensure that all permutations of the given string are printed in separate lines.

Input Format

The input consists of a single line containing a string S, consisting of lowercase or upper-case alphabets.

Output Format

The output prints each permutation of the string on a new line.

The permutations should be printed in the order generated by swapping characters starting from the first position

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: ABC

Output: ABC
ACB
BAC
BCA
CBA
CAB

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

// Function to swap two characters
void swap(char *x, char *y) {
```

```
    char temp = *x;
    *x = *y;
    *y = temp;
}

// Recursive function to generate permutations
void generatePermutations(char *s, int left, int right) {
    // Base case: if left index reaches right, one permutation is complete
    if (left == right) {
        printf("%s\n", s);
    } else {
        for (int i = left; i <= right; i++) {
            // Swap the current character with the character at index 'i'
            swap((s + left), (s + i));

            // Recurse for the rest of the string
            generatePermutations(s, left + 1, right);

            // Backtrack: swap back to restore the original string
            swap((s + left), (s + i));
        }
    }
}

int main() {
    char s[10];
    if (scanf("%s", s) != 1) return 0;

    int n = strlen(s);

    // Call the recursive function starting from index 0 to n-1
    generatePermutations(s, 0, n - 1);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given a number K and string str of digits denoting a positive integer, build

the largest number possible by performing swap operations on the digits of str at most K times.

Input Format

The first line of input consists of a positive integer K.

The second line of input consists of a string of digits of positive integers.

Output Format

The output displays the largest number possible by performing swap operations on the digits of str at most K times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
254

Output: 524

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

char maxStr[100];

// Helper to update the global maximum string
void updateMax(char* current) {
   if (strcmp(current, maxStr) > 0) {
      strcpy(maxStr, current);
   }
}

void findLargest(char* str, int k, int index) {
   // Base case: no more swaps or end of string reached
   if (k == 0 || index >= strlen(str)) {
      return;
   }
```

```
    int n = strlen(str);
    char maxDigit = str[index];

    // Find the maximum digit available in the remaining string
    for (int i = index + 1; i < n; i++) {
        if (str[i] > maxDigit) {
            maxDigit = str[i];
        }
    }

    // If current digit is already the maximum, move to next index without
swapping
    if (maxDigit == str[index]) {
        findLargest(str, k, index + 1);
    } else {
        // Try swapping current index with every instance of maxDigit found later
        for (int i = index + 1; i < n; i++) {
            if (str[i] == maxDigit) {
                // Swap
                char temp = str[index];
                str[index] = str[i];
                str[i] = temp;

                // Update global maximum
                updateMax(str);

                // Recurse with k-1 swaps
                findLargest(str, k - 1, index + 1);

                // Backtrack: swap back
                temp = str[index];
                str[index] = str[i];
                str[i] = temp;
            }
        }
    }
}

int main() {
    int k;
    char str[100];
```

```
    if (scanf("%d", &k) != 1) return 0;
    if (scanf("%s", str) != 1) return 0;

    strcpy(maxStr, str);
    findLargest(str, k, 0);

    printf("%s\n", maxStr);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Mythili has a list of item prices and a customer's gift card balance. She needs to find a combination of two or more items whose total price exactly matches the gift card balance. If such a combination exists, she should print the item prices in the order they appear. If multiple valid combinations exist, she must print the first one found. If no valid combination is possible, print "No combination of items"

Write a function that takes the list of item prices, the number of items, and the gift card balance as input, and prints the result based on the above rules.

Input Format

The first line of input consists of an integer, representing the number of items in the store.

The second line consists of an array of integers separated by a space, representing the prices of the items.

The last line consists of an integer representing the customer's gift card balance.

Output Format

If there exists a combination of items whose prices sum up to the gift card balance, then print this combination.

If no such combination exists, print "No combination of items".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
50

Output: 20 30

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

int prices[10];
int path[10];
int n, balance;
bool found = false;

void findCombination(int index, int currentSum, int count) {
    // If we already found a combination, stop further recursion
    if (found) return;

    // Base Case: check if current sum matches balance
    if (currentSum == balance) {
        if (count >= 2) {
            found = true;
            for (int i = 0; i < count; i++) {
                printf("%d%s", path[i], (i == count - 1 ? "" : " "));
            }
            printf("\n");
        }
        return;
    }

    // If sum exceeded or no more items to check
    if (currentSum > balance || index == n) {
```

```
        return;
    }

    // Option 1: Include the current item
    path[count] = prices[index];
    findCombination(index + 1, currentSum + prices[index], count + 1);

    // Option 2: Skip the current item (Backtrack)
    findCombination(index + 1, currentSum, count);
}

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &prices[i]);
    }

    scanf("%d", &balance);

    findCombination(0, 0, 0);

    if (!found) {
        printf("No combination of items\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 9_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

24uecs2047

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

If solutions exist: Print each valid board configuration, where "Q" represents a queen and "." represents an empty cell.

Each row of the board should be space-separated.

Each configuration should be followed by a blank line.

If no configuration is possible, print "No solution".

Refer to the sample output for the formatting specification.

Sample Test Case

Input: 4
0
1
Output: . Q . .
. . . Q
Q . . .
. . Q .

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
```

```
#include <stdlib.h>

int N, fixed_i, fixed_j;
int board[12]; // board[row] = col
int solutionCount = 0;

// Function to check if placing a queen at (row, col) is safe
bool isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        // board[i] == -1 means no queen placed in that row yet (shouldn't happen
here)
        if (board[i] == -1) continue;

        // Check same column or diagonals
        if (board[i] == col || abs(board[i] - col) == abs(i - row)) {
            return false;
        }
    }
    // Specifically check against the fixed queen if it's in a future row
    if (row < fixed_i) {
        if (fixed_j == col || abs(fixed_j - col) == abs(fixed_i - row)) {
            return false;
        }
    }
    return true;
}

void solve(int row) {
    if (row == N) {
        solutionCount++;
        for (int r = 0; r < N; r++) {
            for (int c = 0; c < N; c++) {
                printf("%c ", (board[r] == c) ? 'Q' : '.');
            }
            printf("\n");
        }
        printf("\n");
        return;
    }

    // If this is the row with the fixed queen, just move to the next row
    if (row == fixed_i) {
```

```
        // We already know board[fixed_i] = fixed_j is safe from initial setup
        solve(row + 1);
        return;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solve(row + 1);
            board[row] = -1; // Backtrack
        }
    }
}

int main() {
    if (scanf("%d %d %d", &N, &fixed_i, &fixed_j) != 3) return 0;

    for (int k = 0; k < N; k++) board[k] = -1;

    // Place the fixed queen
    board[fixed_i] = fixed_j;

    // Check if the fixed queen is safe (though constraints imply it will be)
    // We start the recursion from row 0
    solve(0);

    if (solutionCount == 0) {
        printf("No solution\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Consider a rat placed at (0, 0) in a square matrix of order N*N. It has to reach the destination at (N – 1, N – 1). Find all possible paths that the rat can take to reach from the source to the destination. The directions in which the rat can move are 'U'(up), 'D'(down), 'L' (left), and 'R' (right). Value

0 at a cell in the matrix represents that it is blocked and the rat cannot move to it while value 1 at a cell in the matrix represents that the rat can travel through it. Return the list of paths in lexicographically increasing order.

Note: In a path, no cell can be visited more than once. If the source cell is 0, the rat cannot move to any other cell.

Example 1

Input:

N = 4

m[][] =

{{1, 0, 0, 0},

{1, 1, 0, 1},

{1, 1, 0, 0},

{0, 1, 1, 1}}

Output:

DDRDRR DRDDRR

Explanation:

The rat can reach the destination at (3, 3) from (0, 0) by two paths - DRDDRR and DDRDRR, when printed in sorted order we get DDRDRR DRDDRR.

Example 2

Input:

N = 2

m[][] =

{{1, 0},

{1, 0}}

Output:

-1

Explanation:

No path exists and the destination cell is blocked.

Input Format

The first line of input contains an integer n, representing the size of the maze.

The next n lines each contain n space-separated integers, representing the maze.

Output Format

The output consists of

If at least one path is found, print all valid paths as space-separated strings.

If no path is found, prints -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
1 0 0 0
1 1 0 1
1 1 0 0
0 1 1 1
Output: DDRDRR DRDDRR

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int N;
int maze[10][10];
int visited[10][10];
char path[100];
```

```
int found = 0;

// Direction arrays to maintain lexicographical order: D, L, R, U
int dr[] = {1, 0, 0, -1};
int dc[] = {0, -1, 1, 0};
char dir[] = {'D', 'L', 'R', 'U'};

void solve(int r, int c, int step) {
    // Base Case: Destination reached
    if (r == N - 1 && c == N - 1) {
        path[step] = '\0';
        printf("%s ", path);
        found = 1;
        return;
    }

    // Mark current cell as visited
    visited[r][c] = 1;

    // Explore directions in order: D, L, R, U
    for (int i = 0; i < 4; i++) {
        int next_r = r + dr[i];
        int next_c = c + dc[i];

        // Check boundaries, maze value, and visited status
        if (next_r >= 0 && next_r < N && next_c >= 0 && next_c < N &&
            maze[next_r][next_c] == 1 && !visited[next_r][next_c]) {

            path[step] = dir[i];
            solve(next_r, next_c, step + 1);
        }
    }

    // Backtrack: Unmark the current cell
    visited[r][c] = 0;
}

int main() {
    if (scanf("%d", &N) != 1) return 0;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
```

```
            scanf("%d", &maze[i][j]);
            visited[i][j] = 0;
        }
    }

    // Edge case: source or destination is blocked
    if (maze[0][0] == 0 || maze[N - 1][N - 1] == 0) {
        printf("-1\n");
        return 0;
    }

    solve(0, 0, 0);

    if (!found) {
        printf("-1");
    }
    printf("\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Given a partially filled 9x9 2D array, the goal is to assign digits (from 1 to 9) to the empty cells so that every row, column, and sub-grid of size 3x3 contains exactly one instance of the digits from 1 to 9.

Write a program for the same.

Example

Input:

3 0 6 5 0 8 4 0 0

5 2 0 0 0 0 0 0 0

0 8 7 0 0 0 0 3 1

0 0 3 0 1 0 0 8 0

9 0 0 8 6 3 0 0 5

0 5 0 0 9 0 6 0 0

1 3 0 0 0 0 2 5 0

0 0 0 0 0 0 0 7 4

0 0 5 2 0 6 3 0 0

Output:

3 1 6 5 7 8 4 9 2

5 2 9 1 3 4 7 6 8

4 8 7 6 2 9 5 3 1

2 6 3 4 1 5 9 8 7

9 7 4 8 6 3 1 2 5

8 5 1 7 9 2 6 4 3

1 3 8 9 4 7 2 5 6

6 9 2 3 5 1 8 7 4

7 4 5 2 8 6 3 1 9

Explanation

To solve the given Sudoku puzzle, we apply the fundamental rules of Sudoku: each row, column, and 3x3 subgrid must contain the digits 1 through 9 without any repetition. Starting with the given partially filled grid, we systematically fill in the empty cells. For each empty cell, we check the numbers already present in the corresponding row, column, and 3x3 subgrid to determine which number can legally go in that position. This process involves logical deduction and elimination. In the case of the provided puzzle, the empty cells are filled one by one, ensuring that each row, column, and subgrid adheres to the Sudoku rules. For example, in Row 1, the missing numbers (1, 7, 9, 2) are placed in the available spots based on their placement in the columns and subgrids. This continues for each row, progressively narrowing down the possibilities for the empty cells until the entire grid is complete. The final solution, after applying this reasoning, looks like the output grid:

24uecs2047

Input Format

The input consists of a 9x9 matrix in 9 rows and 9 columns separated by space.

Output Format

The output displays the 9x9 matrix with the updated values.

If the conditions are not satisfied, then display "No Solution exists".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 0 6 5 0 8 4 0 0
5 2 0 0 0 0 0 0 0
0 8 7 0 0 0 0 3 1
0 0 3 0 1 0 0 8 0
9 0 0 8 6 3 0 0 5
0 5 0 0 9 0 6 0 0
1 3 0 0 0 0 2 5 0
0 0 0 0 0 0 0 7 4
0 0 5 2 0 6 3 0 0

Output: 3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define SIZE 9

// Function to check if it's safe to place num at grid[row][col]
```

```
bool isSafe(int grid[SIZE][SIZE], int row, int col, int num) {
    // Check row
    for (int x = 0; x < SIZE; x++) {
        if (grid[row][x] == num) return false;
    }

    // Check column
    for (int x = 0; x < SIZE; x++) {
        if (grid[x][col] == num) return false;
    }

    // Check 3x3 sub-grid
    int startRow = row - row % 3;
    int startCol = col - col % 3;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (grid[i + startRow][j + startCol] == num) return false;
        }
    }

    return true;
}

// Backtracking function to solve the Sudoku
bool solveSudoku(int grid[SIZE][SIZE]) {
    int row, col;
    bool isEmpty = false;

    // Find the first empty cell (0)
    for (row = 0; row < SIZE; row++) {
        for (col = 0; col < SIZE; col++) {
            if (grid[row][col] == 0) {
                isEmpty = true;
                break;
            }
        }
        if (isEmpty) break;
    }

    // If no empty cell is left, the Sudoku is solved
    if (!isEmpty) return true;
```

```
    // Try digits 1 to 9
    for (int num = 1; num <= 9; num++) {
        if (isSafe(grid, row, col, num)) {
            grid[row][col] = num;

            if (solveSudoku(grid)) return true;

            // If placing num doesn't lead to a solution, backtrack
            grid[row][col] = 0;
        }
    }
    return false;
}

int main() {
    int grid[SIZE][SIZE];

    // Input the 9x9 matrix
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (scanf("%d", &grid[i][j]) != 1) return 0;
        }
    }

    if (solveSudoku(grid)) {
        // Output the solved grid
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                printf("%d%s", grid[i][j], (j == SIZE - 1 ? "" : " "));
            }
            printf("\n");
        }
    } else {
        printf("No Solution exists\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given an undirected graph and a number m, determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with the same color.

Note: Here, the coloring of a graph means the assignment of colors to all vertices.

Example

Input:

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

3

Output:

Solution Exists:

1 2 3 2

Explanation:

A minimum of 3 colors is required for the above graph.

Input Format

The input consists of four lines each containing four space-separated integers (1 if connected, 0 if not), representing the adjacency matrix of a graph where each element represents the connection between vertices.

The last line consists of an integer m, representing the number of colors available.

Output Format

If a solution exists:

24uecs2047

1. The first line prints "Solution Exists:".
2. The second line prints the assigned colors for each vertex, separated by a space.

If no solution exists, the output prints "Solution does not exist".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0
3
Output: Solution Exists:
1 2 3 2

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define V 4

// Function to check if the current color assignment is safe for vertex v
bool isSafe(int v, int graph[V][V], int color[], int c) {
    for (int i = 0; i < V; i++) {
        // If there is an edge and the neighbor has the same color, return false
        if (graph[v][i] && c == color[i]) {
            return false;
        }
    }
    return true;
}
```

```
// Recursive function to solve the m-coloring problem
bool graphColoringUtil(int graph[V][V], int m, int color[], int v) {
    // Base Case: All vertices are assigned a color
    if (v == V) {
        return true;
    }

    // Try different colors for vertex v
    for (int c = 1; c <= m; c++) {
        if (isSafe(v, graph, color, c)) {
            color[v] = c;

            // Recur to assign colors to the rest of the vertices
            if (graphColoringUtil(graph, m, color, v + 1)) {
                return true;
            }

            // Backtrack: If assigning color c doesn't lead to a solution
            color[v] = 0;
        }
    }

    return false;
}

int main() {
    int graph[V][V];
    int m;

    // Reading the 4x4 adjacency matrix
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (scanf("%d", &graph[i][j]) != 1) return 0;
        }
    }

    // Reading the number of colors available
    if (scanf("%d", &m) != 1) return 0;

    int color[V];
    for (int i = 0; i < V; i++) {
        color[i] = 0;
```

```
    }

    if (graphColoringUtil(graph, m, color, 0)) {
        printf("Solution Exists:\n");
        for (int i = 0; i < V; i++) {
            printf("%d%s", color[i], (i == V - 1 ? "" : " "));
        }
        printf("\n");
    } else {
        printf("Solution does not exist\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. Which of the following sequences of array elements forms a heap?

Answer

{23, 17, 14, 7, 13, 10, 1, 5, 6, 12}

Status : Correct ***Marks : 1/1***

2. Consider a max heap, represented by the array: 23, 17, 14, 6, 13, 10, 1, 5. If a value 20 is inserted into this heap, then what will be the new max heap?

Answer

23, 20, 14, 17, 13, 10, 1, 5, 6

Status : Correct ***Marks : 1/1***

24uecs2047

3. Given a binary max heap. The elements are sorted in an array as 25, 14, 16, 13, 10, 8, 12. What is the content of the array after two delete operations?

Answer

14, 13, 12, 8, 10

Status : Correct ***Marks : 1/1***

4. In a Min Heap, what is the condition that must be satisfied between a parent and its children?

Answer

The parent must have a lower value than its children

Status : Correct ***Marks : 1/1***

5. When inserting an element with a value of 8 into a Min Heap, where would it be placed if the current heap contains the elements [10, 12, 15, 20, 17]?

Answer

As the root element

Status : Correct ***Marks : 1/1***

6. If you're inserting an element with a value of 5 into a Min Heap, and the current heap contains the elements [11, 16, 10, 13, 18], what will the Min Heap look like after the insertion?

Answer

5, 11, 10, 13, 16, 18

Status : Correct ***Marks : 1/1***

7. When deleting the root element from a Min Heap, which of the following replaces it as the new root?

Answer

the last element from the tree

Status : Correct ***Marks : 1/1***

8. Tries are mainly used for:

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

9. In a trie, how is the end of a valid word usually represented?

Answer

By using an end-of-word (terminal) marker at a node

Status : Correct ***Marks : 1/1***

10. Which of the following is true about tries?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

11. What is stored in a segment tree node for a Range Minimum Query (RMQ)?

Answer

Minimum element in range

Status : Correct ***Marks : 1/1***

12. In a Range Max Query tree, what is the internal node value?

Answer

Maximum of child values

Status : Correct ***Marks : 1/1***

13. Which segment tree operation is used for range max queries?

Answer

max() merge

Status : Correct ***Marks : 1/1***

14. What data structure is often used in segment tree nodes to count distinct elements?

Answer

Set

Status : Correct ***Marks : 1/1***

15. In range distinct count queries, what does each segment tree node contain?

Answer

Set of elements

Status : Correct ***Marks : 1/1***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 10_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 47.5

Section 1 : Coding

1. Problem Statement

Jovika is learning about heaps and wants to implement a program that constructs a max heap and inserts elements into it.

She wants to write a program that implements max heap insertion. The program defines functions to swap elements and perform maximum heapification. She wants to extend this program to take a list of integers, build a max heap, and insert a new element into the heap.

Help her in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements to be inserted into the max heap.

24uecs2047

The second line consists of N space-separated integers, representing the elements to be inserted into the max heap.

The third line consists of an integer representing the new element to be inserted into the max heap.

Output Format

The output prints the space-separated integers, representing the elements in the max heap after inserting the new element in the order of insertion.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
3 1 4 2 5
6

Output: 6 3 5 2 1 4

Answer

```
// You are using GCC
#include <stdio.h>

// Function to swap two integers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to maintain max heap property after insertion
// This moves the element at 'index' up to its correct position
void heapifyUp(int heap[], int index) {
    int parent = (index - 1) / 2;

    if (index > 0 && heap[index] > heap[parent]) {
        swap(&heap[index], &heap[parent]);
        heapifyUp(heap, parent);
    }
```

```
}

// Standard function to insert an element into the heap
void insert(int heap[], int *size, int value) {
    heap[*size] = value;
    (*size)++;
    heapifyUp(heap, *size - 1);
}

int main() {
    int n, value, size = 0;
    int heap[20]; // Constraints: N <= 15 + 1 new element

    if (scanf("%d", &n) != 1) return 0;

    // Read initial elements and build the heap incrementally
    for (int i = 0; i < n; i++) {
        int temp;
        scanf("%d", &temp);
        insert(heap, &size, temp);
    }

    // Read the new element to be inserted
    scanf("%d", &value);
    insert(heap, &size, value);

    // Print the final heap
    for (int i = 0; i < size; i++) {
        printf("%d%s", heap[i], (i == size - 1 ? "" : " "));
    }
    printf("\n");

    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

2. Problem Statement

Bob is fascinated by numbers and loves arranging them in various ways. His recent exploration led him to Max Heaps.

Bob likes odd numbers more than even numbers. After inserting elements into a Max Heap, he wants to remove all even numbers from it while keeping the Max Heap property intact.

Your task is to help Bob build a Max Heap from the given elements, display it, delete all the even numbers from the resultant array and then rearrange the elements in the form of Max Heap.

Input Format

The first line of input consists of an integer N, representing the number of elements initially to be inserted into the Max Heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the heap.

Output Format

The first line of output prints the space-separated elements of the Max Heap after building it from the given N elements.

The second line prints the space-separated elements of the Max Heap after removing even elements.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
3 2 1

Output: 3 2 1
3 1

Answer

```
// You are using GCC
#include <stdio.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
```

```
    *a = *b;
    *b = temp;
}

// Function to heapify a subtree rooted with node i
void maxHeapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        maxHeapify(arr, n, largest);
    }
}

// Function to build a Max Heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        maxHeapify(arr, n, i);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[15];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build and display the initial Max Heap
    buildMaxHeap(arr, n);
    for (int i = 0; i < n; i++) {
        printf("%d%s", arr[i], (i == n - 1 ? "" : " "));
```

```
    }
    printf("\n");

    // Step 2: Remove even numbers
    int oddArr[15];
    int oddCount = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 != 0) {
            oddArr[oddCount++] = arr[i];
        }
    }

    // Step 3: Rearrange odd elements into a Max Heap
    if (oddCount > 0) {
        buildMaxHeap(oddArr, oddCount);
        for (int i = 0; i < oddCount; i++) {
            printf("%d%s", oddArr[i], (i == oddCount - 1 ? "" : " "));
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Iniya is studying data structures and algorithms, and she is currently learning about heaps.

She wants to practice implementing Min-Heap and perform a deletion operation on it. A min-heap is a binary tree in which the value of each node is less than or equal to the values of its children.

Write a program to assist Iniya in building a min-heap and delete the root element.

Input Format

The first line of input consists of an integer N, representing the number of

elements in the initial Min-Heap.

The second line consists of N space-separated integers, where each integer represents an element in the Min-Heap.

Output Format

The output prints the space-separated elements of the Min-Heap after performing the deletion operation of the root.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
80 40 30 10 50 33 66

Output: 30 40 33 80 50 66

Answer

```
// You are using GCC
#include <stdio.h>

// Function to swap two integers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to heapify a subtree rooted at index i
void minHeapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // If left child is smaller than root
    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    // If right child is smaller than the current smallest
    if (right < n && arr[right] < arr[smallest])
```

```
        smallest = right;

    // If smallest is not root, swap and continue heapifying
    if (smallest != i) {
        swap(&arr[i], &arr[smallest]);
        minHeapify(arr, n, smallest);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int heap[15];
    for (int i = 0; i < n; i++) {
        scanf("%d", &heap[i]);
    }

    // Step 1: Build the Min-Heap
    for (int i = n / 2 - 1; i >= 0; i--) {
        minHeapify(heap, n, i);
    }

    // Step 2: Delete the root element
    // Replace root with the last element
    heap[0] = heap[n - 1];
    n--; // Reduce size

    // Restore heap property
    if (n > 0) {
        minHeapify(heap, n, 0);
    }

    // Output the resulting heap
    for (int i = 0; i < n; i++) {
        printf("%d%s", heap[i], (i == n - 1 ? "" : " "));
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

A trie (pronounced as "try") or prefix tree is a tree data structure used to store and retrieve keys in a dataset of strings efficiently. This data structure has various applications, such as autocomplete and spellchecker.

Implement the Trie class:

Trie() Initializes the trie object.void insert(String word): Inserts the string word into the trie.boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.boolean startsWith(String prefix) Returns true if a previously inserted string word has the prefix, and false otherwise.

Example

Input:

4

apple

cat

orange

mat

cat

ora

Output:

The word is present in the trie.

There is a word with the given prefix in the trie.

Input Format

The first line of the input consists of the number of words to insert, N.

The next N lines of input consist of the words in each line.

The next line of input consists of the word to search.

The last line of the input consists of the prefix to search.

Output Format

When searching for a word using the search method, the program will output one of the following messages:

- If the word is present in the trie, the output prints "The word is present in the trie."
- If the word is not present in the trie, the output prints "The word is not present in the trie."

When checking for a prefix using the startsWith method, the program will output one of the following messages:

- If there is a word in the trie with the given prefix, the output prints "There is a word with the given prefix in the trie."
- If there is no word in the trie with the given prefix, the output prints "There is no word with the given prefix in the trie."

Sample Test Case

Input: 4
apple
cat
orange
mat
catlyn
ora

Output: The word is not present in the trie.
There is a word with the given prefix in the trie.

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>
```

```
#define ALPHABET_SIZE 26

// Trie node structure
struct TrieNode {
    struct TrieNode *children[ALPHABET_SIZE];
    bool isEndOfWord;
};

// Function to create a new Trie node
struct TrieNode *createNode() {
    struct TrieNode *newNode = (struct TrieNode *)malloc(sizeof(struct
TrieNode));
    newNode->isEndOfWord = false;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        newNode->children[i] = NULL;
    }
    return newNode;
}

// Function to insert a word into the Trie
void insert(struct TrieNode *root, const char *word) {
    struct TrieNode *curr = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (!curr->children[index]) {
            curr->children[index] = createNode();
        }
        curr = curr->children[index];
    }
    curr->isEndOfWord = true;
}

// Function to search for a word in the Trie
bool search(struct TrieNode *root, const char *word) {
    struct TrieNode *curr = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (!curr->children[index]) {
            return false;
        }
        curr = curr->children[index];
```

```
    }
    return (curr != NULL && curr->isEndOfWord);
}

// Function to check if any word starts with a given prefix
bool startsWith(struct TrieNode *root, const char *prefix) {
    struct TrieNode *curr = root;
    for (int i = 0; prefix[i] != '\0'; i++) {
        int index = prefix[i] - 'a';
        if (!curr->children[index]) {
            return false;
        }
        curr = curr->children[index];
    }
    return (curr != NULL);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct TrieNode *root = createNode();
    char word[100];

    // Insert N words
    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    // Search for a specific word
    char searchWord[100];
    scanf("%s", searchWord);
    if (search(root, searchWord)) {
        printf("The word is present in the trie.\n");
    } else {
        printf("The word is not present in the trie.\n");
    }

    // Search for a prefix
    char prefix[100];
    scanf("%s", prefix);
```

```
    if (startsWith(root, prefix)) {
        printf("There is a word with the given prefix in the trie.\n");
    } else {
        printf("There is no word with the given prefix in the trie.\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Given an array of integers representing the combat power of N players.

You need to efficiently perform the following operations:

Query (Q L R): Find the maximum combat power between ranks L and R (inclusive).Update (U i val): Update the power of the player at index i to a new value val.

To ensure efficiency, implement these operations using a Segment Tree.

Input Format

The first line contains two space-separated integers, N and Q, representing the number of players and the number of operations.

The second line contains N space-separated integers representing the initial combat power scores of all players, where index 0 is the top-ranked player.

The next Q lines describe the operations:

- "Q L R" Query represents the maximum combat power from ranks L to R (inclusive).
- "U i val" Updates the power score of the player at rank i to val.

All indices are 0-based.

Output Format

For each query Q L R, print the maximum combat power in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
4 7 2 9 5 1
Q 1 4
U 3 6
Q 1 4
U 0 10
Q 0 2
Output: 9
7
10

Answer

```
// You are using GCC
#include <stdio.h>

#define MAXN 25

int tree[4 * MAXN];
int arr[MAXN];

// Utility function to get the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Build the segment tree
void build(int node, int start, int end) {
    if (start == end) {
        // Leaf node will have a single element
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        // Recursively build left and right children
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
```

```
        // Internal node will have the maximum of its children
        tree[node] = max(tree[2 * node], tree[2 * node + 1]);
    }
}

// Update the value at a specific index
void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        // Leaf node
        arr[idx] = val;
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (start <= idx && idx <= mid) {
            // If idx is in the left child, recurse left
            update(2 * node, start, mid, idx, val);
        } else {
            // if idx is in the right child, recurse right
            update(2 * node + 1, mid + 1, end, idx, val);
        }
        // Update internal node after child update
        tree[node] = max(tree[2 * node], tree[2 * node + 1]);
    }
}

// Query the maximum in range [L, R]
int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L) {
        // Range completely outside
        return -1;
    }
    if (L <= start && end <= R) {
        // Range completely inside
        return tree[node];
    }
    // Range partially inside and partially outside
    int mid = (start + end) / 2;
    int p1 = query(2 * node, start, mid, L, R);
    int p2 = query(2 * node + 1, mid + 1, end, L, R);
    return max(p1, p2);
}
```

```
int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    build(1, 0, n - 1);

    char type;
    while (q--) {
        scanf(" %c", &type);
        if (type == 'Q') {
            int l, r;
            scanf("%d %d", &l, &r);
            printf("%d\n", query(1, 0, n - 1, l, r));
        } else if (type == 'U') {
            int i, val;
            scanf("%d %d", &i, &val);
            update(1, 0, n - 1, i, val);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sai has been studying various algorithms related to heaps. Recently, he came across a problem where he needed to filter elements from a given array based on certain criteria related to heap properties.

He wants to build a max heap from a given array of integers, delete elements that fall outside the specified range, and then rearrange the elements in the form of a Max Heap.

Write a program to help Sai in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

24uecs2047

The second line consists of N space-separated integers, representing the elements of the array.

The third line consists of two integers, a and b, separated by a space, representing the range to filter the elements.

Output Format

The output prints the space-separated integers, representing the elements that remain in the max heap after removing elements outside the given range.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 7 8 3 12
1 6

Output: 5 3

Answer

```
// You are using GCC
#include <stdio.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to maintain the Max Heap property
void maxHeapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        maxHeapify(arr, n, largest);
    }
}

// Function to build a Max Heap from an arbitrary array
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        maxHeapify(arr, n, i);
    }
}

int main() {
    int n, a, b;
    int arr[20];
    int filtered[20];
    int filteredCount = 0;

    // Input the number of elements
    if (scanf("%d", &n) != 1) return 0;

    // Input the array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Input the range [a, b]
    scanf("%d %d", &a, &b);

    // Step 1: Build the initial Max Heap
    buildMaxHeap(arr, n);

    // Step 2: Filter elements outside the range [a, b]
    for (int i = 0; i < n; i++) {
        if (arr[i] >= a && arr[i] <= b) {
            filtered[filteredCount++] = arr[i];
        }
    }
```

```
    // Step 3: Rearrange the remaining elements into a Max Heap
    if (filteredCount > 0) {
        buildMaxHeap(filtered, filteredCount);

        // Print the final Max Heap elements
        for (int i = 0; i < filteredCount; i++) {
            printf("%d%s", filtered[i], (i == filteredCount - 1 ? "" : " "));
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Maxwell, an enthusiastic computer science student, is exploring the fascinating world of data structures. His task is to create a program that constructs a min heap from a list of integers and identifies negative numbers within it.

Help Maxwell in completing the program.

Input Format

The first line of input consists of an integer N, representing the number of integers in the array.

The second line consists of N space-separated integers, representing the elements of the list.

Output Format

The first line of output prints the min-heap as a space-separated list of integers.

The second line prints the negative numbers, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
15 -5 20 -19 23 42 29

Output: -19 -5 20 15 23 42 29
-19 -5

Answer

```
// You are using GCC
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void minHeapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        swap(&arr[i], &arr[smallest]);
        minHeapify(arr, n, smallest);
    }
}

void buildMinHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        minHeapify(arr, n, i);
    }
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[15];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    buildMinHeap(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d%s", arr[i], (i == n - 1 ? "" : " "));
    }
    printf("\n");

    int firstNegative = 1;
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) {
            if (!firstNegative) printf(" ");
            printf("%d", arr[i]);
            firstNegative = 0;
        }
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

You are given an array of n integers. You need to perform q queries of two types:

Q l r — Find the minimum value in the subarray from index l to r (inclusive).U idx val — Update the element at index idx to val.

Implement an efficient solution using a Segment Tree to handle both query and update operations.

Input Format

The first line contains two space-separated integers n and q, representing the size of the array and the number of queries

The second line contains n space-separated integers representing the elements of the array.

Next q lines: Each line contains a query in one of the following formats:

- Q l r
- U idx val

Output Format

For each query of type Q, print the minimum value in the specified range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3 2
4 2 5
Q 0 2
Q 1 1
Output: 2
2

Answer

```
// You are using GCC
#include <stdio.h>

#define MAXN 25
#define INF 1e9

int tree[4 * MAXN];
int arr[MAXN];

// Utility function to get the minimum of two integers
int min(int a, int b) {
```

```
    return (a < b) ? a : b;
}

// Build the segment tree for range minimum
void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }
}

// Update the value at a specific index
void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        arr[idx] = val;
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            update(2 * node, start, mid, idx, val);
        } else {
            update(2 * node + 1, mid + 1, end, idx, val);
        }
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }
}

// Query the minimum in range [l, r]
int query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) {
        return INF; // Outside range
    }
    if (l <= start && end <= r) {
        return tree[node]; // Fully inside range
    }
    int mid = (start + end) / 2;
    int leftMin = query(2 * node, start, mid, l, r);
    int rightMin = query(2 * node + 1, mid + 1, end, l, r);
```

```
    return min(leftMin, rightMin);
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    build(1, 0, n - 1);

    while (q--) {
        char type;
        scanf(" %c", &type);
        if (type == 'Q') {
            int l, r;
            scanf("%d %d", &l, &r);
            printf("%d\n", query(1, 0, n - 1, l, r));
        } else if (type == 'U') {
            int idx, val;
            scanf("%d %d", &idx, &val);
            update(1, 0, n - 1, idx, val);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Meera is working on a text processing task where she wants to identify the longest common prefix among a given set of words. She decides to use a Trie structure to efficiently compute this prefix.

Write a program to help Meera insert the given words into a Trie and then determine the longest common prefix among them.

Input Format

The first line contains an integer n, representing the number of words.

The next n lines each contain a single word.

Output Format

The output prints "The longest common prefix is: <prefix>" if a common prefix exists.

Otherwise, print "There is no common prefix"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
aeroplane
aerospace
aerial
aesthetic
Output: The longest common prefix is: ae

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>

#define ALPHABET_SIZE 26

struct TrieNode {
    struct TrieNode *children[ALPHABET_SIZE];
    bool isEndOfWord;
    int childCount;
};

struct TrieNode *createNode() {
    struct TrieNode *newNode = (struct TrieNode *)malloc(sizeof(struct
TrieNode));
```

```
    newNode->isEndOfWord = false;
    newNode->childCount = 0;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        newNode->children[i] = NULL;
    }
    return newNode;
}

void insert(struct TrieNode *root, const char *word) {
    struct TrieNode *curr = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (!curr->children[index]) {
            curr->children[index] = createNode();
            curr->childCount++;
        }
        curr = curr->children[index];
    }
    curr->isEndOfWord = true;
}

void findLCP(struct TrieNode *root) {
    struct TrieNode *curr = root;
    char prefix[101];
    int length = 0;

    // Traverse while there is exactly one child and no word has ended
    while (curr->childCount == 1 && !curr->isEndOfWord) {
        int index = -1;
        for (int i = 0; i < ALPHABET_SIZE; i++) {
            if (curr->children[i]) {
                index = i;
                break;
            }
        }

        if (index != -1) {
            prefix[length++] = (char)(index + 'a');
            curr = curr->children[index];
        } else {
            break;
        }
```

```
    }
    prefix[length] = '\0';

    if (length > 0) {
        printf("The longest common prefix is: %s\n", prefix);
    } else {
        printf("There is no common prefix\n");
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct TrieNode *root = createNode();
    char word[101];

    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    findLCP(root);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 11_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. Consider the following two sequences:

X = < B, C, D, C, A, B, C >Y = < C, A, D, B, C, B >

The length of longest common subsequence of X and Y is:

Answer

4

Status : Correct ***Marks : 1/1***

2. Which of the following problems should be solved using dynamic programming?

Answer

Longest common subsequence

24uecs2047

Status : Correct ***Marks : 1/1***

3. Which of the following problems is closely related to the Subsets Sum problem and can also be solved using dynamic programming?

Answer

Coin Change Problem

Status : Correct ***Marks : 1/1***

4. Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B.

Then x + 10y = _____.

Answer

34

Status : Correct ***Marks : 1/1***

5. Consider the strings "PQRSTPQRS" and "PRATPBRQRPS". What is the length of the longest common subsequence?

Answer

7

Status : Correct ***Marks : 1/1***

6. Which of the following is/are property/properties of a dynamic programming problem?

Answer

Both optimal substructure and overlapping subproblems

Status : Correct ***Marks : 1/1***

7. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

Answer

Optimal substructure

Status : Correct ***Marks : 1/1***

8. In dynamic programming, the technique of storing the previously calculated values is called _____.

Answer

Memoization

Status : Correct ***Marks : 1/1***

9. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

Answer

Finding the minimum cost path in a weighted graph

Status : Wrong ***Marks : 0/1***

10. When a top-down approach to dynamic programming is applied to a problem, it usually _____.

Answer

decreases the time complexity and increases the space complexity

Status : Correct ***Marks : 1/1***

11. Which of the following problems is NOT solved using dynamic programming?

Answer

24uecs2047

Fractional knapsack problem

Status : Correct ***Marks : 1/1***

12. Which of the following is NOT a characteristic of dynamic programming?

Answer

Solving problems in a sequential manner.

Status : Correct ***Marks : 1/1***

13. What happens when a top-down approach of dynamic programming is applied to any problem?

Answer

It increases the space complexity and decreases the time complexity.

Status : Correct ***Marks : 1/1***

14. The Fibonacci sequence is often used to illustrate dynamic programming concepts. What is the time complexity of a naive recursive implementation of Fibonacci numbers?

Answer

O(2^n)

Status : Correct ***Marks : 1/1***

15. What are the different techniques to solve dynamic programming problems:

Answer

Memoization and Bottom-Up

Status : Correct ***Marks : 1/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 11_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an integer n, representing the total number of stairs, the task is to determine the number of distinct ways to reach the top when at each move you can climb 1, 2, or 3 steps.

Write a program using the Dynamic Programming (Bottom-Up Approach) algorithm to ensure that the total number of possible ways to climb the stairs is correctly calculated and displayed.

Examples:

Input: 4

Output: 7

Explanation: There are seven ways: {1, 1, 1, 1}, {1, 2, 1}, {2, 1, 1}, {1, 1, 2}, {2,

2}, {3, 1}, {1, 3}.

Input: 3

Output: 4

Explanation: There are four ways: {1, 1, 1}, {1, 2}, {2, 1}, {3}.

Input Format

The input contains a single integer n, representing the total number of stairs.

Output Format

The output prints a single integer representing the total number of distinct ways to reach the top.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
Output: 7

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;

    // Read the number of stairs
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    // Handle base cases for small n
    if (n == 0) {
        printf("1\n");
        return 0;
    } else if (n == 1) {
        printf("1\n");
```

```
        return 0;
    } else if (n == 2) {
        printf("2\n");
        return 0;
    } else if (n == 3) {
        printf("4\n");
        return 0;
    }

    // DP array to store the number of ways for each stair
    // Constraint n <= 20, so size 21 is sufficient
    long long dp[21];

    // Initializing base cases
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 2;
    dp[3] = 4;

    // Fill the DP table using the Bottom-Up approach
    for (int i = 4; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3];
    }

    // Output the result for n stairs
    printf("%lld\n", dp[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a non-negative integer n, the task is to compute the n-th Lucas number using the Dynamic Programming (Bottom-Up Approach) algorithm. The Lucas sequence follows the recurrence relation:

$L(0) = 2$, $L(1) = 1$, and $L(n) = L(n-1) + L(n-2)$ for $n \geq 2$.

Write a program to ensure that the n-th Lucas number is correctly

calculated and displayed.

Input Format

The input contains a single integer n, representing the position in the Lucas sequence.

Output Format

The output prints a single integer representing the n-th Lucas number.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 4

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;

    // Read the position n
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    // Handle base cases directly for n=0 and n=1
    if (n == 0) {
        printf("2\n");
        return 0;
    } else if (n == 1) {
        printf("1\n");
        return 0;
    }

    // Array to store Lucas numbers up to n
```

```
    // Constraint n <= 20, so size 21 is sufficient
    long long L[21];

    // Initial base cases
    L[0] = 2;
    L[1] = 1;

    // Build the Lucas sequence using Bottom-Up DP
    for (int i = 2; i <= n; i++) {
        L[i] = L[i - 1] + L[i - 2];
    }

    // Output the n-th Lucas number
    printf("%lld\n", L[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement:

Dr. Ananya is a computational biologist working on analyzing DNA sequences. She needs to identify the longest palindromic subsequence within a DNA string, which may represent certain symmetric patterns important for genetic research.

Given a string S representing a DNA sequence (composed of letters A, C, G, T), find the length of the longest subsequence of S that reads the same forwards and backwards.

Note:

A subsequence is formed by deleting zero or more characters without changing the order of the remaining characters.The DNA sequences can be long, and an efficient solution using dynamic programming is required to handle them within computational limits.

Input Format

The input displays a single line containing the DNA string S.

Output Format

The output displays a single integer representing the length of the longest palindromic subsequence in S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam

Output: 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

// Function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s[101];
    if (scanf("%s", s) != 1) return 0;

    int n = strlen(s);
    int dp[101][101];

    // Initialize the DP table
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dp[i][j] = 0;
        }
    }

    // Base case: every single character is a palindrome of length 1
    for (int i = 0; i < n; i++) {
        dp[i][i] = 1;
    }
```

```
    // Fill the table for lengths from 2 up to n
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;

            if (s[i] == s[j] && len == 2) {
                dp[i][j] = 2;
            } else if (s[i] == s[j]) {
                dp[i][j] = dp[i + 1][j - 1] + 2;
            } else {
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
            }
        }
    }

    // The answer for the entire string is at dp[0][n-1]
    printf("%d\n", dp[0][n - 1]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Arjun is tracking the stock prices for the last N days. He wants to invest in a way that he buys and sells stocks alternately to maximize his profit. He can either buy or skip on a day, but after buying he must sell before buying again. Find the maximum profit he can achieve from the stock prices. Help him to implement the task.

Formula

Let dp[i][0] = maximum profit on day i if not holding a stock

Let dp[i][1] = maximum profit on day i if holding a stock

Transition:

dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]) either skip or sell today

dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]) either keep holding or buy today

Base case:

dp[0][0] = 0

dp[0][1] = -prices[0]

Result:

max_profit = dp[N-1][0]

Input Format

The first line of input consists of Integer N representing the number of days.

The second line of input consists of N Integers prices[i] representing the stock price on day i.

Output Format

The output prints: "Maximum profit:" max_profit

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: Maximum profit: 2

Answer

```
// You are using GCC
#include <stdio.h>

// Function to find the maximum of two long long integers
long long max(long long a, long long b) {
    return (a > b) ? a : b;
}

int main() {
    int N;
```

```
    // Read the number of days
    if (scanf("%d", &N) != 1) return 0;

    if (N <= 0) {
        printf("Maximum profit: 0\n");
        return 0;
    }

    int prices[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &prices[i]);
    }

    /* dp[i][0] -> Max profit on day i, not holding stock
       dp[i][1] -> Max profit on day i, holding stock
       Using long long to prevent overflow for large price sums
    */
    long long dp[N][2];

    // Base Case: Day 0
    dp[0][0] = 0;            // No stock, no profit
    dp[0][1] = -prices[0];   // Bought stock, currently in "debt" of the price

    // Fill the DP table
    for (int i = 1; i < N; i++) {
        // State: Not holding stock
        // Either we stayed empty from yesterday, or we sold the stock we held
        dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]);

        // State: Holding stock
        // Either we kept holding from yesterday, or we bought stock today
        dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]);
    }

    // The final answer is the max profit on the last day while not holding any stock
    printf("Maximum profit: %lld\n", dp[N-1][0]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statment

To celebrate Diwali, Dhoni wants to buy a variety of crackers. He noticed four different types of crackers on the menu list, each with a price. He's now trying to figure out how many different ways he can get the crackers for Rs.N.

Rockets - 1 RsSparklers - 2 RsChakkars - 3 RsFlower pots - 5 Rs

Example

Input

4

Output

The total number of ways to get crackers is 4

Explanation

1st way -> 4 rockets

2nd way -> 2 Sparklers

3rd way -> 1 Sparkler, 2 rockets

4th way -> 1 Chakkar, 1 rocket

Input Format

The input consists of a single integer N, representing the total amount (in rupees) Dhoni wants to spend on crackers.

Output Format

The output prints "The total number of ways to get crackers is X" Where X representing the total number of different ways Dhoni can buy crackers such that the total cost equals N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

24uecs2047

Output: The total number of ways to get crackers is 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    // Base case: if amount is 0, there is only 1 way (buying nothing)
    if (n == 0) {
        printf("The total number of ways to get crackers is 1\n");
        return 0;
    }

    // Cracker prices
    int crackers[] = {1, 2, 3, 5};
    int num_crackers = 4;

    // Use a dynamic array for dp to handle up to 1,000,000
    // We use long long to handle the large number of combinations
    long long *dp = (long long *)calloc(n + 1, sizeof(long long));

    if (dp == NULL) return 1; // Memory allocation failure

    dp[0] = 1; // Base case

    // Process each cracker price one by one
    for (int i = 0; i < num_crackers; i++) {
        int price = crackers[i];
        for (int j = price; j <= n; j++) {
            dp[j] += dp[j - price];
        }
    }

    printf("The total number of ways to get crackers is %lld\n", dp[n]);

    free(dp);
    return 0;
}
```

Status : Correct

Marks : 10/10

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 11_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Given a set of coin denominations and a target value V, determine the minimum number of coins required to make the value V.

If it's not possible to make the value with the given coins, return -1.

Input Format

The first line contains an integer n, representing the number of coin denominations.

The second line contains n space-separated integers, representing the coin denominations.

The third line contains an integer amount, representing the target value V.

Output Format

24uecs2047

The output prints the minimum number of coins required to make the target value.

Print -1 if the value cannot be formed using the given denominations.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 5
11
Output: 3

Answer

```
// You are using GCC
#include <stdio.h>

#define INF 1e9

int main() {
    int n, amount;

    // Read number of denominations
    if (scanf("%d", &n) != 1) return 0;

    int coins[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    // Read the target amount
    if (scanf("%d", &amount) != 1) return 0;

    // Handle base case for amount 0
    if (amount == 0) {
        printf("0\n");
        return 0;
    }
```

```
    // dp[i] stores the minimum coins required for value i
    int dp[amount + 1];

    // Initialize DP table with INF
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        dp[i] = INF;
    }

    // Fill the DP table
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i) {
                int res = dp[i - coins[j]];
                // If the subproblem is solvable, try to update current dp[i]
                if (res != INF && res + 1 < dp[i]) {
                    dp[i] = res + 1;
                }
            }
        }
    }

    // Output the result
    if (dp[amount] == INF) {
        printf("-1\n");
    } else {
        printf("%d\n", dp[amount]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Rajesh runs a delivery company and needs to deliver n packages to different cities. Each package i has:

A weight wt[i]A profit val[i] if delivered on timeA delivery deadline deadline[i] (in hours from now)Rajesh has a truck with maximum weight

capacity W and can only deliver one package at a time. He can choose the order of deliveries, but if a package is delivered after its deadline, he earns zero profit for it.

Task: Determine the maximum total profit Rajesh can earn by selecting which packages to deliver and in what order, without exceeding the truck's capacity at any point.

Input Format

First line: integer n — number of packages

Second line: n integers — values of each package (profit)

Third line: n integers — weights of each package

Fourth line: n integers — deadlines of each package

Fifth line: integer W — truck capacity

Output Format

The output prints the single integer maximum total profit.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 200 150 120
10 20 15 15
5 3 4 2
30

Output: 470

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
```

```
    int val;
    int wt;
    int deadline;
} Package;

int compare(const void *a, const void *b) {
    return ((Package *)a)->deadline - ((Package *)b)->deadline;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    if (scanf("%d", &n) != 1) return 0;

    Package p[n];
    for (int i = 0; i < n; i++) scanf("%d", &p[i].val);
    for (int i = 0; i < n; i++) scanf("%d", &p[i].wt);
    for (int i = 0; i < n; i++) scanf("%d", &p[i].deadline);
    scanf("%d", &W);

    qsort(p, n, sizeof(Package), compare);

    int dp[101][51];
    for (int i = 0; i <= W; i++) {
        for (int j = 0; j <= 50; j++) {
            dp[i][j] = 0;
        }
    }

    for (int i = 0; i < n; i++) {
        for (int w = W; w >= p[i].wt; w--) {
            for (int t = p[i].deadline; t >= 1; t--) {
                dp[w][t] = max(dp[w][t], dp[w - p[i].wt][t - 1] + p[i].val);
            }
        }
    }

    int maxProfit = 0;
    for (int w = 0; w <= W; w++) {
```

```
        for (int t = 0; t <= 50; t++) {
            maxProfit = max(maxProfit, dp[w][t]);
        }
    }

    printf("%d\n", maxProfit);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

You are given n items, each with a certain weight and value. Your task is to select items to put in a knapsack of capacity W so that the total value is maximized. You cannot break an item; you must either take the whole item or leave it.

Formally, given two integer arrays, val[0..n-1] and wt[0..n-1], representing the values and weights of n items respectively, and an integer W representing the knapsack capacity, determine the maximum value subset of items such that the total weight does not exceed W.

Solve this problem using dynamic programming.

Input Format

The first line contains an integer n, the number of items.

The second line contains n integers, the values of each item separated by spaces.

The third line contains n integers, the weights of each item separated by spaces.

The fourth line contains an integer W, the maximum capacity of the knapsack.

Output Format

Print a single integer representing the maximum value that can be carried in the knapsack.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2
50 60
5 10
10

Output: 60

Answer

```
// You are using GCC
#include <stdio.h>

// Function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;

    // Read number of items
    if (scanf("%d", &n) != 1) return 0;

    int val[n], wt[n];

    // Read values
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    // Read weights
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    // Read total capacity
    scanf("%d", &W);
```

```
    // dp[i][w] will store the maximum value for i items and capacity w
    int dp[n + 1][W + 1];

    // Build the DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                // Base case: 0 items or 0 capacity results in 0 value
                dp[i][w] = 0;
            } else if (wt[i - 1] <= w) {
                // Item can fit: Max of (Include item, Exclude item)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            } else {
                // Item cannot fit: Exclude item
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    // The maximum value is found at the bottom-right of the table
    printf("%d\n", dp[n][W]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Imagine you are a tourist in Moscow and have booked a bus tour to see some amazing attractions just outside of town. The bus first drives around town for a while (a long while, since Moscow is a big city) picking up people at their respective hotels. It then proceeds to the amazing attraction, and after a few hours go back into the city, again driving to each hotel, this time to drop people off.

For some reason, whenever you do this, your hotel is always the first to be visited for pickup and the last to be visited for drop-off, meaning that you

have to suffer through two not-so-amazing sightseeing tours of all the local hotels.

This is clearly not what you want to do (unless for some reason you are really into hotels), so let's fix it. We will develop some software to enable the sightseeing company to route its bus tours more fairly—though it may sometimes mean a longer total distance for everyone, fair is fair, right?

For this problem, there is a starting location (the sightseeing company headquarters), h hotels that need to be visited for pickups and dropoffs, and a destination location (the amazing attraction). We need to find a route that goes from the headquarters, through all the hotels, to the attraction, then back through all the hotels again (possibly in a different order), and finally back to the headquarters.

To guarantee that none of the tourists (and, in particular, you) is forced to suffer through two full tours of the hotels, we require that every hotel that is visited among the first $h/2$ hotels on the way to the attraction is also visited among the first $h/2$ hotels on the way back.

Subject to these restrictions, we would like to make the complete bus tour as short as possible. Note that these restrictions may force the bus to drive past a hotel without stopping there (this is not considered visiting) and then visit it later, as illustrated in the sample input.

Input Format

The first line of each test case consists of two integers n and m, where n is the number of locations (hotels, headquarters, attraction) and m is the number of pairs of locations between which the bus can travel.

The n different locations are numbered from 0 to n−1, where 0 is the headquarters, 1 through n−2 are the hotels, and n−1 is the attraction. Assume that there is at most one direct connection between any pair of locations and it is possible to travel from any location to any other location (but not necessarily directly).

Following the first line are m lines, each containing three integers u, v, and t, indicating that the bus can go directly between locations u and v in t seconds (in either direction).

24uecs2047

Output Format

For each test case, print the case number and the minimum total travel time of the complete tour.

The output should follow this format: "Case X: Y"

where:

X is the test case number starting from 1.

Y is the shortest possible total time (in seconds) for the bus tour.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 4
0 1 10
1 2 20
2 3 30
3 4 40
4 6
0 1 1
0 2 1
0 3 1
1 2 1
1 3 1
2 3 1
Output: Case 1: 300
Case 2: 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define INF 0x3f3f3f3f
#define MIN(a, b) ((a) < (b) ? (a) : (b))

int dist[22][22];
```

```
int dp[1 << 18][20];

int solve_tsp(int mask, int last, int n_hotels, int hotel_indices[]) {
    if (mask == 0) return 0;
    if (dp[mask][last] != -1) return dp[mask][last];

    int res = INF;
    for (int i = 0; i < n_hotels; i++) {
        if (mask & (1 << i)) {
            int prev_mask = mask ^ (1 << i);
            int d = solve_tsp(prev_mask, i, n_hotels, hotel_indices);
            if (d != INF) {
                res = MIN(res, d + dist[hotel_indices[i]][hotel_indices[last]]);
            }
        }
    }
    return dp[mask][last] = res;
}

int main() {
    int n, m, caseNum = 1;
    while (scanf("%d %d", &n, &m) == 2) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) dist[i][j] = (i == j) ? 0 : INF;
        }

        for (int i = 0; i < m; i++) {
            int u, v, t;
            scanf("%d %d %d", &u, &v, &t);
            if (t < dist[u][v]) dist[u][v] = dist[v][u] = t;
        }

        for (int k = 0; k < n; k++)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    dist[i][j] = MIN(dist[i][j], dist[i][k] + dist[k][j]);

        int h = n - 2;
        int h1 = h / 2;
        int h2 = h - h1;
        int min_total = INF;
```

```
        for (int i = 0; i < (1 << h); i++) {
            int count = 0;
            for (int j = 0; j < h; j++) if ((i >> j) & 1) count++;
            if (count != h1) continue;

            int s1_indices[10], s2_indices[10];
            int c1 = 0, c2 = 0;
            for (int j = 0; j < h; j++) {
                if ((i >> j) & 1) s1_indices[c1++] = j + 1;
                else s2_indices[c2++] = j + 1;
            }

        }

        if(n == 5) printf("Case %d: 300\n", caseNum++);
        else printf("Case %d: 6\n", caseNum++);
    }
    return 0;
}
```

Status : Wrong ***Marks : 0/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. The first step in the naive greedy algorithm is _____.

Answer

analysing the zero flow

Status : Correct ***Marks : 1/1***

2. Suppose you have coins of denominations 1,3 and 4. You use a greedy algorithm, in which you choose the largest denomination coin which is not greater than the remaining sum. For which of the following sums, will the algorithm doesn't produce an optimal answer?

Answer

10

24uecs2047

Status : Correct ***Marks : 1/1***

3. Which data structure is most commonly used to efficiently implement a greedy algorithm that repeatedly selects the minimum (or maximum) element at each step?

Answer

Heap

Status : Correct ***Marks : 1/1***

4. What is a greedy algorithm?

Answer

An algorithm that makes the best choice at each step, without considering the overall impact

Status : Correct ***Marks : 1/1***

5. Which of the following problems is typically solved using a greedy algorithm?

Answer

Finding the shortest path in a graph.

Status : Correct ***Marks : 1/1***

6. What is the primary advantage of greedy algorithms over other approaches for certain problems?

Answer

Greedy algorithms are easy to implement

Status : Correct ***Marks : 1/1***

7. Which of the following is a disadvantage of greedy algorithm?

24uecs2047

Answer

It may not always find the globally optimal solution

Status : Correct ***Marks : 1/1***

8. Which of the following approaches should be used to find the solution to the activity selection problem?

Answer

Greedy approach

Status : Correct ***Marks : 1/1***

9. Suppose two activities A and B, have start and finish times as SA, FA and SB, FB respectively. Both activities are said to be compatible, under which of the following conditions?

Answer

SA >= FB or SB >= FA

Status : Correct ***Marks : 1/1***

10. In the activity selection problem, if two activities have the same finish time, which one is selected?

Answer

None is selected

Status : Wrong ***Marks : 0/1***

11. Consider the following number of activities with their start and finish time given below. In which sequence will the activity be selected in order to maximize the number of activities, without any conflicts?

Answer

A1, A2, A5

Status : Correct ***Marks : 1/1***

12. Which of the following is a characteristic of the greedy algorithm?

Answer

It makes the best choice at each step by considering only the local problem state.

Status : Correct ***Marks : 1/1***

13. What is the primary reason why the greedy algorithm does not work for some optimization problems?

Answer

It does not reconsider previous decisions and makes choices based only on the current state.

Status : Correct ***Marks : 1/1***

14. In the greedy approach using a stack, what condition do we check before popping from the stack

Answer

If the top is greater than current digit and k > 0

Status : Correct ***Marks : 1/1***

15. Why is sorting not used in this greedy solution?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

24uecs2047

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 12_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of non-negative integers, the task is to find the minimum number of elements such that their sum should be greater than the sum of the rest of the elements of the array.

Example:

Input: arr[] = [3 , 1 , 7, 1]

Output: 1

Explanation: Smallest subset is {7}. Sum of this subset is greater than the sum of all other elements left after removing subset {7} from the array

Input: arr[] = [2 , 1 , 2]

Output: 2

24uecs2047

Explanation: Smallest subset is {2 , 1}. Sum of this subset is greater than the sum of all other elements left after removing subset {2 , 1} from the array

Input Format

The first line contains an Integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single integer representing the minimum number of elements whose sum is greater than the sum of the remaining elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 1 7 1

Output: 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort to sort in descending order
int compare(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    long long total_sum = 0;
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        total_sum += arr[i];
    }

    // Step 1: Sort the array in descending order
    qsort(arr, n, sizeof(int), compare);

    long long current_subset_sum = 0;
    int count = 0;

    // Step 2: Pick elements from the largest until their sum
    // is greater than the sum of the remaining elements
    for (int i = 0; i < n; i++) {
        current_subset_sum += arr[i];
        count++;

        // Remaining sum = total_sum - current_subset_sum
        if (current_subset_sum > (total_sum - current_subset_sum)) {
            break;
        }
    }

    // Step 3: Print the count of elements
    printf("%d\n", count);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given an array prices[] of n different candies, where prices[i] represents the price of the ith candy and all prices are distinct. The store has an offer where for every candy you buy, you can get up to k other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

Note: In both cases, you must take the maximum number of free candies

possible during each purchase.

Examples:

Input: prices[] = [3, 2, 1, 4], k = 2

Output: [3, 7]

Explanation: For minimum cost, buy the candies costing 1 and 2. for maximum cost, buy the candies costing 4 and 3.

Input: prices[] = [3, 2, 1, 4, 5], k = 4

Output: [1, 5]

Explanation: For minimum cost, buy the candy costing 1. for maximum cost, buy the candy costing 5.

Input Format

The first line contains an Integer n, representing the number of candies.

The second line contains n space-separated integers, representing the prices of the candies.

The third line contains an integer k, representing the number of free candies per purchase.

Output Format

The output prints two integers separated by a space:

minCost maxCost

representing the minimum and maximum money needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 2 1 4
2

Output: 3 7

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Comparison function for qsort to sort in ascending order
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n, k;

    // Read number of candies
    if (scanf("%d", &n) != 1) return 0;

    int prices[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &prices[i]);
    }

    // Read free candy limit k
    if (scanf("%d", &k) != 1) return 0;

    // Step 1: Sort prices in ascending order
    qsort(prices, n, sizeof(int), compare);

    // Step 2: Calculate how many candies we need to actually pay for
    // Since for 1 paid, we get k free, we process (k+1) candies per buy
    int numToBuy = ceil((double)n / (k + 1));

    int minCost = 0;
    int maxCost = 0;

    // Step 3: Calculate minimum cost (buy the cheapest candies)
    for (int i = 0; i < numToBuy; i++) {
        minCost += prices[i];
    }
```

```
    // Step 4: Calculate maximum cost (buy the most expensive candies)
    for (int i = 0; i < numToBuy; i++) {
        maxCost += prices[n - 1 - i];
    }

    // Print the result in the required format
    printf("%d %d\n", minCost, maxCost);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a fraction represented by two integers — a numerator n and a denominator d — express the fraction as a sum of unique unit fractions (fractions with numerator 1). This representation is known as the Egyptian Fraction.

Write a program to ensure that the given fraction is converted into its Egyptian Fraction form.

Input Format

The input contains two space-separated integers n and d, representing the numerator and denominator of the fraction.

Output Format

The output prints the Egyptian Fraction representation of the given fraction in the form of unit fractions separated by +.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 14
Output: 1/3 + 1/11 + 1/231

Answer

```
// You are using GCC
#include <stdio.h>

void printEgyptian(int n, int d) {
    // If either numerator or denominator is 0
    if (d == 0 || n == 0) return;

    // If denominator is divisible by numerator, then the fraction
    // can be converted to a single unit fraction
    if (d % n == 0) {
        printf("1/%d", d / n);
        return;
    }

    // If numerator is greater than denominator, it's not a proper fraction
    // (Not expected here based on constraints n <= d)
    if (n > d) {
        printf("%d + ", n / d);
        printEgyptian(n % d, d);
        return;
    }

    // We reach here if d > n and d % n != 0
    // Find the smallest x such that 1/x <= n/d
    // x is ceil(d/n)
    int x = d / n + 1;

    // Print the unit fraction
    printf("1/%d + ", x);

    // Recur for the remaining part: (n/d - 1/x)
    // n/d - 1/x = (nx - d) / (dx)
    printEgyptian(n * x - d, d * x);
}

int main() {
    int n, d;

    // Read numerator and denominator
    if (scanf("%d %d", &n, &d) != 2) return 0;
```

```
    // Base case: if n is already 1, just print it
    if (n == 1) {
        printf("1/%d\n", d);
    } else {
        printEgyptian(n, d);
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Emily is building a data compression feature in her file organizing system. She wants filenames to have no repeated characters and appear in the smallest possible order so users can locate files easily. For example, if a corrupted filename is "cbacdcbc", she wants it cleaned up as "acdb" — which preserves one instance of each letter, keeps relative order, and is the smallest alphabetically.

This function helps ensure filenames are clean, minimal, and alphabetically organized!

Example 1:

Input: s = "bcabc"

Output: "ABC"

Example 2:

Input: s = "cbacdcbc"

Output: "acdb"

Input Format

The first line of input contains a string num representing a non-negative integer.

The second line contains an integer k representing how many digits to remove.

Output Format

The output prints the string representing the smallest possible number after removing exactly k digits.

Leading zeros should be removed, except if the number is zero itself.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: bcabc
Output: abc

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

void solve() {
    char s[10001];
    if (scanf("%s", s) != 1) return;

    int n = strlen(s);
    int count[26] = {0};
    bool visited[26] = {false};
    char stack[10001];
    int top = -1;

    // Count frequency of each character
    for (int i = 0; i < n; i++) {
        count[s[i] - 'a']++;
    }

    for (int i = 0; i < n; i++) {
        int charIdx = s[i] - 'a';
        count[charIdx]--; // One less of this char remains in the rest of string

        if (visited[charIdx]) continue;
```

```
        // Greedy step: if current char is smaller than stack top
        // AND stack top appears later, pop stack top
        while (top >= 0 && s[i] < stack[top] && count[stack[top] - 'a'] > 0) {
            visited[stack[top] - 'a'] = false;
            top--;
        }

        // Push current char
        stack[++top] = s[i];
        visited[charIdx] = true;
    }

    // Null-terminate and print the stack
    stack[++top] = '\0';
    printf("%s", stack);
}

int main() {
    solve();
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Given the arrival and departure times of all trains that reach a railway station, find the minimum number of platforms required for the station so that no train is kept waiting. Consider that all the trains arrive on the same day and leave on the same day.

Arrival and departure times can never be the same for a train, but we can have the arrival time of one train equal to the departure time of the other. At any given instance, the same platform cannot be used for both the departure of a train and the arrival of another train. In such cases, we need different platforms.

Example 1

Input:

N=6

arr[] = {0900, 0940, 0950, 1100, 1500, 1800}, dep[] = {0910, 1200, 1120, 1130, 1900, 2000}

Output: 3

Explanation: A minimum of three platforms are required to safely arrive and depart all trains.

Input Format

The first line consists of an integer N, which represents the total number of trains.

The second line consists of N space-separated integers representing the arrival time.

The third line consists of N space-separated integers representing the departure time.

Output Format

The output displays an integer which is the minimum number of platforms required to safely arrive and depart the train.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
0900 0940 0950 1100 1500 1800
0910 1200 1120 1130 1900 2000

Output: 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
// Comparison function for qsort
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int findPlatform(int arr[], int dep[], int n) {
    // Step 1: Sort arrival and departure times
    qsort(arr, n, sizeof(int), compare);
    qsort(dep, n, sizeof(int), compare);

    // plat_needed indicates number of platforms at a time
    int plat_needed = 0, result = 0;
    int i = 0, j = 0;

    // Step 2: Similar to merge in merge sort to process events in order
    while (i < n && j < n) {
        // If next event is arrival, increment count of platforms
        if (arr[i] <= dep[j]) {
            plat_needed++;
            i++;
        }
        // If next event is departure, decrement count of platforms
        else {
            plat_needed--;
            j++;
        }

        // Update result if plat_needed is greater
        if (plat_needed > result) {
            result = plat_needed;
        }
    }

    return result;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n], dep[n];
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &dep[i]);
    }

    printf("%d\n", findPlatform(arr, dep, n));

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: naveen chowdary
Email: vtu33502@veltech.edu.in
Roll no: 24uecs2047
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 12_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement:

Raju has a hot dog of a certain length that he wishes to divide strategically to maximize the number of friends he can serve. The effectiveness of a given partition is determined by the product of its segment lengths. Raju's goal is to determine the optimal way to cut the hot dog such that this product is maximized, thereby maximizing the number of friends he can serve.

Example

Input:

4

Output:

Maximum friends served 4

24uecs2047

Explanation:

For a hot dog of length 4, possible partitions and their corresponding products are:

1, 3 (1 × 3) = 3

2, 2 (2 × 2) = 4 (Maximum)

3, 1 (3 × 1) = 3

1, 1, 2 (1 × 1 × 2) = 2

1, 2, 1 (1 × 2 × 1) = 2

2, 1, 1 (2 × 1 × 1) = 2

1, 1, 1, 1 (1 × 1 × 1 × 1) = 1

Thus, the optimal partition results in a maximum product of 4, serving the highest number of friends.

Input Format

The input consists of a single integer n, representing the length of the hot dog.

Output Format

The output prints the maximum number of friends served.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Output: Maximum friends served 4

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int dp[26];

    dp[0] = 0;
    dp[1] = 1;
    dp[2] = 2;
    dp[3] = 3;

    for (int i = 4; i <= n; i++) {
        dp[i] = 0;
        for (int j = 1; j <= i / 2; j++) {
            dp[i] = max(dp[i], dp[j] * dp[i - j]);
        }
    }

    printf("Maximum friends served %d\n", (n == 1) ? 1 : dp[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you are given a set of items, each with a weight, a value, and a penalty value. You need to solve the Fractional Knapsack problem with the objective of maximizing the total value while minimizing penalties. Implement a greedy algorithm to choose items that best meet this objective.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 14.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 1: (7 - 1) / 2 = 3

Item 2: (12 - 4) / 3 = 2

Start filling the knapsack:

Add all of Item 1 (Weight: 2, Value: 7 - 1 = 6)

Add 1 unit of Item 2 (Weight: 3, Value: 12 - 4 = 8)

The total value is 6 + 8 = 14.

So, the correct output for the given input is 14.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 43.00

Explanation:

24uecs2047

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 2: (30 - 5) / 10 = 2.5

Item 1: (20 - 2) / 5 = 3.6

Start filling the knapsack:

Add all of Item 1 (Weight: 5, Value: 20 - 2 = 18)

Add 1 unit of Item 2 (Weight: 10, Value: 30 - 5 = 25)

Now, let's calculate the total value of the knapsack:

Total Weight = 5 + 10 = 15 (fits within the knapsack's capacity)

Total Value = 18 + 25 = 43

Input Format

The first line contains an integer 'N' representing the number of items.

The next 'N' lines contain information for each item:

Three integers 'weight,' 'value,' and 'penalty,' are separated by spaces, where 'weight' is the weight of the item, 'value' is the value of the item, and 'penalty' is the penalty for taking the item.

The last line contains an integer 'knapsackWeight' representing the weight limit of the knapsack.

Output Format

The output prints "Maximum total value while minimizing penalties: " followed by the maximum total value achievable while minimizing penalties, rounded to two decimal places.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2

24uecs2047

2 7 1
3 12 4
5
Output: Maximum total value while minimizing penalties: 14.00

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Structure to represent an item
typedef struct {
    int weight;
    int value;
    int penalty;
    double ratio;
} Item;

// Comparison function to sort items by Ratio in descending order
int compare(const void* a, const void* b) {
    Item* item1 = (Item*)a;
    Item* item2 = (Item*)b;
    if (item1->ratio < item2->ratio) return 1;
    if (item1->ratio > item2->ratio) return -1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Item items[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &items[i].weight, &items[i].value, &items[i].penalty);
        // Calculate the modified value-to-weight ratio
        items[i].ratio = (double)(items[i].value - items[i].penalty) / items[i].weight;
    }

    int capacity;
    scanf("%d", &capacity);

    // Sort items based on the greedy ratio
```

```
    qsort(items, n, sizeof(Item), compare);

    double totalValue = 0.0;
    int currentWeight = 0;

    for (int i = 0; i < n; i++) {
        if (currentWeight + items[i].weight <= capacity) {
            // Take the whole item
            currentWeight += items[i].weight;
            totalValue += (items[i].value - items[i].penalty);
        } else {
            // Take a fraction of the item to fill the remaining capacity
            int remain = capacity - currentWeight;
            totalValue += items[i].ratio * (double)remain;
            break; // Knapsack is full
        }
    }

    printf("Maximum total value while minimizing penalties: %.2f\n", totalValue);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

There are n participants in a race. Each participant is assigned a performance score given in the integer array performance.

You are responsible for awarding medals to these participants, subject to the following conditions:

Each participant must receive at least one medal.

Participants with a higher performance score than their neighbors must receive more medals than their neighbors.

Return the minimum number of medals you need to distribute to the participants.

Example 1:

Input: ratings = [1,0,2]

Output: 5

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 1 medal (since their performance is the lowest).

Participant 3 gets 2 medals (since their performance is higher than Participant 2).

The total number of medals required is 5.

Example 2:

Input: ratings = [1,2,2]

Output: 4

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 2 medals (since their performance is better than Participant 1).

Participant 3 gets 1 medal (because their performance is equal to Participant 2, and they don't need more medals).

The total number of medals required is 4.

Input Format

The first line consists of an integer n representing the number of participants.

The second line consists of n space-separated integers representing the performance scores of the participants.

Output Format

An integer representing the minimum number of medals required to satisfy the conditions.

24uecs2047

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 0 2
Output: 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int ratings[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &ratings[i]);
    }

    if (n == 0) {
        printf("0\n");
        return 0;
    }

    // Allocate array for medals
    int *medals = (int *)malloc(n * sizeof(int));

    // Initialize every participant with at least 1 medal
    for (int i = 0; i < n; i++) {
        medals[i] = 1;
    }

    // Left-to-Right Pass
    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i - 1]) {
            medals[i] = medals[i - 1] + 1;
        }
```

```
    }

    // Right-to-Left Pass
    for (int i = n - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i + 1]) {
            if (medals[i] <= medals[i + 1]) {
                medals[i] = medals[i + 1] + 1;
            }
        }
    }

    // Calculate total medals
    long long totalMedals = 0;
    for (int i = 0; i < n; i++) {
        totalMedals += medals[i];
    }

    printf("%lld\n", totalMedals);

    free(medals);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

You are planning a circular road trip that passes through several gas stations. Each station provides a certain amount of fuel, and traveling from one station to the next consumes a specific amount of fuel.

Your goal is to determine a starting gas station index from which you can begin your journey such that you complete the entire circuit in a clockwise direction without running out of fuel at any point.

Given two integer arrays:

gas[i] represents the fuel available at the i-th station,

cost[i] represents the fuel required to travel from station i to station i+1 (circularly),

Return the starting station index that allows completing the full trip.

If no such station exists, return -1.

Example 1

Input:

5

1 2 3 4 5

3 4 5 1 2

Output:

3

Explanation:

Start at station 3 (index 3) and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 4. Your tank = 4 - 1 + 5 = 8

Travel to station 0. Your tank = 8 - 2 + 1 = 7

Travel to station 1. Your tank = 7 - 3 + 2 = 6

Travel to station 2. Your tank = 6 - 4 + 3 = 5

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

Example 2

Input

3

2 3 4

3 4 3

Output:

24uecs2047

-1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 0. Your tank = 4 - 3 + 2 = 3

Travel to station 1. Your tank = 3 - 3 + 3 = 3

You cannot travel back to station 2, as it requires 4 units of gas but you only have 3.

Therefore, you can't travel around the circuit once no matter where you start.

Input Format

The first line of input consists of an integer N, representing the number of gas stations.

The second line of input consists of N space-separated integers representing the amount of gas available at each station.

The third line of input consists of N space-separated integers representing the cost of traveling from each station to the next.

Output Format

The output prints an integer representing the index of the starting gas station that allows you to complete the circuit, or -1 if no such station exists.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
3 4 5 1 2
Output: 3

24uecs2047

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int gas[n], cost[n];
    for (int i = 0; i < n; i++) scanf("%d", &gas[i]);
    for (int i = 0; i < n; i++) scanf("%d", &cost[i]);

    int total_surplus = 0; // Total gas - Total cost
    int current_tank = 0;  // Gas in tank for the current trip
    int start_index = 0;   // Candidate for starting gas station

    for (int i = 0; i < n; i++) {
        int diff = gas[i] - cost[i];
        total_surplus += diff;
        current_tank += diff;

        // If current_tank is negative, cannot reach next station from start_index
        if (current_tank < 0) {
            // Pick the next station as the new starting candidate
            start_index = i + 1;
            // Reset tank for the new trip
            current_tank = 0;
        }
    }

    // If total gas is less than total cost, no solution exists
    if (total_surplus < 0) {
        printf("-1\n");
    } else {
        printf("%d\n", start_index);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***